NPTEL ONLINE CERTIFICATION

Deep Learning

Course Code: CS7015 • IIT Madras

Complete Exam Preparation Guide, Theoretical Foundations & Assignment Solutions

- **Comprehensive Theory**
  Weeks 1 to 12 de-duplicated from 4,500+ lecture slides
- **240 Solved Problems**
  All weekly assignment MCQs with step-by-step math
- **Formula Cheat Sheets**
  Every architecture, update equation & gradient derivation
- **High-Yield Exam Traps**
  Frequent pitfalls, edge cases, and exam-tested tricks

Instructor: Prof. Mitesh M. Khapra (Dept. of CSE, IIT Madras)

Curated Study Handbook: Full 12-Week Syllabus

Table of Contents

Week 01	Week 1: History of DL, McCulloch-Pitts Neuron, Perceptrons, and Representation Power
Week 02	Week 2: Sigmoid Neurons, Gradient Descent, Feedforward Networks, and Universal Approximation
Week 03	Week 3: Feedforward Neural Networks and the Backpropagation Algorithm
Week 04	Week 4: Optimization Algorithms — GD, Momentum, NAG, AdaGrad, RMSProp, and Adam
Week 05	Week 5: Linear Algebra Foundations — Eigenvalues, Eigenvectors, PCA, and SVD
Week 06	Week 6: Autoencoders, Relation to PCA, and Regularized Architectures
Week 07	Week 7: Regularization Techniques — Bias-Variance, L2/L1, Early Stopping, and Dropout
Week 08	Week 8: Training Deep Networks — Greedy Pre-training, Activations, Initialization, and BN
Week 09	Week 9: Learning Vectorial Representations of Words — Word Embeddings and Word2Vec
Week 10	Week 10: Convolutional Neural Networks (CNNs), Classic Architectures, and Visualization
Week 11	Week 11: Recurrent Neural Networks (RNNs), BPTT, LSTMs, and GRUs
Week 12	Week 12: Encoder-Decoder Models, Attention Mechanisms, and Transformers

MODULE 1

Week 1: History of DL, McCulloch-Pitts Neuron, Perceptrons, and Representation Power

Week 1: History of Deep Learning, McCulloch-Pitts Neuron, Perceptrons, and Representation Power

1. Executive Summary & Exam Roadmap

Week 1 lays the mathematical and historical foundations of modern Deep Learning. In NPTEL CS7015, Week 1 covers: 1. **History of Artificial Neural Networks**: Milestones from biological foundations to modern revival. 2. **McCulloch-Pitts (MP) Neuron (1943)**: Threshold-based binary logic unit, excitatory vs. inhibitory inputs, geometric interpretation, representation capacity. 3. **Rosenblatt's Perceptron (1958)**: Real-valued inputs, real-valued weights, explicit bias / threshold, linear hyperplanes. 4. **Perceptron Learning Algorithm (PLA)**: Iterative weight update rule, geometric intuition, Novikoff's convergence theorem and formal proof. 5. **Linear Separability & Multilayer Perceptrons (MLPs)**: Limitations of single-layer models (the XOR problem), Minsky & Papert's critique, representation power of 2-layer and 3-layer networks.

2. Comprehensive Lecture Notes & Mathematical Foundations

2.1 Historical Timeline & Key Milestones (Lecture 1)

- **1943 - McCulloch & Pitts**: Introduced the first computational abstraction of a biological neuron (threshold logic unit).
- **1949 - Donald Hebb**: Hebbian Learning rule ("neurons that fire together, wire together").
- **1958 - Frank Rosenblatt**: Developed the Perceptron with learnable continuous weights.
- **1969 - Marvin Minsky & Seymour Papert**: Published *Perceptrons*, proving that single-layer perceptrons cannot compute non-linearly separable functions like XOR. This triggered the first **"AI Winter"**.
- **1986 - Rumelhart, Hinton, & Williams**: Popularized the Backpropagation algorithm for training multi-layer neural networks, sparking the connectionist revival.
- **1989 - Cybenko / 1991 - Hornik**: Proved the Universal Approximation Theorem.
- **1990s**: Rise of Support Vector Machines (SVMs) and kernel methods; neural networks fell out of favor due to vanishing gradients, slow training, and overfitting.
- **2006 - Hinton, Salakhutdinov, Bengio**: Deep Belief Networks and Greedy Layer-wise Pre-training revitalized deep architectures (birth of "Deep Learning").
- **2012 - Krizhevsky, Sutskever, & Hinton (AlexNet)**: Outperformed classical computer vision in ImageNet 2012 by an unprecedented margin using GPUs, ReLU activations, and Dropout.

2.2 Biological Neuron vs. Artificial Neuron

A biological neuron consists of: - **Dendrites**: Receive incoming electrochemical signals (inputs x_i). - **Soma (Cell Body)**: Accumulates and processes the electrical potentials. - **Axon**: Transmits the action potential (output signal y) to subsequent neurons. - **Synapses**: Junction points where signal strength is modulated (weights w_i).

```
Biological:  Dendrites (Inputs) ---> Soma (Summation & Threshold) ---> Axon (Output)
Artificial:  x_1, x_2, ..., x_n ---> Sum(w_i * x_i) + b ---> Activation f(z) ---> y
```

2.3 McCulloch-Pitts (MP) Neuron (1943)

Definition and Mathematical Model

- **Inputs**: Binary vector $\mathbf{x} = [x_1, x_2, \ldots, x_n]^\top$, where $x_i \in 0, 1$.
- **Weights**: All inputs have equal, unit weight ($w_i = 1$).
- **Input Types**:
- **Excitatory inputs**: Add to the net potential.
- **Inhibitory inputs**: If any inhibitory input is active (1), the neuron is completely suppressed and output is forcefully 0.
- **Aggregation & Firing Rule**: Assuming no active inhibitory inputs:

$$g(x_1, x_2, \ldots, x_n) = \sum_{i=1}^{n} x_i$$

$$y = f(g(\mathbf{x})) = \{1 \quad \text{if } \sum_{i=1}^{n} x_i \geq \theta \, 0 \quad \text{if } \sum_{i=1}^{n} x_i < \theta$$

where $\theta \in \mathbb{Z}^+$ is a predefined integer threshold.

Boolean Functions Implementable by MP Neuron

1. **AND Function (n inputs)**:
2. Output is 1 if and only if all n inputs are 1.
3. Aggregation: $\sum_{i=1}^{n} x_i \geq n \implies \theta = \mathbf{n}$.
4. **OR Function (n inputs)**:
5. Output is 1 if at least one input is 1.
6. Aggregation: $\sum_{i=1}^{n} x_i \geq 1 \implies \theta = \mathbf{1}$.
7. **NOT Function (1 input)**:
8. Requires an inhibitory input. Let x_1 be inhibitory: if $x_1 = 1$, output is 0; if $x_1 = 0$, output fires with $\theta = 0$.
9. **NAND / NOR Functions**:
10. Implemented via inhibitory inputs or threshold logic combinations.

Geometric Interpretation & Limitations of MP Neuron

- The equation $\sum_{i=1}^{n} x_i = \theta$ represents a hyperplane in n-dimensional space with normal vector $\mathbf{w} = [1, 1, \ldots, 1]^\top$.
- **Severe limitation**: The orientation of the separating hyperplane is strictly fixed at 45° (all weights must be 1). It can only be translated by changing the threshold θ. It cannot rotate to fit general linear boundaries, nor can it assign different importances to different inputs.

2.4 Frank Rosenblatt's Perceptron (1958)

Mathematical Formulation

Rosenblatt generalized the MP neuron by allowing continuous real-valued inputs and learnable real-valued weights: - **Inputs**: $\mathbf{x} = [x_1, x_2, \ldots, x_n]^\top \in \mathbb{R}^n$. - **Weights**: $\mathbf{w} = [w_1, w_2, \ldots, w_n]^\top \in \mathbb{R}^n$. - **Bias**: $b \in \mathbb{R}$ (often written as w_0 with dummy input $x_0 = 1$, or as $-\theta$). - **Net Input (Logit / Score)**:

$$z = \mathbf{w}^\top \mathbf{x} + b = \sum_{i=1}^{n} w_i x_i + b$$

- **Activation Function (Signum / Step Function)**: - **0/1 formulation**:

$$y = \{1 \quad \text{if } \mathbf{w}^\top \mathbf{x} + b \geq 0\ 0 \quad \text{otherwise}$$

- **-1/+1 formulation**:

$$y = \{+1 \quad \text{if } \mathbf{w}^\top \mathbf{x} + b \geq 0\ -1 \quad \text{otherwise}$$

Geometric Interpretation

- The decision boundary is the hyperplane given by:

$$\mathbf{w}^\top \mathbf{x} + b = 0$$

- The vector $\mathbf{w}$ is **orthogonal (perpendicular)** to the decision hyperplane and points in the direction of the positive half-space ($\mathbf{w}^\top \mathbf{x} + b > 0$).
- The perpendicular distance of any point $\mathbf{x}$ to the hyperplane is:

$$d = \frac{|\mathbf{w}^\top \mathbf{x} + b|}{|\mathbf{w}|_2}$$

2.5 Perceptron Learning Algorithm (PLA)

The Update Rule

For training data ${(\mathbf{x}*i, y_i)}* = [1, x_1, \dots, x_n]^\top) : $-Condition for misclassification : y_i(\mathbf{w}^\top \mathbf{x}_i) \leq 0 - When misclassified, update : \mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} + \eta\, y_i \mathbf{x}_i where$ \eta > 0 is the learning rate (typically \eta = 1Extra close brace or missing open brace with labels $y_i \in -1, +1$ and no explicit bias (or absorbed into augmented $\mathbf{x

Intuition Behind the Update

- If $y_i = +1$ and $\mathbf{w}^\top \mathbf{x}*i \le 0 (false negative) :$\mathbf{w}} = \mathbf{w}_{\text{old}} + \mathbf{x}*i \implies \mathbf{w}}^\top \mathbf{x}*i = \mathbf{w}}^\top \mathbf{x}*i + \|\mathbf{x}_i\|^2 > \mathbf{w}_{\text{old}}^\top \mathbf{x}_i$$ The angle between $\mathbf{w}$ and $\mathbf{x}_i$ decreases, pulling the boundary toward the correct side.
- If $y_i = -1$ and $\mathbf{w}^\top \mathbf{x}*i \ge 0 (false positive) :$\mathbf{w}} = \mathbf{w}_{\text{old}} - \mathbf{x}*i \implies \mathbf{w}}^\top \mathbf{x}*i = \mathbf{w}}^\top \mathbf{x}*i - \|\mathbf{x}_i\|^2 < \mathbf{w}_{\text{old}}^\top \mathbf{x}_i$$ The angle increases, pushing the hyperplane away.

2.6 Novikoff's Perceptron Convergence Theorem & Formal Proof

Theorem Statement

Let $\mathcal{D} = {(\mathbf{x}*i, y_i)}^N be a linearly separable dataset with y_i \in {-1, +1}. Assume : 1. All inputs are bounded : \|\mathbf{x}_i\| \le R for all i . 2. There exists an optimal unit weight vector \mathbf{w}^(\|\mathbf{w}^\| = 1) and margin \gamma > 0 such that : y_i(\mathbf{w}^{*\top} \mathbf{x}_i) \geq \gamma, \quad \forall i Then, starting from \mathbf{w}^{(0)} = \mathbf{0}, the perceptron learning algorithm makes at most k mistake updates, where : k \leq \left(\frac{R}{\gamma}\right)^2$

Formal Mathematical Proof (Step-by-Step)

Suppose an update occurs at step t on misclassified point $(\mathbf{x}_i, y_i)$ where $y_i(\mathbf{w}^{(t)\top} \mathbf{x}_i) \leq 0$:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + y_i \mathbf{x}_i$$

*Step 1: Lower bounding the inner product with $\mathbf{w}^$ Take the dot product with $\mathbf{w}^$:$\mathbf{w}^{\top} \mathbf{w}^{(t+1)} = \mathbf{w}^{\top} \mathbf{w}^{(t)} + y_i (\mathbf{w}^{\top} \mathbf{x}_i) \ge \mathbf{w}^{\top} \mathbf{w}^{(t)} + \gamma$

$$By induction over k updates starting from $\mathbf{w}^{(0)} = \mathbf{0}$:$$

\mathbf{w}^{*\top} \mathbf{w}^{(k)} \ge k \gamma$$

Step 2: Upper bounding the squared norm $|\mathbf{w}^{(k)}|^2$

$$|\mathbf{w}^{(t+1)}|^2 = |\mathbf{w}^{(t)} + y_i \mathbf{x}_i|^2 = |\mathbf{w}^{(t)}|^2 + 2y_i(\mathbf{w}^{(t)\top} \mathbf{x}_i) + |\mathbf{x}_i|^2$$

Since an update occurs only when $y_i(\mathbf{w}^{(t)\top} \mathbf{x}_i) \le 0$ and $|\mathbf{x}_i| \le R$:

$$|\mathbf{w}^{(t+1)}|^2 \le |\mathbf{w}^{(t)}|^2 + R^2$$

By induction over k updates starting from $\mathbf{w}^{(0)} = \mathbf{0}$:

$$|\mathbf{w}^{(k)}|^2 \le kR^2$$

Step 3: Combining using Cauchy-Schwarz inequality By Cauchy-Schwarz: $$\mathbf{w}^{\top} \mathbf{w}^{(k)} \le |\mathbf{w}^| \, |\mathbf{w}^{(k)}| = 1 \cdot |\mathbf{w}^{(k)}| = |\mathbf{w}^{(k)}|$$

$$Squaring both sides :$$

(k \gamma)^2 \le (\mathbf{w}^{*\top} \mathbf{w}^{(k)})^2 \le |\mathbf{w}^{(k)}|^2 \le k R^2

k^2 \gamma^2 \le k R^2 \implies k \le \frac{R^2}{\gamma^2} = \left(\frac{R}{\gamma}\right)^2$$ ■

2.7 Linear Separability & Representation Power of Networks

Boolean Functions of n Variables

- For n binary variables, there are 2^n possible input combinations.
- A Boolean function assigns an output 0 or 1 to each input combination.
- Total number of distinct Boolean functions of n inputs:

$$\text{Total functions} = 2^{2^n}$$

- For $n = 2$: $2^{2^2} = 2^4 = 16$ functions.
- Of these 16 functions, **14 are linearly separable**; only **2 (XOR and XNOR)** are non-linearly separable.
- For $n \ge 3$, the fraction of linearly separable functions approaches zero exponentially!

The XOR Problem

x_1	x_2	y (XOR)
0	0	0
0	1	1
1	0	1
1	1	0

Geometrically, the positive instances $(0, 1)$ and $(1, 0)$ lie on one diagonal of the unit square, while the negative instances $(0, 0)$ and $(1, 1)$ lie on the opposite diagonal. No single straight line $w_1 x_1 + w_2 x_2 + b = 0$ can separate them.

Representation Power Theorems

1. **Any Boolean function can be represented by a 2-layer network of perceptrons**:
2. Hidden layer: Up to 2^n perceptrons (each perceptron activates for exactly one positive input minterm).
3. Output layer: 1 perceptron computing an OR over the active hidden units.
4. **Geometric Regions**:
5. **Single Perceptron**: Can separate half-spaces (open linear decision boundary).
6. **1 Hidden Layer (2 layers total)**: Can form any open or closed **convex polygon** by taking the intersection of half-spaces.
7. **2 Hidden Layers (3 layers total)**: Can form **arbitrary, disjoint, non-convex regions** by taking unions of convex polygons.

---

3. Core Formulas & Cheat Sheet

Concept	Mathematical Equation	Notes / Key Constraints
MP Neuron Firing	$y = 1 \iff \sum_{i=1}^{n} x_i \geq \theta$	Inputs $x_i \in 0, 1$, all weights $w_i = 1$, threshold $\theta \in \mathbb{Z}^+$
Perceptron Net Input	$z = \mathbf{w}^\top \mathbf{x} + b$	Real inputs $x_i \in \mathbb{R}$, real weights $w_i \in \mathbb{R}$, real bias $b \in \mathbb{R}$
Perceptron Decision Rule	$y = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$	1 if ≥ 0, else 0 (or -1)
Perceptron Update Rule	$\mathbf{w} \leftarrow \mathbf{w} + y_i \mathbf{x}_i$	Applied only when $y_i(\mathbf{w}^\top \mathbf{x}_i) \leq 0$
PLA Max Updates Bound	$k \leq \left(\frac{R}{\gamma}\right)^2$	$R = \max \lvert\mathbf{x}_i\rvert$, $\gamma = \min y_i(\mathbf{w}^{*\top} \mathbf{x}_i)$
Decision Boundary	$\mathbf{w}^\top \mathbf{x} + b = 0$	Normal vector is $\mathbf{w}$; distance to origin is $\frac{\lvert b\rvert}{\lvert\mathbf{w}\rvert}$
Total Boolean Functions	$N = 2^{2^n}$	Grows double-exponentially with number of inputs n
Sufficient Hidden Units	2^n perceptrons	Sufficient to represent any arbitrary n-input Boolean function

4. Assignment Solutions with Detailed Step-by-Step Explanations

Case Study 1: Smart Classroom Ventilation Control (Questions 1 to 5)

Context & Data

A smart classroom controller uses four binary indicators: - x_1: Carbon dioxide level is high - x_2: Room occupancy is high - x_3: Indoor temperature is above comfort limit - x_4: Outside air quality is acceptable Each indicator is 1 if true, 0 otherwise. The ventilation fan is governed by an MP-style threshold unit:

$$y = \{1 \quad \text{if } x_1 + x_2 + x_3 + x_4 \geq \theta \, 0 \quad \text{otherwise}$$

Requirement: The fan switches on ($y = 1$) only when at least 3 of the 4 indicators are active (1).

Question 1

Which threshold value implements the desired fan-control rule? - A) $\theta = 1$ - B) $\theta = 2$ - C) $\theta = 3$ - D) $\theta = 4$

Correct Answer: C

- **Detailed Step-by-Step Explanation**: The condition states: "fan switches on only when at least three of the four indicators are active." Thus, $\sum_{i=1}^{4} x_i \geq 3$. Comparing with the decision rule $\sum_{i=1}^{4} x_i \geq \theta$, we get $\theta = 3$.
- If $\theta = 1$, fan turns on if ≥ 1 indicator is active (OR logic).
- If $\theta = 2$, fan turns on if ≥ 2 indicators are active (majority of 4).
- If $\theta = 4$, fan turns on only if all 4 are active (AND logic).

Question 2

For input $(1, 0, 1, 1)$, what output does the controller produce? - A) $y = 0$, because only one condition is absent - B) $y = 1$, because three indicators are active - C) $y = 0$, because all four indicators must be active - D) $y = 1$, because the first indicator is active

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Input vector: $\mathbf{x} = (1, 0, 1, 1)$. Sum of inputs: $\sum_{i=1}^{4} x_i = 1 + 0 + 1 + 1 = 3$. Since $\theta = 3$, we check: $3 \geq 3$, which is True. Therefore, $y = 1$. The fan switches on because three indicators are active.

Question 3

How many binary input patterns switch on the fan? - A) 5 - B) 4 - C) 6 - D) 8

Correct Answer: A

- **Detailed Step-by-Step Explanation**: The total number of binary inputs of length 4 is $2^4 = 16$. The fan turns on when the number of active indicators is either 3 or 4:
- Number of patterns with exactly 3 ones: $\binom{4}{3} = \frac{4!}{3!1!} = 4$. Specifically: $(1, 1, 1, 0), (1, 1, 0, 1), (1, 0, 1, 1), (0, 1, 1, 1)$.
- Number of patterns with all 4 ones: $\binom{4}{4} = 1$. Specifically: $(1, 1, 1, 1)$.
- Total firing patterns: $\binom{4}{3} + \binom{4}{4} = 4 + 1 = 5$.

Question 4

If the threshold is accidentally changed to $\theta = 2$, what happens? - A) The fan switches on less often than intended. - B) The fan switches on when two or more indicators are active. - C) The fan switches on only when all inputs are 1, making activation too strict. - D) The fan never switches on.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: With threshold $\theta = 2$, the condition for activation becomes $\sum_{i=1}^{4} x_i \geq 2$. This triggers whenever 2, 3, or 4 indicators are active (total patterns = $\binom{4}{2} + \binom{4}{3} + \binom{4}{4} = 6 + 4 + 1 = 11$ patterns). Hence, the fan turns on when two or more indicators are active, which is more frequent than intended.

Question 5

Which property makes this controller a threshold-logic system? - A) It predicts future classroom occupancy states. - B) It updates weights after every incorrect output during a training process. - C) It requires a hidden layer to make a decision. - D) It compares a summed input value with a fixed threshold.

Correct Answer: D

- **Detailed Step-by-Step Explanation**: A threshold-logic unit (such as the McCulloch-Pitts neuron or threshold gate) is fundamentally defined by taking an unweighted or weighted sum of inputs and comparing it against a predetermined numerical threshold.

Case Study 2: Scholarship Application Screening (Questions 6 to 10)

Context & Data

A scholarship office screening aid flags applications for immediate review based on two binary features: - x_1: Academic score above cutoff - x_2: Financial-need certificate verified The office wants the system to flag an application when **at least one** of these two conditions is true. A perceptron with decision rule:

$$\hat{y} = \{1 \quad \text{if } w_1 x_1 + w_2 x_2 + b \geq 0\ 0 \quad \text{otherwise}$$

is tested.

Question 6

Which logical behavior is described by the desired screening rule? - A) AND - B) XOR - C) OR - D) XNOR

Correct Answer: C

- **Detailed Step-by-Step Explanation**: The phrase "at least one of these two conditions is true" corresponds directly to the logical **OR** operator: $y = x_1 \vee x_2$. The output is 1 if $(x_1, x_2) \in (0,1), (1,0), (1,1)$ and 0 only when $(0,0)$.

Question 7

Which parameter setting correctly implements this rule? - A) $w_1 = 1, w_2 = 1, b = -1.5$ - B) $w_1 = 1, w_2 = 1, b = -0.5$ - C) $w_1 = -1, w_2 = -1, b = 0.5$ - D) $w_1 = 1, w_2 = -1, b = 0$

Correct Answer: B

- **Detailed Step-by-Step Explanation**: We evaluate $z = w_1 x_1 + w_2 x_2 + b$ for all input combinations under option B ($w_1 = 1, w_2 = 1, b = -0.5$):
- $(0,0) : 1(0) + 1(0) - 0.5 = -0.5 < 0 \implies \hat{y} = 0$ (Correct)
- $(0,1) : 1(0) + 1(1) - 0.5 = +0.5 \geq 0 \implies \hat{y} = 1$ (Correct)
- $(1,0) : 1(1) + 1(0) - 0.5 = +0.5 \geq 0 \implies \hat{y} = 1$ (Correct)
- $(1,1) : 1(1) + 1(1) - 0.5 = +1.5 \geq 0 \implies \hat{y} = 1$ (Correct) Option A gives $b = -1.5$, which yields AND logic since $(0,1) \implies -0.5 < 0$.

Question 8

For $w_1 = 1, w_2 = 1, b = -0.5$, what is the score for input $(0,1)$? - A) 0.5 - B) -0.5 - C) 0 - D) 1.5

Correct Answer: A

- **Detailed Step-by-Step Explanation**: Score $z = w_1 x_1 + w_2 x_2 + b = 1(0) + 1(1) - 0.5 = 1 - 0.5 = 0.5$.

Question 9

What is the decision boundary for the parameter setting $w_1 = 1, w_2 = 1, b = -0.5$? - A) $x_1 - x_2 - 0.5 = 0$ - B) $x_1 + x_2 + 0.5 = 0$ - C) $2x_1 + x_2 = 0$ - D) $x_1 + x_2 - 0.5 = 0$

Correct Answer: D

- **Detailed Step-by-Step Explanation**: The decision boundary of a perceptron is the set of points where the linear score equals zero:

$$w_1x_1 + w_2x_2 + b = 0 \implies 1 \cdot x_1 + 1 \cdot x_2 - 0.5 = 0 \implies x_1 + x_2 - 0.5 = 0$$

Question 10

Why can a single perceptron represent this screening rule? - A) The positive and negative cases are linearly separable. - B) The rule requires remembering earlier applications before deciding the next output. - C) The rule has no decision boundary. - D) The output changes randomly for the same input.

Correct Answer: A

- **Detailed Step-by-Step Explanation**: A single perceptron can represent any function if and only if the positive class ($y = 1$) and negative class ($y = 0$) are linearly separable by a hyperplane. For OR, $(0, 0)$ is separated from $(0, 1), (1, 0), (1, 1)$ by the line $x_1 + x_2 - 0.5 = 0$.

Case Study 3: Equipment Handover Record Review (Questions 11 to 15)

Context & Data

An engineering lab tracks: - x_1: Battery pack returned - x_2: Sensor module returned Rule: Flag ($y = 1$) when **exactly one** of the two items has been returned. If neither or both are returned, $y = 0$. Truth table: - $(0, 0) \to 0$ - $(0, 1) \to 1$ - $(1, 0) \to 1$ - $(1, 1) \to 0$

Question 11

Which logical behavior does the handover rule represent? - A) AND - B) OR - C) XOR - D) NAND

Correct Answer: C

- **Detailed Step-by-Step Explanation**: The output is 1 when inputs differ, and 0 when inputs are the same. This is the exclusive-OR (**XOR**) logic function.

Question 12

What output is assigned to the input pattern $(1, 1)$? - A) $y = 1$, because one item has been returned - B) $y = 0$, because exactly one item has not been returned - C) $y = 1$, because both checks are complete - D) $y = 0$, because both values are the same

Correct Answer: D

- **Detailed Step-by-Step Explanation**: For $(1, 1)$, both items have been returned. The rule explicitly states that if both items have been returned, the record is not flagged ($y = 0$), because both values are identical.

Question 13

Which geometric property prevents a single perceptron from representing this rule? - A) The positive cases lie on opposite corners of the input square. - B) The inputs are binary rather than real valued, so the rule cannot use a separator. - C) The output has only two possible labels. - D) The rule uses two input variables.

Correct Answer: A

- **Detailed Step-by-Step Explanation**: In the unit square $[0, 1]^2$, the positive cases $(0, 1)$ and $(1, 0)$ lie on one diagonal, and the negative cases $(0, 0)$ and $(1, 1)$ lie on the opposite diagonal. Any single straight line dividing the square will leave at least one pair on the wrong side. Thus, the positive and negative points cannot be separated by a single hyperplane.

Question 14

Which modified lab rule would be linearly separable? - A) Flag exactly one returned item. - B) Flag when the two item statuses are different. - C) Flag only when both items have been returned. - D) Flag when neither or both items have been returned.

Correct Answer: C

- **Detailed Step-by-Step Explanation**: "Flag only when both items have been returned" represents the **AND** rule: $(1, 1) \to 1$ and all other inputs $\to 0$. The AND rule is linearly separable (e.g., using $x_1 + x_2 - 1.5 = 0$). Options A and B are XOR, and Option D is XNOR, both of which are non-linearly separable.

Question 15

What does linear separability mean in this setting? - A) Every input must have label 1 before a straight boundary can be checked. - B) A line can separate the 1-labeled and 0-labeled inputs. - C) The perceptron must avoid using weights. - D) The input values must be stored in a sequence.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Linear separability in 2D Euclidean space means there exists a straight line $w_1x_1 + w_2x_2 + b = 0$ such that all positive points lie strictly on one side of the line and all negative points lie strictly on the other side.

Case Study 4: Parcel Routing Decision Review (Questions 16 to 20)

Context & Data

A courier service trains a perceptron without bias ($b = 0$). - Features: x_1 (compactness), x_2 (priority label). - Labels: $y \in -1, +1$ ($+1$ for express, -1 for standard). - Perceptron update rule:

$$\mathbf{w} \leftarrow \mathbf{w} + y\mathbf{x} \quad \text{whenever } y(\mathbf{w}^\top \mathbf{x}) \leq 0$$

- Training dataset: - P_1: $\mathbf{x} = (2, 1)$, $y = +1$ - P_2: $\mathbf{x} = (1, 2)$, $y = +1$ - P_3: $\mathbf{x} = (-1, -1)$, $y = -1$ - Initial weights: $\mathbf{w}^{(0)} = (0, 0)$.

Question 16

What is the weight vector after processing P_1 once? - A) $\mathbf{w} = (2, 1)$ - B) $\mathbf{w} = (1, 2)$ - C) $\mathbf{w} = (-1, -1)$ - D) $\mathbf{w} = (0, 0)$

Correct Answer: A

- **Detailed Step-by-Step Explanation**:
- Initially $\mathbf{w}^{(0)} = (0, 0)$.
- Evaluate on $P_1 = (2, 1)$ with $y_1 = +1$:

$$\mathbf{w}^{(0)\top}\mathbf{x}_1 = 0(2) + 0(1) = 0$$

$$y_1(\mathbf{w}^{(0)\top}\mathbf{x}_1) = 1(0) = 0 \leq 0 \implies \text{Update triggered!}$$

- Perform update:

$$\mathbf{w}^{(1)} = \mathbf{w}^{(0)} + y_1\mathbf{x}_1 = (0, 0) + 1(2, 1) = (2, 1)$$

Question 17

After the update from P_1, what is $\mathbf{w}^\top\mathbf{x}$ for P_2? - A) 2 - B) 3 - C) 4 - D) 5

Correct Answer: C

- **Detailed Step-by-Step Explanation**: Current weight vector: $\mathbf{w} = (2, 1)$. Feature vector for P_2: $\mathbf{x}2 = (1, 2)$. $Dot product : $|mathbf{w}^|top |mathbf{x}2 = w_1 x + w_2 x$ = 2(1) + 1(2) = 2 + 2 = 4$$

Question 18

With $\mathbf{w} = (2, 1)$, is P_3 correctly classified? - A) No, because the raw score $\mathbf{w}^\top\mathbf{x}$ is greater than zero. - B) No, because $y(\mathbf{w}^\top\mathbf{x}) < 0$. - C) Yes, because $\mathbf{w}^\top\mathbf{x} = 0$. - D) Yes, because $y(\mathbf{w}^\top\mathbf{x}) > 0$.

Correct Answer: D

- **Detailed Step-by-Step Explanation**: Current weight vector: $\mathbf{w} = (2, 1)$. Sample P_3: $\mathbf{x}_3 = (-1, -1)$, $y_3 = -1$. Score:

$$\mathbf{w}^\top\mathbf{x}_3 = 2(-1) + 1(-1) = -2 - 1 = -3$$

Check classification condition:

$$y_3(\mathbf{w}^\top\mathbf{x}_3) = (-1) \cdot (-3) = +3 > 0$$

Since $y(\mathbf{w}^\top\mathbf{x}) > 0$, the classification is correct.

Question 19

Which condition is needed for the perceptron convergence guarantee? - A) Every feature must be binary. - B) The data must be linearly separable. - C) The learning rate must be zero throughout all training updates. - D) All labels must be positive.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Novikoff's Perceptron Convergence Theorem guarantees that the Perceptron Learning Algorithm will terminate and find a separating hyperplane in a finite number of steps if and only if the training dataset is **linearly separable**. If the data is not linearly separable, the algorithm can cycle indefinitely.

Question 20

Which vector separates the three listed parcels? - A) $\mathbf{w} = (-1, 1)$ - B) $\mathbf{w} = (1, -1)$ - C) $\mathbf{w} = (-1, -1)$ - D) $\mathbf{w} = (1, 1)$

Correct Answer: D

- **Detailed Step-by-Step Explanation**: A separating vector $\mathbf{w}$ must satisfy $y_i(\mathbf{w}^\top \mathbf{x}_i) > 0$ for all three parcels: Let's test option D: $\mathbf{w} = (1, 1)$:
- For $P_1 = (2, 1), y = +1$: $\mathbf{w}^\top \mathbf{x} = 1(2) + 1(1) = 3 > 0 \implies y(\mathbf{w}^\top \mathbf{x}) = 3 > 0$ (Correct)
- For $P_2 = (1, 2), y = +1$: $\mathbf{w}^\top \mathbf{x} = 1(1) + 1(2) = 3 > 0 \implies y(\mathbf{w}^\top \mathbf{x}) = 3 > 0$ (Correct)
- For $P_3 = (-1, -1), y = -1$: $\mathbf{w}^\top \mathbf{x} = 1(-1) + 1(-1) = -2 < 0 \implies y(\mathbf{w}^\top \mathbf{x}) = (-1)(-2) = 2 > 0$ (Correct) All samples are strictly correctly separated by $\mathbf{w} = (1, 1)$.

---

5. NPTEL Exam Traps & High-Yield Pitfalls

1. **Boundary Condition on Updates**:
2. In PLA, an update is triggered if $y_i(\mathbf{w}^\top \mathbf{x}_i) \leq 0$. If the score is exactly 0, **an update is performed** because the point lies directly on the boundary, which is not strictly separated.
3. **MP Neuron Weights**:
4. Remember that in a standard McCulloch-Pitts neuron, all weights are strictly uniform ($w_i = 1$) and inputs are binary (0 or 1). There are no negative weights; negative influences are modeled strictly through **veto/inhibitory inputs**.
5. **Number of Boolean Functions Formula**:
6. Be careful with exponents: 2^{2^n}, not 2^n and not 2^{n^2}.
7. For $n = 2$: $2^{2^2} = 2^4 = 16$.
8. For $n = 3$: $2^{2^3} = 2^8 = 256$.
9. For $n = 4$: $2^{2^4} = 2^{16} = 65,536$.
10. **Perceptron Margin and Bound**:
11. The convergence bound is $k \leq \frac{R^2}{\gamma^2}$. If margin γ is doubled, the maximum number of updates drops by a factor of 4.

MODULE 2

Week 2: Sigmoid Neurons, Gradient Descent, Feedforward Networks, and Universal Approximation

Week 2: Sigmoid Neurons, Gradient Descent, Feedforward Neural Networks, and Universal Approximation

1. Executive Summary & Exam Roadmap

Week 2 transitions the course from discrete, non-differentiable threshold logic (Perceptrons) to continuous, smooth, differentiable optimization: 1. **The Need for Smoothness**: The limitations of step functions; small weight changes cause catastrophic, discontinuous output flips. 2. **The Sigmoid (Logistic) Neuron**: Mathematical properties, output interpretation as posterior probability, derivative derivation $\sigma'(z) = \sigma(z)(1-\sigma(z))$, geometric behavior. 3. **Formal Derivation of Gradient Descent**: Why move opposite to the gradient? Taylor Series expansion, directional derivative, Cauchy-Schwarz condition ($\cos\beta = -1$). 4. **Learning Parameters for a Sigmoid Neuron**: Deriving gradients $\frac{\partial L}{\partial w}$ and $\frac{\partial L}{\partial b}$ under squared error loss. 5. **Feedforward Neural Networks & Universal Approximation Theorem**: How networks of sigmoids compose "tower functions" (bump functions) to approximate any continuous mathematical function to arbitrary precision (Cybenko, 1989).

2. Comprehensive Lecture Notes & Mathematical Foundations

2.1 Why Not Perceptrons? The Discontinuity Bottleneck

In a standard Perceptron with step activation:

$$y = f(z) = \{1 \quad \text{if } \mathbf{w}^\top \mathbf{x} + b \geq 0\ 0 \quad \text{otherwise}$$

The activation function has two fatal flaws for continuous optimization: 1. **Zero gradient almost everywhere**: $\frac{df}{dz} = 0$ for all $z \neq 0$. 2. **Undefined / Infinite gradient at origin**: At $z = 0$, $f(z)$ is non-differentiable.

Because the gradient is zero, gradient-based optimization algorithms cannot receive any informative signal indicating whether a small weight adjustment $\Delta\mathbf{w}$ moves the model closer to or further from the desired threshold.

2.2 The Sigmoid Neuron (Logistic Activation)

Mathematical Definition

The sigmoid neuron replaces the step threshold with the **logistic function**:

$$\hat{y} = f(\mathbf{x}) = \sigma(z) = \frac{1}{1+e^{-z}}$$

where $z = \mathbf{w}^\top \mathbf{x} + b = \sum_{i=1}^{n} w_i x_i + b$.

Key Algebraic and Geometric Properties

1. **Range**: $\sigma(z) \in (0,1)$ for all $z \in \mathbb{R}$.
2. **Symmetry**:

$$\sigma(-z) = \frac{1}{1+e^{z}} = \frac{e^{-z}}{1+e^{-z}} = 1 - \frac{1}{1+e^{-z}} = 1 - \sigma(z)$$

3. **Asymptotes**:

$$\lim_{z\to+\infty} \sigma(z) = 1, \quad \lim_{z\to-\infty} \sigma(z) = 0, \quad \sigma(0) = \frac{1}{2} = 0.5$$

4. **Controllable Steepness**:
5. For $z = wx + b$, if $w \to \infty$, $\sigma(wx+b)$ approaches a sharp Heaviside step function centered at $x = -b/w$.
6. If w is small, the transition is very gradual and linear-like near the origin.

Complete Derivation of the Sigmoid Derivative

The derivative of $\sigma(z)$ with respect to z is:

$$\frac{d\sigma(z)}{dz} = \frac{d}{dz}(1+e^{-z})^{-1} = -(1+e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1+e^{-z})^2}$$

Factor the expression:

$$\frac{d\sigma(z)}{dz} = \left(\frac{1}{1+e^{-z}}\right)\left(\frac{e^{-z}}{1+e^{-z}}\right) = \sigma(z)\cdot(1-\sigma(z))$$

IMPORTANT

> **Maximum Gradient Value**: > Since $\sigma(z) \in (0,1)$, the product $\sigma(z)(1-\sigma(z))$ achieves its maximum at $\sigma(z) = 0.5$ (which occurs at $z = 0$): >

$$\max \sigma'(z) = 0.5 \cdot (1 - 0.5) = 0.25$$

> For large $|z|$, $\sigma'(z) \to 0$. This gives rise to the **vanishing gradient problem** in deep networks.

2.3 Formal Derivation of Gradient Descent via Taylor Series

Consider a loss function $\mathcal{L}(\boldsymbol{\theta})$ where $\boldsymbol{\theta} = [w_1, \ldots, w_n, b]^\top$. We wish to find a small perturbation $\Delta\boldsymbol{\theta} = \eta\mathbf{u}$ (where $|\mathbf{u}| = 1$ and step size $\eta > 0$) such that the loss decreases:

$$\mathcal{L}(\boldsymbol{\theta} + \eta\mathbf{u}) - \mathcal{L}(\boldsymbol{\theta}) < 0$$

Taylor Series Expansion

Expanding $\mathcal{L}(\boldsymbol{\theta} + \eta\mathbf{u})$ around $\boldsymbol{\theta}$:

$$\mathcal{L}(\boldsymbol{\theta} + \eta\mathbf{u}) = \mathcal{L}(\boldsymbol{\theta}) + \eta\mathbf{u}^\top \nabla_\theta \mathcal{L}(\boldsymbol{\theta}) + \frac{\eta^2}{2!}\mathbf{u}^\top \nabla^2 \mathcal{L}(\boldsymbol{\theta})\mathbf{u} + \mathcal{O}(\eta^3)$$

For sufficiently small learning rate $\eta \ll 1$, higher-order terms vanish:

$$\Delta\mathcal{L} \approx \eta\, \mathbf{u}^\top \nabla_\theta \mathcal{L}(\boldsymbol{\theta})$$

Maximizing the Rate of Decrease

To achieve the maximum decrease in $\mathcal{L}$, we must minimize the inner product $\mathbf{u}^\top \nabla_\theta \mathcal{L}(\boldsymbol{\theta})$. By the definition of the dot product:

$$\mathbf{u}^\top \nabla_\theta \mathcal{L}(\boldsymbol{\theta}) = |\mathbf{u}|\, |\nabla_\theta \mathcal{L}(\boldsymbol{\theta})| \cos\beta = |\nabla_\theta \mathcal{L}(\boldsymbol{\theta})| \cos\beta$$

where β is the angle between the update direction $\mathbf{u}$ and the gradient vector $\nabla_\theta \mathcal{L}(\boldsymbol{\theta})$. - The cosine function $\cos\beta$ is minimized when $\cos\beta = -1 \implies \beta = 180^\circ$ (π radians). - Therefore, the optimal direction $\mathbf{u}$ is **directly opposite** to the gradient:

$$\mathbf{u}^* = -\frac{\nabla_\theta \mathcal{L}(\boldsymbol{\theta})}{|\nabla_\theta \mathcal{L}(\boldsymbol{\theta})|}$$

Gradient Descent Update Rule

Absorbing constants into the learning rate η:

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta\, \nabla_\theta \mathcal{L}(\boldsymbol{\theta}^{(t)})$$

Specifically for weight w and bias b:

$$w^{(t+1)} = w^{(t)} - \eta \frac{\partial \mathcal{L}}{\partial w}, \quad b^{(t+1)} = b^{(t)} - \eta \frac{\partial \mathcal{L}}{\partial b}$$

2.4 Gradient Derivation for a Single Sigmoid Neuron

Let the squared error loss for a sample $(\mathbf{x}, y)$ be:

$$\mathcal{L}(w, b) = \frac{1}{2}(\hat{y} - y)^2 = \frac{1}{2}(\sigma(z) - y)^2, \quad z = \mathbf{w}^\top \mathbf{x} + b$$

Applying the chain rule:

$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_j}$$

- $\frac{\partial \mathcal{L}}{\partial \hat{y}} = (\hat{y} - y)$ - $\frac{\partial \hat{y}}{\partial z} = \sigma'(z) = \hat{y}(1 - \hat{y})$ - $\frac{\partial z}{\partial w_j} = x_j$, and $\frac{\partial z}{\partial b} = 1$

Therefore:

$$\frac{\partial \mathcal{L}}{\partial w_j} = (\hat{y} - y)\, \hat{y}(1 - \hat{y})\, x_j$$

$$\frac{\partial \mathcal{L}}{\partial b} = (\hat{y} - y)\, \hat{y}(1 - \hat{y})$$

For N training points, the gradient is the sum over all points:

$$\nabla_{\mathbf{w}} \mathcal{L} = \sum_{i=1}^{N} (\hat{y}_i - y_i)\, \hat{y}_i (1 - \hat{y}_i)\, \mathbf{x}_i$$

2.5 Representation Power & Universal Approximation Theorem

Building "Tower" (Bump) Functions

1. **Single Sigmoid Step**: $f_1(x) = \sigma(w(x - s_1))$. With a large w, this approximates a step jumping from 0 to 1 at $x = s_1$.
2. **Second Shifted Step**: $f_2(x) = \sigma(w(x - s_2))$ where $s_2 > s_1$. This jumps from 0 to 1 at $x = s_2$.
3. **Subtracting Two Steps (Tower / Pulse Function)**:

$$h(x) = f_1(x) - f_2(x) = \sigma(w(x - s_1)) - \sigma(w(x - s_2))$$

$h(x)$ is approximately 0 for $x < s_1$, jumps to 1 for $s_1 \leq x \leq s_2$, and drops back to 0 for $x > s_2$. This forms a rectangular "tower" of width $(s_2 - s_1)$ and height 1.

4. **Weighted Combination of Towers**: Any arbitrary 1D continuous curve can be sliced into narrow rectangles of height A_k and width Δs_k:

$$f(x) \approx \sum_k A_k \left[\sigma(w(x - s_k)) - \sigma(w(x - s_{k+1}))\right]$$

Generalization to Multiple Dimensions (2D / nD)

- In 2D, two perpendicular 1D step combinations along x_1 and x_2 are fed into an AND-like sigmoid neuron, producing a localized 3D cylinder/box tower.
- By summing thousands of tiny towers, any continuous 3D surface can be reconstructed.

Universal Approximation Theorem (Cybenko 1989, Hornik 1991)

> A feedforward neural network with a **single hidden layer** containing a finite number of non-linear (e.g. sigmoid) activation functions can approximate any continuous function on a compact subset of $\mathbb{R}^n$ to arbitrary precision $\epsilon > 0$.

Catch: The theorem guarantees *existence*, not *learnability*. The required number of hidden neurons may grow exponentially with the input dimension, and finding those weights via gradient descent is non-trivial.

---

3. Core Formulas & Cheat Sheet

Concept	Mathematical Formula	Key Properties / Usage
Sigmoid Function	$\sigma(z) = \frac{1}{1+e^{-z}}$	Maps $\mathbb{R} \to (0,1)$; $\sigma(0) = 0.5$
Sigmoid Derivative	$\sigma'(z) = \sigma(z)(1 - \sigma(z))$	Maximum value is 0.25 at $z = 0$
Squared Error Loss	$\mathcal{L} = (y - \hat{y})^2$	Derivative w.r.t prediction: $\frac{\partial \mathcal{L}}{\partial \hat{y}} = 2(\hat{y} - y)$
Half-Squared Loss	$\mathcal{L} = \frac{1}{2}(y - \hat{y})^2$	Derivative w.r.t prediction: $\frac{\partial \mathcal{L}}{\partial \hat{y}} = (\hat{y} - y)$
Mean Squared Error (MSE)	$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$	Average squared penalty over n samples
Gradient Descent Step	$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}$	Moves in direction of steepest descent ($\beta = 180^\circ$)
Sigmoid Weight Gradient	$\frac{\partial \mathcal{L}}{\partial w_j} = (\hat{y} - y)\hat{y}(1 - \hat{y})x_j$	Product of error, activation derivative, and input
Tower Function	$h(x) = \sigma(w(x - s_1)) - \sigma(w(x - s_2))$	Requires 2 hidden neurons to produce a 1D pulse

---

4. Assignment Solutions with Detailed Step-by-Step Explanations

Case Study 1: Clinic Follow-up Priority Scoring (Questions 1 to 5)

Context & Data

A clinic produces a probability-like score $\hat{p}$ using normalized inputs: - x_1: Symptom persistence score - x_2: Abnormal reading score Linear score:

$$z = 3x_1 + 2x_2 - 2$$

Priority score:

$$\hat{p} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Decision rule: Send for priority follow-up when $\hat{p} \geq 0.5$.

Question 1

Why is the sigmoid function suitable for this priority score? - A) It maps a real-valued input to a value between 0 and 1. - B) It removes the need to compute any weighted input value. - C) It always gives a hard class label before thresholding. - D) It forces both input features to have identical weights.

Correct Answer: A

- **Detailed Step-by-Step Explanation**: The logistic sigmoid function maps the entire real line $(-\infty, \infty)$ into the open interval $(0, 1)$. This makes its output smooth, bounded, and naturally interpretable as a probability or priority score.

Question 2

For $x_1 = 0.4$ and $x_2 = 0.6$, what is the value of z? - A) 0.2 - B) 0.6 - C) 0.4 - D) 0.8

Correct Answer: C

- **Detailed Step-by-Step Explanation**: Substitute the values into $z = 3x_1 + 2x_2 - 2$:

$$z = 3(0.4) + 2(0.6) - 2 = 1.2 + 1.2 - 2 = 2.4 - 2 = 0.4$$

Question 3

The rule $\hat{p} \geq 0.5$ is equivalent to which condition? - A) $z < 0$ - B) $z \geq 0$ - C) $z > 1$ - D) $z \leq -1$

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Recall that $\sigma(0) = \frac{1}{1+e^0} = \frac{1}{2} = 0.5$. Because $\sigma(z)$ is a strictly monotonically increasing function of z:

$$\sigma(z) \geq 0.5 \iff z \geq 0$$

Question 4

Which input is not sent for priority follow-up? - A) $x_1 = 0.8, x_2 = 0.1$ - B) $x_1 = 0.2, x_2 = 0.8$ - C) $x_1 = 0.5, x_2 = 0.4$ - D) $x_1 = 0.1, x_2 = 0.4$

Correct Answer: D

- **Detailed Step-by-Step Explanation**: A record is sent for follow-up if $z \geq 0$. Let's compute $z = 3x_1 + 2x_2 - 2$ for each option:
- Option A: $z = 3(0.8) + 2(0.1) - 2 = 2.4 + 0.2 - 2 = +0.6 \geq 0$ (Sent)
- Option B: $z = 3(0.2) + 2(0.8) - 2 = 0.6 + 1.6 - 2 = +0.2 \geq 0$ (Sent)
- Option C: $z = 3(0.5) + 2(0.4) - 2 = 1.5 + 0.8 - 2 = +0.3 \geq 0$ (Sent)
- Option D: $z = 3(0.1) + 2(0.4) - 2 = 0.3 + 0.8 - 2 = -0.9 < 0 \implies \hat{p} < 0.5$ (**Not sent**)

Question 5

If the clinic wants fewer records to satisfy $\hat{p} \geq 0.5$, what should happen to the bias term -2? - A) It should become more negative. - B) It should be removed from the model entirely. - C) It should be made equal to $x_1 + x_2$. - D) It should be increased to make every score larger.

Correct Answer: A

- **Detailed Step-by-Step Explanation**: Decreasing the bias term (making it more negative, e.g. from -2 to -3) shifts the linear score z downward for all patient records. As a result, fewer inputs will satisfy $z \geq 0$, reducing the number of priority follow-up flags.

Case Study 2: Delivery Time Prediction Review (Questions 6 to 10)

Context & Data

A regression model predicts delivery delay in minutes: | Order | Actual Delay (y) | Predicted Delay ($\hat{y}$) | |:---:|:---:|:---:| | 1 | 12 | 10 | | 2 | 20 | 24 | | 3 | 15 | 13 | | 4 | 18 | 21 |

Formulas:

$$\mathcal{L} = (y - \hat{y})^2, \quad \text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2, \quad \frac{\partial \mathcal{L}}{\partial \hat{y}} = 2(\hat{y} - y)$$

Question 6

What is the squared error for Order 2? - A) 4 - B) 16 - C) 9 - D) 25

Correct Answer: B

- **Detailed Step-by-Step Explanation**: For Order 2: $y = 20, \hat{y} = 24$. Error $e = y - \hat{y} = 20 - 24 = -4$. Squared error $\mathcal{L} = (-4)^2 = 16$.

Question 7

What is the mean squared error over the four orders? - A) 7.25 - B) 8.00 - C) 9.50 - D) 8.25

Correct Answer: D

- **Detailed Step-by-Step Explanation**: Calculate squared error for each order:
- Order 1: $(12 - 10)^2 = 2^2 = 4$
- Order 2: $(20 - 24)^2 = (-4)^2 = 16$
- Order 3: $(15 - 13)^2 = 2^2 = 4$
- Order 4: $(18 - 21)^2 = (-3)^2 = 9$ Sum of squared errors $= 4 + 16 + 4 + 9 = 33$.

$$\text{MSE} = \frac{33}{4} = 8.25$$

Question 8

Which order contributes the largest squared error? - A) Order 1 - B) Order 3 - C) Order 2 - D) Order 4

Correct Answer: C

- **Detailed Step-by-Step Explanation**: Errors are: Order 1 (4), Order 2 (16), Order 3 (4), Order 4 (9). The maximum squared error is 16, coming from Order 2.

Question 9

If $y = 20$ and $\hat{y} = 24$, what is $\frac{\partial \mathcal{L}}{\partial \hat{y}}$? - A) 8 - B) -8 - C) 4 - D) -4

Correct Answer: A

- **Detailed Step-by-Step Explanation**: Using the given formula $\frac{\partial \mathcal{L}}{\partial \hat{y}} = 2(\hat{y} - y)$:

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = 2(24 - 20) = 2(4) = 8$$

Question 10

A second model predicts $11, 21, 14, 19$. Which model has lower MSE? - A) The original model, because its MSE is 8.25. - B) The second model, because its MSE is 1.00. - C) Both models, because their errors are identical. - D) Neither model, because MSE cannot be computed here.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Calculate squared errors for Model 2 against actual values $(12, 20, 15, 18)$:
- Order 1: $(12 - 11)^2 = 1^2 = 1$
- Order 2: $(20 - 21)^2 = (-1)^2 = 1$
- Order 3: $(15 - 14)^2 = 1^2 = 1$
- Order 4: $(18 - 19)^2 = (-1)^2 = 1$ Sum $= 1 + 1 + 1 + 1 = 4 \implies \text{MSE} = \frac{4}{4} = 1.00$. Since $1.00 < 8.25$, the second model has a substantially lower MSE.

Case Study 3: Machine Energy Use Prediction (Questions 11 to 15)

Context & Data

A feedforward neural network computes:

$$h_1 = \sigma(x_1 + x_2 - 1)$$
$$h_2 = \sigma(2x_1 - x_2)$$
$$\hat{y} = \sigma(3h_1 + 2h_2 - 2)$$

Inputs x_1 (duration score) and x_2 (load score).

Question 11

Which quantities are computed first in the feedforward direction? - A) The final loss value - B) The parameter update - C) The hidden units h_1 and h_2 - D) The target label chosen by the model

Correct Answer: C

- **Detailed Step-by-Step Explanation**: In forward propagation, computations proceed strictly layer by layer from input to output:
  $\text{Inputs } (x_1, x_2) \longrightarrow \text{Hidden activations } (h_1, h_2) \longrightarrow \text{Output } \hat{y} \longrightarrow \text{Loss } \mathcal{L}$. Therefore, the hidden unit activations h_1 and h_2 must be computed before the output or loss.

Question 12

For $x_1 = 1$ and $x_2 = 1$, what is the input to the sigmoid in h_1? - A) 1 - B) 0 - C) 2 - D) 3

Correct Answer: A

- **Detailed Step-by-Step Explanation**: The net input (pre-activation) to h_1 is:

$$z_{h1} = x_1 + x_2 - 1 = 1 + 1 - 1 = 1$$

Question 13

For $x_1 = 1$ and $x_2 = 0$, what is the input to the sigmoid in h_2? - A) -2 - B) -1 - C) 1 - D) 2

Correct Answer: D

- **Detailed Step-by-Step Explanation**: The pre-activation to h_2 is:

$$z_{h2} = 2x_1 - x_2 = 2(1) - 0 = 2$$

Question 14

Why can sigmoid hidden units introduce nonlinearity? - A) They copy inputs without changing any value. - B) They apply a nonlinear function to weighted input combinations. - C) They remove all weights from the network. - D) They force the output to be exactly linear for all possible inputs.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: If hidden units were linear ($f(z) = cz$), any sequence of layers would simply collapse into a single linear matrix multiplication: $\mathbf{W}_2(\mathbf{W}_1 \mathbf{x}) = (\mathbf{W}_2 \mathbf{W}*1)|mathbf{x} = |mathbf{W}*$. Applying a nonlinear function like the sigmoid to the affine combination creates a nonlinear feature space, allowing the network to model complex boundaries.}} \mathbf{x

---

Question 15

What is the final output range of $\hat{y}$? - A) Any real number from $-\infty$ to ∞ - B) Only the two values -1 and 1 - C) A value between 0 and 1 - D) A value greater than 1 for all inputs

Correct Answer: C

- **Detailed Step-by-Step Explanation**: Since $\hat{y} = \sigma(3h_1 + 2h_2 - 2)$ uses a logistic sigmoid as its output activation, its range is strictly the open interval $(0, 1)$.

Case Study 4: Warehouse Routing Time Analysis (Questions 16 to 20)

Context & Data

One-parameter regression model:

$$\hat{y} = wx$$

For a single training example: $x = 3, y = 15$. Squared error loss:

$$\mathcal{L}(w) = (y - \hat{y})^2 = (15 - 3w)^2$$

Question 16

What is $\mathcal{L}(w)$ when $w = 4$? - A) 6 - B) 4 - C) 12 - D) 9

Correct Answer: D

- **Detailed Step-by-Step Explanation**: Predicted value: $\hat{y} = wx = 4 \times 3 = 12$. Loss: $\mathcal{L}(4) = (15 - 12)^2 = 3^2 = 9$.

Question 17

Which value of w gives zero loss? - A) $w = 4$ - B) $w = 5$ - C) $w = 6$ - D) $w = 3$

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Zero loss requires $\mathcal{L}(w) = 0 \implies (15 - 3w)^2 = 0 \implies 15 - 3w = 0 \implies 3w = 15 \implies w = 5$.

Question 18

What is $\frac{d\mathcal{L}}{dw}$ at $w = 4$? - A) -18 - B) -6 - C) 18.0 - D) 6

Correct Answer: A

- **Detailed Step-by-Step Explanation**: Using the chain rule:

$$\frac{d\mathcal{L}}{dw} = \frac{d}{dw}(15 - 3w)^2 = 2(15 - 3w) \cdot \frac{d}{dw}(15 - 3w) = 2(15 - 3w)(-3) = -6(15 - 3w)$$

Evaluating at $w = 4$:

$$\left.\frac{d\mathcal{L}}{dw}\right|_{w=4} = -6(15 - 3(4)) = -6(15 - 12) = -6(3) = -18$$

Question 19

If gradient descent uses $w \leftarrow w - \eta \frac{d\mathcal{L}}{dw}$ with $\eta = 0.1$, what is the next value from $w = 4$? - A) 3.2 - B) 4.6 - C) 5.8 - D) 2.2

Correct Answer: C

- **Detailed Step-by-Step Explanation**:

$$w^{(1)} = w^{(0)} - \eta \frac{d\mathcal{L}}{dw} = 4 - 0.1 \times (-18) = 4 + 1.8 = 5.8$$

Question 20

What does the gradient indicate in this example? - A) It gives the class label directly without a model. - B) It removes the input feature from the prediction. - C) It makes the squared error impossible to compute. - D) It shows how the loss changes as w changes.

Correct Answer: D

- **Detailed Step-by-Step Explanation**: The gradient $\frac{d\mathcal{L}}{dw}$ represents the derivative of the loss function with respect to the parameter w. It quantifies the instantaneous rate of change and the direction of greatest increase of the loss function.

---

5. NPTEL Exam Traps & High-Yield Pitfalls

1. **Sign of Gradient Update**:
2. The update is $w \leftarrow w - \eta \nabla \mathcal{L}$.
3. If the gradient is negative (e.g. -18), subtracting a negative quantity **increases** the parameter ($4 - 0.1(-18) = 5.8$). Do not forget the minus sign!
4. **Derivative Factor of 2**:
5. Pay close attention to whether the loss is written as $\mathcal{L} = (y - \hat{y})^2$ or $\mathcal{L} = \frac{1}{2}(y - \hat{y})^2$.
6. For $(y - \hat{y})^2$, the derivative w.r.t $\hat{y}$ is $2(\hat{y} - y)$.
7. For $\frac{1}{2}(y - \hat{y})^2$, the factor of 2 cancels out, leaving $(\hat{y} - y)$.
8. **Sigmoid Slope Maximum**:
9. A common multiple choice question asks for the maximum derivative of the sigmoid function: it is **0.25** (occurring at $z = 0$, where $\sigma(0) = 0.5$).
10. **Tower Function Construction**:
11. To make a tower function in 1D, you need **at least 2 hidden neurons** (one for the rising edge, one for the falling edge).

MODULE 3

Week 3: Feedforward Neural Networks and the Backpropagation Algorithm

Week 3: Feedforward Neural Networks and the Backpropagation Algorithm

1. Executive Summary & Exam Roadmap

Week 3 explores the operational engine of deep learning: 1. **Multilayer Feedforward Architecture**: Layer-by-layer forward propagation notation ($\mathbf{x} = \mathbf{h}0, pre - activations|mathbf{a}k, post - activations|mathbf{h}k$). 2. ***Output Functions and Loss Functions***: - Regression: Linear output activation with Squared Error loss. - Binary Classification: Sigmoid activation with Binary Cross-Entropy loss. - Multi-class Classification: ***Softmax activation*** with ***Categorical Cross-Entropy loss***. 3. ***The Softmax + Cross-Entropy Gradient***: Full mathematical derivation of $\frac{\partial \mathcal{L}}{\partial a_i} = \hat{y}i - y_i$. 4. ***Computational Graphs & The Chain Rule***: Forward pass evaluation followed by backward pass local gradient accumulation. 5. ***The Vectorized Backpropagation Algorithm***: Complete derivation of output error vectors, parameter gradients ($\nablak}, \nablak}), and backward message passing(\nabla, \nabla}$).}

2. Comprehensive Lecture Notes & Mathematical Foundations

2.1 Multilayer Network Architecture and Notation

Consider a deep neural network with L layers ($L-1$ hidden layers and 1 output layer): - $\mathbf{x} \in \mathbb{R}^d$: Input vector, defined as $\mathbf{h}_0 = \mathbf{x}$. - For each layer $k \in 1, 2, \ldots, L$: - $\mathbf{W}k \in \mathbb{R}^{n_k \times n$: Weight matrix connecting layer k-1 to layer k. - $\mathbf{b}_k \in \mathbb{R}^{n_k}$: Bias vector for layer k. - $\mathbf{a}_k \in \mathbb{R}^{n_k}$: }**Pre-activation** vector: $$\mathbf{a}_k = \mathbf{b}_k + \mathbf{W}k \mathbf{h}$$ - $\mathbf{h}_k \in \mathbb{R}^{n_k}$: **Post-activation** vector:

$$\mathbf{h}_k = g(\mathbf{a}_k)$$

where $g(\cdot)$ is an element-wise activation function (e.g. sigmoid, tanh, ReLU). - At the output layer ($k = L$):

$$\hat{\mathbf{y}} = O(\mathbf{a}_L)$$

where $O(\cdot)$ is the output activation function.

```
x = h_0 ---> [ W_1, b_1 ] ---> a_1 ---> g(.) ---> h_1 ... ---> a_L ---> O(.) ---> y_hat ---> Loss L(y, y_hat)
```

2.2 Output Functions and Loss Functions

The choice of output activation and loss function depends directly on the task:

Task	Output Activation $O(\mathbf{a}_L)$	Target y	Loss Function $\mathcal{L}(\boldsymbol{\theta})$	Loss Formula
Regression	Linear: $\hat{y}i = a$	Real values $\mathbb{R}^K$	**Mean Squared Error**	$\frac{1}{2}\sum_i (\hat{y}_i - y_i)^2$
Binary Classification	Sigmoid: $\hat{y} = \sigma(a_L)$	Binary $y \in 0, 1$	**Binary Cross-Entropy**	$-[y \log \hat{y} + (1-y)\log(1-\hat{y})]$
Multi-class Classification	**Softmax**: $\hat{y}i = \frac{e^{a$}}{\sum_j e^{a_{L,j}}	One-hot vector $\mathbf{y} \in 0, 1^K$	**Categorical Cross-Entropy**	$-\sum_{c=1}^K y_c \log \hat{y}c = -\log \hat{y}\ell$

The Softmax Function

For a K-dimensional pre-activation vector $\mathbf{a}L:$\hat{y}i = \frac{e^{a{\sum}}, \quad \forall i \in {1, \dots, K}$$ Properties: 1. $0 < \hat{y}}^K e^{a_{L,j}}i < 1 \forall i. 2. \sum_i = 1$, representing a valid discrete probability distribution. 3. Exponentiation accentuates differences, causing the largest input to dominate the output probabilities.}^K \hat{y

2.3 Elegant Derivation: Softmax with Cross-Entropy Loss

Let $\mathbf{y}$ be a one-hot vector with $y_\ell = 1$ for the true class ℓ and $y_c = 0$ for all $c \neq \ell$. The loss is: $$\mathcal{L} = -\sum_{c=1}^K y_c \log \hat{y}c = -\log \hat{y}\ell

$$We want to find the gradient with respect to any output pre - activation$a_{L,i}$:$$

\frac{\partial \mathcal{L}}{\partial a_{L,i}} = -\frac{1}{\hat{y}\ell} \frac{\partial \hat{y}\ell}{\partial a_{L,i}}$$

Step 1: Derivative of Softmax $\frac{\partial \hat{y}\ell}{\partial a$}

Recall $\hat{y}\ell = \frac{e^{aExtra close brace or missing open brace}. Let $S = \sum_{j=1}^K e^{a_{L,j}}$**Case 1:** $i = \ell$ (derivative w.r.t the true class pre-activation): $$\frac{\partial \hat{y}\ell}{\partial a}} = \frac{e^{a_{L,\ell}} \cdot S - e^{a_{L,\ell}} \cdot e^{a_{L,\ell}}}{S^2} = \frac{e^{a_{L,\ell}}}{S} - \left(\frac{e^{a_{L,\ell}}}{S}\right)^2 = \hat{y}\ell (1 - \hat{y}\ell)$$ - **Case 2:** $i \neq \ell$ (derivative w.r.t an incorrect class pre-activation): $$\frac{\partial \hat{y}\ell}{\partial a_i$}} = \frac{0 \cdot S - e^{a_{L,\ell}} \cdot e^{a_{L,i}}}{S^2} = -\frac{e^{a_{L,\ell}}}{S} \cdot \frac{e^{a_{L,i}}}{S} = -\hat{y}_\ell \hat{y}

Step 2: Substituting into the Loss Derivative

- For $i = \ell$: $$\frac{\partial \mathcal{L}}{\partial a_{L,\ell}} = -\frac{1}{\hat{y}_\ell} \cdot \hat{y}_\ell(1 - \hat{y}_\ell) = -(1 - \hat{y}_\ell) = \hat{y}_\ell - 1 = \hat{y}_\ell - y_\ell$$
- For $i \neq \ell$: $$\frac{\partial \mathcal{L}}{\partial a_{L,i}} = -\frac{1}{\hat{y}_\ell} \cdot (-\hat{y}_\ell \hat{y}_i) = \hat{y}_i = \hat{y}_i - 0 = \hat{y}_i - y_i$$

In vector notation:

$$\nabla_{\mathbf{a}_L} \mathcal{L} = \hat{\mathbf{y}} - \mathbf{y}$$

> **NOTE**
> > The derivative of Categorical Cross-Entropy with Softmax simplifies exactly to $(\hat{y} - y)$, the raw difference between the predicted probability and the true one-hot target.

2.4 Computational Graphs & Backpropagation Intuition

In a computational graph: - **Forward Pass**: Nodes compute output values from inputs along directed edges. - **Backward Pass**: Reverse traversal. Every node receives an incoming gradient from downstream:

$$\frac{\partial \mathcal{L}}{\partial \text{output}}$$

and computes its local gradient w.r.t each input using the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \text{input}} = \frac{\partial \mathcal{L}}{\partial \text{output}} \times \frac{\partial \text{output}}{\partial \text{input}}$$

- **Branching Nodes**: If an intermediate quantity u feeds into multiple subsequent nodes $v_1, v_2, \ldots, v_m$, its gradient is the sum of gradients from all outgoing paths:

$$\frac{\partial \mathcal{L}}{\partial u} = \sum_{j=1}^{m} \frac{\partial \mathcal{L}}{\partial v_j} \frac{\partial v_j}{\partial u}$$

2.5 The Full Matrix Backpropagation Algorithm

Given pre-activations $\mathbf{a}_k$ and post-activations $\mathbf{h}_k$ from the forward pass:

1. Output Layer Gradient

$$\nabla_{\mathbf{a}_L} \mathcal{L} = \hat{\mathbf{y}} - \mathbf{y}$$

2. Backward Recursion (for $k = L$ down to 1):

- **Gradient w.r.t. Weights Matrix $\mathbf{W}_k$**: $$\nabla_{\mathbf{W}_k} \mathcal{L} = (\nabla_{\mathbf{a}_k} \mathcal{L}) \, (\mathbf{h}_{k-1})^\top$$ *(Outer product of layer k's error vector and layer $k-1$'s activation vector)*
- **Gradient w.r.t. Bias Vector $\mathbf{b}_k$**: $$\nabla_{\mathbf{b}_k} \mathcal{L} = \nabla_{\mathbf{a}_k} \mathcal{L}$$
- **Backpropagate Error to Previous Post-Activations $\mathbf{h}_{k-1}$**: $$\nabla_{\mathbf{h}_{k-1}} \mathcal{L} = \mathbf{W}_k^\top (\nabla_{\mathbf{a}_k} \mathcal{L})$$
- **Pass Through Previous Activation Derivative to Pre-Activations $\mathbf{a}_{k-1}$**: $$\nabla_{\mathbf{a}_{k-1}} \mathcal{L} = (\nabla_{\mathbf{h}_{k-1}} \mathcal{L}) \odot g'(\mathbf{a}_{k-1})$$ *(where $\odot$ denotes element-wise Hadamard product)*

3. Parameter Update:

$$\mathbf{W}_k \leftarrow \mathbf{W}_k - \eta \nabla_{\mathbf{W}_k} \mathcal{L}$$

$$\mathbf{b}_k \leftarrow \mathbf{b}_k - \eta \nabla_{\mathbf{b}_k} \mathcal{L}$$

---

3. Core Formulas & Cheat Sheet

Step	Vector / Matrix Equation	Dimensions
Forward Pre-activation	$\mathbf{a}_k = \mathbf{W}_k \mathbf{h}_k + \mathbf{b}_k$	$(n_k \times 1) = (n_k \times n_{k-1})(n_{k-1} \times 1) + (n_k \times 1)$
Forward Post-activation	$\mathbf{h}_k = g(\mathbf{a}_k)$	$(n_k \times 1)$
Output Error	$\nabla_{\mathbf{a}_L} \mathcal{L} = \hat{\mathbf{y}} - \mathbf{y}$	$(n_L \times 1)$
Weight Gradient	$\nabla_{\mathbf{W}_k} \mathcal{L} = (\nabla_{\mathbf{a}_k} \mathcal{L}) \, (\mathbf{h}_{k-1})^\top$	$(n_k \times n_{k-1}) = (n_k \times 1)(1 \times n_{k-1})$
Bias Gradient	$\nabla_{\mathbf{b}_k} \mathcal{L} = \nabla_{\mathbf{a}_k} \mathcal{L}$	$(n_k \times 1)$
Hidden Activation Grad	$\nabla_{\mathbf{h}_{k-1}} \mathcal{L} = \mathbf{W}_k^\top (\nabla_{\mathbf{a}_k} \mathcal{L})$	$(n_{k-1} \times 1) = (n_{k-1} \times n_k)(n_k \times 1)$
Previous Pre-act Grad	$\nabla_{\mathbf{a}_{k-1}} \mathcal{L} = (\nabla_{\mathbf{h}_{k-1}} \mathcal{L}) \odot g'(\mathbf{a}_{k-1})$	$(n_{k-1} \times 1)$
Computational Chain Rule	$\frac{\partial \mathcal{L}}{\partial x} = \sum_m \frac{\partial \mathcal{L}}{\partial q_m} \frac{\partial q_m}{\partial x}$	Sum over all forward paths branching from x

4. Assignment Solutions with Detailed Step-by-Step Explanations

Case Study 1: Tutoring Session Readiness Score (Questions 1 to 5)

Context & Data

A tutoring platform uses a feedforward network: Inputs: $x_1 = 2$ (student preparation score), $x_2 = 1$ (topic difficulty score). Equations:

$$z_1 = 2x_1 + x_2$$
$$z_2 = x_1 - x_2 + 1$$
$$\hat{y} = z_1 + 2z_2$$

Question 1

What is the value of z_1 for this session? - A) 6 - B) 5 - C) 4 - D) 3

Correct Answer: B

- **Detailed Step-by-Step Explanation**:

$$z_1 = 2x_1 + x_2 = 2(2) + 1 = 4 + 1 = 5$$

Question 2

What is the value of z_2? - A) 0 - B) 1 - C) 3 - D) 2

Correct Answer: D

- **Detailed Step-by-Step Explanation**:

$$z_2 = x_1 - x_2 + 1 = 2 - 1 + 1 = 2$$

Question 3

What is the final value of $\hat{y}$? - A) 9 - B) 7 - C) 8 - D) 6

Correct Answer: A

- **Detailed Step-by-Step Explanation**:

$$\hat{y} = z_1 + 2z_2 = 5 + 2(2) = 5 + 4 = 9$$

Question 4

Which computation must be completed before $\hat{y}$ is evaluated? - A) The loss must be minimized over all stored tutoring sessions. - B) The target value must be changed by the model before scoring. - C) The hidden quantities z_1 and z_2 must be computed. - D) The learning rate must be set to zero.

Correct Answer: C

- **Detailed Step-by-Step Explanation**: In a feedforward network, functional evaluation follows topological ordering. Since $\hat{y} = z_1 + 2z_2$, its inputs are z_1 and z_2. Therefore, z_1 and z_2 must be computed before $\hat{y}$ can be evaluated.

Question 5

What makes this a feedforward computation? - A) The output is repeatedly fed back as a new input. - B) Information flows from inputs to hidden quantities to output. - C) The target label is used before the input features. - D) The network stores a separate state for each previous session.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: A feedforward neural network computes activations strictly in a forward acyclic direction from input nodes through intermediate hidden layers to output nodes, without cycles or loops.

Case Study 2: Inventory Forecast Calculation Review (Questions 6 to 10)

Context & Data

A computational graph:

$$u = 2x$$
$$v = u + 3$$

$$\mathcal{L} = v^2$$

For the current product, $x = 1$.

Question 6

What is the value of u? - A) 2 - B) 1 - C) 3 - D) 4

Correct Answer: A

- **Detailed Step-by-Step Explanation**:

$$u = 2x = 2(1) = 2$$

Question 7

What is the value of v? - A) 3 - B) 4 - C) 6 - D) 5

Correct Answer: D

- **Detailed Step-by-Step Explanation**:

$$v = u + 3 = 2 + 3 = 5$$

Question 8

What is the loss $\mathcal{L}$? - A) 20 - B) 15 - C) 25 - D) 10

Correct Answer: C

- **Detailed Step-by-Step Explanation**:

$$\mathcal{L} = v^2 = 5^2 = 25$$

Question 9

What is $\frac{\partial \mathcal{L}}{\partial v}$ at $v = 5$? - A) 5 - B) 10 - C) 25 - D) 2

Correct Answer: B

- **Detailed Step-by-Step Explanation**:

$$\frac{\partial \mathcal{L}}{\partial v} = \frac{d}{dv}(v^2) = 2v$$

At $v = 5$:

$$\frac{\partial \mathcal{L}}{\partial v} = 2(5) = 10$$

Question 10

What is $\frac{\partial \mathcal{L}}{\partial x}$? - A) 10 - B) 5 - C) 40 - D) 20

Correct Answer: D

- **Detailed Step-by-Step Explanation**: Applying the chain rule through the computational graph:

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial v} \cdot \frac{\partial v}{\partial u} \cdot \frac{\partial u}{\partial x}$$

- $\frac{\partial \mathcal{L}}{\partial v} = 2v = 10$
- $\frac{\partial v}{\partial u} = \frac{d}{du}(u + 3) = 1$
- $\frac{\partial u}{\partial x} = \frac{d}{dx}(2x) = 2$

$$\frac{\partial \mathcal{L}}{\partial x} = 10 \cdot 1 \cdot 2 = 20$$

Case Study 3: Sensor Calibration Error Analysis (Questions 11 to 15)

Context & Data

A model with one hidden unit:

$$h = w_1 x + b$$

$$\hat{y} = w_2 h$$

Parameters: $w_1 = 1, b = 0, w_2 = 2$. Training example: $x = 2, y = 5$. Loss function:

$$\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$$

Question 11

What is the hidden value h? - A) 1 - B) 4 - C) 2 - D) 5

Correct Answer: C

- **Detailed Step-by-Step Explanation**:

$$h = w_1 x + b = 1(2) + 0 = 2$$

Question 12

What is the prediction $\hat{y}$? - A) 3 - B) 5 - C) 2 - D) 4

Correct Answer: D

- **Detailed Step-by-Step Explanation**:

$$\hat{y} = w_2 h = 2(2) = 4$$

Question 13

What is $\frac{\partial \mathcal{L}}{\partial \hat{y}}$? - A) -1 - B) 1 - C) -2 - D) 2

Correct Answer: A

- **Detailed Step-by-Step Explanation**:

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \frac{d}{d\hat{y}}\left[\frac{1}{2}(\hat{y} - y)^2\right] = \hat{y} - y = 4 - 5 = -1$$

Question 14

What is $\frac{\partial \mathcal{L}}{\partial w_2}$? - A) -1 - B) -2 - C) 2 - D) 1

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Since $\hat{y} = w_2 h$, by chain rule:

$$\frac{\partial \mathcal{L}}{\partial w_2} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_2} = (\hat{y} - y) \cdot h = (-1) \cdot 2 = -2$$

Question 15

What is $\frac{\partial \mathcal{L}}{\partial w_1}$? - A) -1 - B) 1 - C) -4 - D) 4

Correct Answer: C

- **Detailed Step-by-Step Explanation**: By backpropagation through h:

$$\frac{\partial \mathcal{L}}{\partial h} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h} = (-1) \cdot w_2 = (-1)(2) = -2$$

  Since $h = w_1 x + b$, $\frac{\partial h}{\partial w_1} = x = 2$.

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial \mathcal{L}}{\partial h} \cdot \frac{\partial h}{\partial w_1} = (-2) \cdot 2 = -4$$

Case Study 4: Course Recommendation Score Update (Questions 16 to 20)

Context & Data

Linear prediction model:

$$\hat{y} = w_1 x_1 + w_2 x_2 + b$$

Training example: $x_1 = 1, x_2 = 2, y = 6$. Initial parameters: $w_1 = 1, w_2 = 1, b = 0$. Loss: $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$. Learning rate: $\eta = 0.1$. Update rule: $\theta \leftarrow \theta - \eta \frac{\partial \mathcal{L}}{\partial \theta}$.

Question 16

What is the initial prediction $\hat{y}$? - A) 4 - B) 3 - C) 5 - D) 2

Correct Answer: B

- **Detailed Step-by-Step Explanation**:

$$\hat{y} = w_1 x_1 + w_2 x_2 + b = 1(1) + 1(2) + 0 = 1 + 2 + 0 = 3$$

Question 17

What is $\frac{\partial \mathcal{L}}{\partial \hat{y}}$? - A) -3 - B) 3 - C) -6 - D) 6

Correct Answer: A

- **Detailed Step-by-Step Explanation**:

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \hat{y} - y = 3 - 6 = -3$$

Question 18

What is $\frac{\partial \mathcal{L}}{\partial w_2}$? - A) 3 - B) -3 - C) 6 - D) -6

Correct Answer: D

- **Detailed Step-by-Step Explanation**:

$$\frac{\partial \mathcal{L}}{\partial w_2} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_2} = (\hat{y} - y) \cdot x_2 = (-3) \cdot 2 = -6$$

Question 19

What is the updated value of w_1? - A) 0.7 - B) 1.6 - C) 1.3 - D) 0.4

Correct Answer: C

- **Detailed Step-by-Step Explanation**: First compute gradient w.r.t w_1:

$$\frac{\partial \mathcal{L}}{\partial w_1} = (\hat{y} - y) \cdot x_1 = (-3) \cdot 1 = -3$$

  Apply gradient update:

$$w_1^{(1)} = w_1^{(0)} - \eta \frac{\partial \mathcal{L}}{\partial w_1} = 1 - 0.1 \times (-3) = 1 + 0.3 = 1.3$$

Question 20

What is the updated value of b? - A) 0.3 - B) -0.3 - C) 0.6 - D) -0.6

Correct Answer: A

- **Detailed Step-by-Step Explanation**: Compute gradient w.r.t b:

$$\frac{\partial \mathcal{L}}{\partial b} = (\hat{y} - y) \cdot \frac{\partial \hat{y}}{\partial b} = (-3) \cdot 1 = -3$$

  Apply gradient update:

$$b^{(1)} = b^{(0)} - \eta \frac{\partial \mathcal{L}}{\partial b} = 0 - 0.1 \times (-3) = 0 + 0.3 = 0.3$$

---

5. NPTEL Exam Traps & High-Yield Pitfalls

1. **Matrix Dimensions in Backpropagation**:
2. Remember: if $\mathbf{h}_{k-1} is n_k \times 1 and \mathbf{a}_k is n_k \times 1, the weight matrix \mathbf{W}_k has dimensions n_k \times n$.
3. The gradient $\nabla_{\mathbf{W}_k} \mathcal{L} must have the same dimension n_k \times n$. It is computed as $(\nabla_{\mathbf{a}_k} \mathcal{L}) (\mathbf{h}_{k-1})^\top$, **NOT** $(\mathbf{h}_{k-1}) (\nabla_{\mathbf{a}_k} \mathcal{L})^\top$.
4. **Transpose in Backpropagating Hidden Errors**:
5. When pulling the error back through weights to layer $k-1$: $\nabla_{\mathbf{h}_{k-1}} \mathcal{L} = \mathbf{W}_k^\top (\nabla_{\mathbf{a}_k} \mathcal{L}$. The weight matrix **must be transposed**!
6. **Cross-Entropy Derivative Simplification**:

7. For Softmax + Cross-Entropy, the derivative w.r.t logits $a_{L,i}$ simplifies completely to $\hat{y}_i - y_i$. You do not need to evaluate messy quotients during exams!

MODULE 4

Week 4: Optimization Algorithms — GD, Momentum, NAG, AdaGrad, RMSProp, and Adam

Week 4: Optimization Algorithms — Gradient Descent, Momentum, NAG, AdaGrad, RMSProp, and Adam

1. Executive Summary & Exam Roadmap

Week 4 focuses on training modern deep architectures efficiently: 1. **Gradient Descent Variants**: - **Batch GD**: Computes exact gradients over the entire dataset N. - **Stochastic GD (SGD)**: Computes noisy gradients on 1 sample at a time. - **Mini-Batch GD**: Evaluates gradients over a batch of size B (e.g. 32, 64, 128), balancing vectorization, memory, and variance. 2. **Pathologies of Standard Gradient Descent**: - Ravines/valleys (steep in one dimension, flat in another) cause wild cross-valley oscillations and slow progress along the valley floor. - Saddle points and plateaus with near-zero gradients. 3. **Momentum-Based Gradient Descent**: Uses exponential averaging of previous gradients to accelerate along persistent directions and dampen oscillations. 4. **Nesterov Accelerated Gradient (NAG)**: Looks ahead along the momentum trajectory before evaluating the gradient, preventing overshoot. 5. **Adaptive Learning Rate Optimizers**: - **AdaGrad**: Accumulates sum of squared gradients to scale learning rates inversely per parameter. - **RMSProp**: Resolves AdaGrad's premature stopping using an exponentially decaying moving average of squared gradients. - **Adam**: Combines first-moment momentum with second-moment RMSProp, including critical initial bias correction steps.

2. Comprehensive Lecture Notes & Mathematical Foundations

2.1 The Spectrum of Gradient Descent: Batch vs. Stochastic vs. Mini-Batch

Let the objective function be the average loss over N training examples:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N}\mathcal{L}_i(\boldsymbol{\theta})$$

Attribute	Batch Gradient Descent (BGD)	Stochastic Gradient Descent (SGD)	Mini-Batch Gradient Descent (MBGD)
Batch Size (B)	$B = N$ (all samples)	$B = 1$ (single sample)	$1 < B < N$ (typically 32 to 512)
Updates per Epoch	Exactly 1 update per epoch	N updates per epoch	N/B updates per epoch
Gradient Nature	True, exact gradient $\nabla\mathcal{L}$	High-variance, noisy estimate	Moderately smooth, low variance
Computational Cost	High memory; cannot process massive datasets in RAM	Very cheap per step, but misses GPU matrix parallelization	Highly efficient with GPU vectorization
Convergence Behavior	Smooth, monotonic loss decrease	Erratic/noisy trajectory; can escape shallow local minima	Smooth convergence with slight stochastic noise

2.2 Pathologies of Standard Gradient Descent

1. **Ill-Conditioned Curvature (Ravines)**:
2. When the Hessian matrix has a high condition number ($\lambda_{\max} \gg \lambda_{\min}$), the error surface resembles an elongated ravine.
3. Standard GD oscillates wildly back and forth across the steep walls while making agonizingly slow progress along the flat base.
4. **Saddle Points and Local Plateaus**:
5. At a saddle point or zero-gradient plateau, $|\nabla_\theta\mathcal{L}| \approx 0$.
6. Because standard GD updates are strictly proportional to the gradient magnitude ($\Delta\boldsymbol{\theta} = -\eta\nabla\mathcal{L}$), the optimizer can become trapped or take thousands of iterations to crawl out.

2.3 Momentum-Based Gradient Descent

Physical Intuition

Think of a heavy ball rolling down a hilly terrain. As it rolls down the slope, it accumulates velocity. When it encounters slight bumps or flat plateaus, its accumulated inertia carries it forward. Across narrow ravines, opposing vertical forces cancel out while forward forces reinforce each other.

Mathematical Formulation

At step t, maintain a velocity vector $\mathbf{v}t:$|mathbf{v}t = |beta |mathbf{v} + |eta |nabla)

\boldsymbol{\theta}}} \mathcal{L}(\boldsymbol{\theta}_{t-1*t = |boldsymbol{|theta}_t$$ } - \mathbf{v *(where $\beta \in [0, 1)$ is the momentum decay factor, typically 0.9, and η is the learning rate).*

Expanding the Velocity History

Expanding $\mathbf{v}_{t recursively}:$*|mathbf{v}t = |eta |sum |nabla}^t |beta^{t-|tau}} \mathcal{L}(\boldsymbol{\theta*{|tau-1})*

$-Alongoscillatingdirectionswheregradientsalternatesigns(\$ + g, -g\$), thetermscancel : \$ \sum \beta^k g_k \approx 0\$. -Alongconsistentdownhilldirectionswheregradientshavethesamesign(\$*

|mathbf{v}$$ For $\beta = 0.9$, the step size along consistent directions is amplified by a factor of $\frac{1}{1-0.9} = 10\times$!}} = \frac{\eta}{1 - \beta} \mathbf{g

---

2.4 Nesterov Accelerated Gradient (NAG)

The Problem with Momentum: Overshooting

Because momentum accumulates velocity, a ball rolling at high speed down a slope might overshoot the minimum and roll up the opposite slope before turning around.

The Nesterov Solution: "Look-Ahead" Gradient

Instead of computing the gradient at the current position $\boldsymbol{\theta}*{t-1}, computethegradientatthepredicted"look-ahead"point|boldsymbol{|theta}$ - \beta \mathbf{v*{t-1}:$|boldsymbol{|theta}$}} = \boldsymbol{\theta*{t-1} - |beta |mathbf{v}*

\mathbf{v}*t = |beta |mathbf{v}* + \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta*{t-1} - |beta |mathbf{v}*)

\boldsymbol{\theta}*t = |boldsymbol{|theta}_t$$} - \mathbf{v

```
Standard Momentum:
Current Position ----(Momentum Step)----> Intermediate ----(Gradient Step)----> Final Position
                       \                                  /
                        \------- Net Update Vector -----/

Nesterov Momentum:
Current Position ----(Momentum Step)----> Look-ahead Point ----(Corrective Gradient)----> Final Position
```

If the look-ahead point starts climbing up the opposite valley wall, the gradient at that point immediately points backwards, acting as an automatic brake to prevent overshoot!

2.5 AdaGrad (Adaptive Gradient Algorithm)

Motivation: Parameter-Specific Learning Rates

In many tasks (e.g. NLP, sparse features), some weights receive frequent, large gradients, while others receive infrequent, tiny gradients. A single global learning rate η either causes frequent features to explode or sparse features to never train.

Update Equations

Accumulate the sum of historical squared gradients for each parameter:

$$v_t = v_{t-1} + g_t^2$$

Update parameter:

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{v_t + \epsilon}} g_t$$

(where $\epsilon > 0$, typically 10^{-8}, prevents division by zero).

Drawback of AdaGrad

Since $g_t^2 \geq 0$, the accumulator v_t **monotonically increases** with every iteration:

$$v_t \geq v_{t-1} \implies \frac{\eta}{\sqrt{v_t + \epsilon}} \to 0 \quad \text{as } t \to \infty$$

The effective learning rate continuously decays and eventually becomes infinitesimal, causing the algorithm to freeze and stop learning before reaching the optimum.

2.6 RMSProp (Root Mean Square Propagation)

Fixing AdaGrad's Shrinking Learning Rate

Geoff Hinton proposed modifying AdaGrad by replacing the indefinite sum with an **exponentially decaying moving average** of squared gradients:

$$v_t = \beta v_{t-1} + (1 - \beta) g_t^2$$

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{v_t + \epsilon}} g_t$$

(typically $\beta = 0.9$ or 0.99).

- The denominator $\sqrt{v_t}$ now tracks the *recent* magnitude of gradients rather than the entire infinite history.
- If recent gradients are large, the step is downscaled; if recent gradients are small, the step is scaled up. Learning never prematurely grinds to a halt.

2.7 Adam (Adaptive Moment Estimation)

Adam combines the advantages of **Momentum** (first moment: running average of gradients) and **RMSProp** (second moment: running average of squared gradients).

Complete Algorithm Formulation

1. **Compute Current Gradient**: $$\mathbf{g}*t = \nabla*{\boldsymbol{\theta}*{t-1}) \mathcal{L}(\boldsymbol{\theta}*{t-1}$$
2. **Update Biased First Moment (Mean)**: $$\mathbf{m}*t = \beta_1 \mathbf{m}*{t-1}$$ + (1 - \beta_1) \mathbf{g}*t$$
3. **Update Biased Second Raw Moment (Uncentered Variance)**: $$\mathbf{v}*t = \beta_2 \mathbf{v}*{t-1}$$ + (1 - \beta_2) \mathbf{g}*t^2$$
4. **Bias Correction**: Since $\mathbf{m}_0 = \mathbf{0}$ and $\mathbf{v}_0 = \mathbf{0}$, both estimates are heavily biased towards zero during early iterations (especially when $\beta_1, \beta_2 \approx 1$).

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}$$

$$\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}$$

   (Note: β^t is β raised to the power of step count t. As $t \to \infty$, $1 - \beta^t \to 1$ and correction vanishes).
5. **Update Parameters**: $$\boldsymbol{\theta}*t = \boldsymbol{\theta}*{t-1}$$ - \frac{\eta}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \hat{\mathbf{m}}_t

Standard default hyperparameters:

$$\eta = 0.001, \quad \beta_1 = 0.9, \quad \beta_2 = 0.999, \quad \epsilon = 10^{-8}$$

---

3. Core Formulas & Cheat Sheet

Optimizer	Accumulator / Velocity Update	Parameter Update Rule	Key Advantage / Drawback
Standard GD	—	$\boldsymbol{\theta}*t = \boldsymbol{\theta}*{t-1}$ - \eta \mathbf{g}*t$	Simple; oscillates in ravines
Momentum	$\mathbf{v}*t = \beta \mathbf{v}*{t-1}$ + \eta \mathbf{g}*t$	$\boldsymbol{\theta}*t = \boldsymbol{\theta}*{t-1}$ - \mathbf{v}*t$	Accelerates in flat ravines; may overshoot
NAG	$\mathbf{v}*t = \beta \mathbf{v}*{t-1}$) $ + \eta \nabla \mathcal{L}(\boldsymbol{\theta} - \beta \mathbf{v}*{t-1}	$\boldsymbol{\theta}*t = \boldsymbol{\theta}*{t-1}$ - \mathbf{v}*t$	Look-ahead brakes prevent overshoot
AdaGrad	$\mathbf{v}*t = \mathbf{v}*{t-1}^2$ + \mathbf{g}*t^2$	$\boldsymbol{\theta}*t = \boldsymbol{\theta}*{t-1}$ - \frac{\eta}{\sqrt{\mathbf{v}*t + \epsilon}} \mathbf{g}*t$	Parameter-specific steps; learning rate dies
RMSProp	$\mathbf{v}*t = \beta \mathbf{v}*{t-1}^2$ + (1-\beta)\mathbf{g}*t^2$	$\boldsymbol{\theta}*t = \boldsymbol{\theta}*{t-1}$ - \frac{\eta}{\sqrt{\mathbf{v}*t + \epsilon}} \mathbf{g}*t$	Solves dying LR using exponential moving average
Adam	$\mathbf{m}*t = \beta_1 \mathbf{m}*{t-1}$ + (1-\beta_1)\mathbf{g}*t $ $\mathbf{v}*t = \beta_2 \mathbf{v}*{t-1}^2$ + (1-\beta_2)\mathbf{g}*t^2$	$\boldsymbol{\theta}*t = \boldsymbol{\theta}*{t-1}$ - \frac{\eta}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \hat{\mathbf{m}}_t$	Combines momentum + RMSProp with bias corrections

---

4. Assignment Solutions with Detailed Step-by-Step Explanations

Case Study 1: Delivery Delay Parameter Tuning (Questions 1 to 5)

Context & Data

One-parameter loss function:

$$\mathcal{L}(w) = (w - 4)^2$$

Gradient descent rule:

$$w_{t+1} = w_t - \eta \frac{d\mathcal{L}}{dw}$$

Current state: $w_t = 1$, learning rate $\eta = 0.1$.

Question 1

What is $\frac{d\mathcal{L}}{dw}$ at $w = 1$? - A) 6 - B) -3 - C) 3 - D) -6

Correct Answer: D

- **Detailed Step-by-Step Explanation**:

$$\frac{d\mathcal{L}}{dw} = \frac{d}{dw}(w-4)^2 = 2(w-4)$$

At $w = 1$:

$$\left.\frac{d\mathcal{L}}{dw}\right|_{w=1} = 2(1-4) = 2(-3) = -6$$

Question 2

What is the next value of w? - A) 1.3 - B) 1.6 - C) 0.4 - D) 2.0

Correct Answer: B

- **Detailed Step-by-Step Explanation**:

$$w_{t+1} = w_t - \eta\frac{d\mathcal{L}}{dw} = 1 - 0.1(-6) = 1 + 0.6 = 1.6$$

Question 3

Why does w increase in this update? - A) The learning rate is negative. - B) The loss is already zero. - C) The gradient at $w = 1$ is negative. - D) The update ignores the derivative.

Correct Answer: C

- **Detailed Step-by-Step Explanation**: Gradient descent subtracts the scaled gradient: $-\eta\frac{d\mathcal{L}}{dw}$. Because the gradient is negative (-6), subtracting a negative quantity adds a positive offset ($+0.6$), increasing w toward the minimum at $w = 4$.

Question 4

Which value of w minimizes this loss? - A) 1 - B) 2 - C) 3 - D) 4

Correct Answer: D

- **Detailed Step-by-Step Explanation**: Since $\mathcal{L}(w) = (w-4)^2 \geq 0$, its global minimum occurs where $(w-4)^2 = 0 \implies w = 4$.

Question 5

What is the role of the learning rate η? - A) It controls the update step size. - B) It stores the final target value. - C) It removes the need for a gradient. - D) It makes every loss value equal.

Correct Answer: A

- **Detailed Step-by-Step Explanation**: The learning rate η is a positive scalar hyperparameter that determines how large of a step the parameters take in the direction of the negative gradient.

Case Study 2: Energy Forecast Training Schedule (Questions 6 to 10)

Context & Data

A dataset contains $N = 12$ training records. In one experiment, the mini-batch size is $B = 3$. One epoch means that all 12 records have been processed exactly once.

Question 6

How many mini-batches are processed in one epoch? - A) 3 - B) 4 - C) 6 - D) 12

Correct Answer: B

- **Detailed Step-by-Step Explanation**:

$$\text{Number of mini-batches per epoch} = \frac{\text{Total records}}{\text{Mini-batch size}} = \frac{12}{3} = 4$$

Question 7

Which method uses all 12 records for each parameter update? - A) Stochastic gradient descent - B) Mini-batch gradient descent - C) Batch gradient descent - D) Momentum-only update

Correct Answer: C

- **Detailed Step-by-Step Explanation**: Batch Gradient Descent (BGD) computes the cost function and its gradients over the complete training set (all 12 records) before performing a single parameter update.

Question 8

Which method uses one record per parameter update? - A) Batch gradient descent - B) Adam without gradients or single-example updates - C) Full-batch averaging - D) Stochastic gradient descent

Correct Answer: D

- **Detailed Step-by-Step Explanation**: Stochastic Gradient Descent (SGD) updates parameters sequentially after inspecting each individual training record ($B = 1$).

Question 9

If the mini-batch gradients for four batches are 2, 4, 6, 8, what is their average? - A) 5 - B) 4 - C) 6 - D) 8

Correct Answer: A

- **Detailed Step-by-Step Explanation**:

$$\text{Average gradient} = \frac{2+4+6+8}{4} = \frac{20}{4} = 5$$

Question 10

Why might mini-batch gradient descent be preferred in practice? - A) It never uses gradients. - B) It balances noisy updates and computational efficiency. - C) It requires the loss to be exactly zero before every mini-batch update. - D) It updates only after training is complete.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Mini-batch GD combines the computational advantages of vectorized hardware (GPUs) with the variance reduction of batching, while retaining enough stochasticity to escape shallow saddles.

Case Study 3: Queue Prediction Parameter Update (Questions 11 to 15)

Context & Data

A momentum-based update rule:

$$v_t = \beta v_{t-1} + \eta g_t$$
$$w_t = w_{t-1} - v_t$$

Given: - Previous velocity: $v_{t-1} = 0.5$ - Momentum parameter: $\beta = 0.8$ - Learning rate: $\eta = 0.1$ - Current gradient: $g_t = 3$ - Previous parameter: $w_{t-1} = 5$

Question 11

What is the new velocity v_t? - A) 0.5 - B) 0.6 - C) 0.7 - D) 0.8

Correct Answer: C

- **Detailed Step-by-Step Explanation**:

$$v_t = \beta v_{t-1} + \eta g_t = 0.8(0.5) + 0.1(3) = 0.4 + 0.3 = 0.7$$

Question 12

What is the updated parameter w_t? - A) 4.3 - B) 5.7 - C) 4.7 - D) 5.3

Correct Answer: A

- **Detailed Step-by-Step Explanation**:

$$w_t = w_{t-1} - v_t = 5 - 0.7 = 4.3$$

Question 13

What does the momentum term βv_{t-1} represent? - A) It removes all history from the update and uses only the current gradient. - B) It sets the learning rate to zero for every future update. - C) It replaces the loss function. - D) It carries part of the previous update direction.

Correct Answer: D

- **Detailed Step-by-Step Explanation**: The term βv_{t-1} retains a decayed fraction of the past velocity vector, preserving directional inertia.

Question 14

If $\beta = 0$, what would v_t be for the same gradient and learning rate? - A) 0.3 - B) 0.1 - C) 0.2 - D) 0.4

Correct Answer: A

- **Detailed Step-by-Step Explanation**: With $\beta = 0$:

$$v_t = 0(0.5) + 0.1(3) = 0.3$$

Question 15

Why can momentum help optimization? - A) It makes gradients unnecessary. - B) It always increases the loss. - C) It can smooth updates across steps. - D) It forces all parameters to be equal.

Correct Answer: C

- **Detailed Step-by-Step Explanation**: Momentum averages out high-frequency orthogonal oscillations in narrow valleys while reinforcing consistent descent directions, yielding smoother trajectories.

Case Study 4: Demand Prediction Step-Size Analysis (Questions 16 to 20)

Context & Data

AdaGrad-style adaptive step size for a single parameter:

$$r_t = r_{t-1} + g^2$$

Update magnitude:

$$\Delta = \frac{\eta g}{\sqrt{r_t}}$$

Given: - Current gradient: $g = 4$ - Previous accumulated squared gradient: $r_{t-1} = 9$ - Learning rate: $\eta = 0.5$

Question 16

What is r_t? - A) 16 - B) 25 - C) 13 - D) 9

Correct Answer: B

- **Detailed Step-by-Step Explanation**:

$$r_t = r_{t-1} + g^2 = 9 + 4^2 = 9 + 16 = 25$$

Question 17

What is $\sqrt{r_t}$? - A) 3 - B) 4 - C) 5 - D) 6

Correct Answer: C

- **Detailed Step-by-Step Explanation**:

$$\sqrt{r_t} = \sqrt{25} = 5$$

Question 18

What is the update magnitude? - A) 0.2 - B) 0.4 - C) 0.8 - D) 0.6

Correct Answer: B

- **Detailed Step-by-Step Explanation**:

$$\text{Update magnitude} = \frac{\eta g}{\sqrt{r_t}} = \frac{0.5 \times 4}{5} = \frac{2}{5} = 0.4$$

Question 19

What is the main idea behind AdaGrad-style scaling? - A) It ignores all previous gradient information while claiming to adapt steps. - B) It requires the same update for every parameter. - C) It removes the need for a loss function. - D) It adapts the step size using accumulated squared gradients.

Correct Answer: D

- **Detailed Step-by-Step Explanation**: AdaGrad individually divides each parameter's update by the square root of its cumulative historical squared gradients, automatically performing smaller steps for frequently updated weights and larger steps for infrequent weights.

Question 20

Which optimizer combines momentum-like averaging with adaptive scaling? - A) Adam - B) Plain batch gradient descent - C) Stochastic gradient descent without memory - D) A zero-learning-rate update

Correct Answer: A

- **Detailed Step-by-Step Explanation**: Adam (Adaptive Moment Estimation) explicitly integrates first-moment exponential averaging (like momentum) with second-moment exponential averaging of squared gradients (like RMSProp).

---

5. NPTEL Exam Traps & High-Yield Pitfalls

1. **Epoch vs Iteration**:
2. $\text{Iterations per epoch} = \lceil N/B \rceil$.
3. If $N = 1000$ and $B = 50$, there are 20 iterations per epoch.
4. **Velocity Formula Variations**:
5. In some textbooks, the learning rate η is placed inside velocity: $v_t = \beta v_{t-1} + \eta g_t, w_t = w_{t-1} - v_t$.
6. In others, η is placed in the parameter step: $v_t = \beta v_{t-1} + g_t, w_t = w_{t-1} - \eta v_t$.
7. Always check the given formula in the exam question prompt!
8. **Adam Bias Correction Formula**:
9. The correction is $\hat{m}_t = \frac{m_t}{1-\beta_1^t}$, where t is the exponent (iteration number), not a product $\beta_1 \cdot t$!
10. Since $0 < \beta_1 < 1$, $\beta_1^t \to 0$ as t increases, so the denominator $1 - \beta_1^t \to 1$.

MODULE 5

Week 5: Linear Algebra Foundations — Eigenvalues, Eigenvectors, PCA, and SVD

Week 5: Linear Algebra Foundations — Eigenvalues, Eigenvectors, PCA, and SVD

1. Executive Summary & Exam Roadmap

Week 5 covers the linear algebra principles underpinning dimensionality reduction and representation learning in Deep Learning: 1. **Eigenvalues & Eigenvectors**: Algebraic and geometric definitions ($A\mathbf{u} = \lambda\mathbf{u}$), characteristic equation $\det(A - \lambda I) = 0$, trace and determinant relations. 2. **Eigenvalue Decomposition (Spectral Theorem)**: Diagonalization of symmetric matrices $A = U\Lambda U^\top$ with orthogonal eigenvectors ($U^\top U = I$). 3. **Principal Component Analysis (PCA)**: - Data centering and the empirical covariance matrix $\Sigma = \frac{1}{m} X^\top X$. - **Dual perspectives**: Maximizing projected variance vs. minimizing reconstruction error. - Eigenvectors of Σ as principal directions; fraction of total variance explained. 4. **Singular Value Decomposition (SVD)**: - Factorization $A = U\Sigma V^\top$ for arbitrary rectangular matrices. - Left singular vectors ($AA^\top$), right singular vectors ($A^\top A$), and singular values $\sigma_i = \sqrt{\lambda_i}$. - **Eckart-Young-Mirsky Theorem**: Optimal low-rank matrix approximation.

2. Comprehensive Lecture Notes & Mathematical Foundations

2.1 Eigenvalues and Eigenvectors

Definition

Let $A \in \mathbb{R}^{n \times n}$ be a square matrix. A non-zero vector $\mathbf{u} \in \mathbb{R}^n \setminus \mathbf{0}$ is an **eigenvector** of A corresponding to **eigenvalue** $\lambda \in \mathbb{C}$ if:

$$A\mathbf{u} = \lambda\mathbf{u}$$

Geometric Interpretation

Multiplying a vector $\mathbf{u}$ by matrix A generally changes both its length and its direction. An eigenvector represents a special invariant direction: A merely stretches, shrinks, or reverses $\mathbf{u}$ along its original line of action by a scaling factor of λ. - If $\lambda > 1$: vector is stretched. - If $0 < \lambda < 1$: vector is shrunk. - If $\lambda < 0$: vector direction is reversed. - If $\lambda = 0$: vector is mapped to the null space ($A\mathbf{u} = \mathbf{0}$).

Characteristic Equation

Rewriting $A\mathbf{u} = \lambda\mathbf{u}$:

$$(A - \lambda I)\mathbf{u} = \mathbf{0}$$

For a non-trivial solution $\mathbf{u} \neq \mathbf{0}$ to exist, the matrix $(A - \lambda I)$ must be singular:

$$\det(A - \lambda I) = 0$$

Fundamental Properties

1. **Sum of Eigenvalues equals Trace**:

$$\sum_{i=1}^{n} \lambda_i = \mathrm{Tr}(A) = \sum_{i=1}^{n} A_{ii}$$

2. **Product of Eigenvalues equals Determinant**:

$$\prod_{i=1}^{n} \lambda_i = \det(A)$$

3. **Diagonal Matrices**: For a diagonal matrix $D = \mathrm{diag}(d_1, d_2, \ldots, d_n)$, the eigenvalues are simply the diagonal entries d_i, and the eigenvectors are the standard Cartesian basis vectors $\mathbf{e}_i$.

2.2 Eigenvalue Decomposition & Symmetric Matrices

If A has n linearly independent eigenvectors $\mathbf{u}_1, \ldots, \mathbf{u}_n$ with eigenvalues $\lambda_1, \ldots, \lambda_n$:

$$AU = U\Lambda \implies A = U\Lambda U^{-1}$$

where $U = [\mathbf{u}_1, \mathbf{u}_2, \ldots, \mathbf{u}_n]$ and $\Lambda = \mathrm{diag}(\lambda_1, \lambda_2, \ldots, \lambda_n)$.

The Spectral Theorem for Real Symmetric Matrices

If $A = A^\top \in \mathbb{R}^{n \times n}$: 1. All eigenvalues λ_i are **guaranteed to be real** ($\lambda_i \in \mathbb{R}$). 2. Eigenvectors corresponding to distinct eigenvalues are **mutually orthogonal**: $$\mathbf{u}_i^\top \mathbf{u}*j = 0 \quad (\forall i \neq j)*

$$3. The eigenvector matrix \$U\$ can be chosen to be orthonormal (\$U^\top = U^{-1}\$):$$

A = U |Lambda U^|top = |sum_i^\top$$}^n \lambda_i \mathbf{u}_i \mathbf{u

---

2.3 Principal Component Analysis (PCA)

Data Preprocessing: Centering

Given a dataset of m vectors ${\mathbf{x}_1, \dots, \mathbf{x}*m} |subset |mathbb{R}^d, compute the sample mean |boldsymbol{|mu} = |frac{1}{m}|sum_i . Center the data :$\tilde{\mathbf{x}}_i = \mathbf{x}}^m \mathbf{x*i - |boldsymbol{|mu}*

$$The empirical covariance matrix \$\Sigma \in \mathbb{R}^{d\times d}\$ is:$$

|Sigma = |frac{1}{m} |sum X^\top X$$ where $X \in \mathbb{R}^{m\times d}$ is the mean-centered design matrix. Note that Σ is symmetric and positive semi-definite ($\Sigma = \Sigma^\top, \lambda_i \geq 0$).}^m \tilde{\mathbf{x}}_i \tilde{\mathbf{x}}_i^\top = \frac{1}{m

Perspective 1: Maximizing Variance of Projection

We want to project data onto a unit direction $\mathbf{u}$ ($|\mathbf{u}| = 1$) such that the variance of the projected points is maximized: $$\text{Var}(\mathbf{u}^\top \tilde{\mathbf{x}}) = \frac{1}{m}\sum_{i=1}^m (\mathbf{u}^\top \tilde{\mathbf{x}}*i)^2 = |mathbf{u}^|top |left(|frac{1}{m}|sum}^m \tilde{\mathbf{x}*i |tilde{|mathbf{x}}i^|top |right) |mathbf{u} = |mathbf{u}^|top |Sigma |mathbf{u}*

$$Formulate the Lagrangian with constraint \$\mathbf{u}^\top \mathbf{u} = 1\$:$$

|mathcal{L}(|mathbf{u}, |lambda) = |mathbf{u}^|top |Sigma |mathbf{u} - |lambda (|mathbf{u}^|top |mathbf{u} - 1)

$$Setting the gradient w.r.t \$\mathbf{u}\$ to zero:$$

|nabla

$$The projected variance is:$$

*|mathbf{u}^|top |Sigma |mathbf{u} = |mathbf{u}^|top (|lambda |mathbf{u}) = |lambda |mathbf{u}|^2 = |lambda$$ To maximize variance, $\mathbf{u}$ must be the eigenvector of Σ corresponding to the }} |mathcal{L} = 2|Sigma |mathbf{u} - 2|lambda |mathbf{u} = |mathbf{0} |implies |Sigma |mathbf{u} = |lambda |mathbf{u**largest eigenvalue** $|lambda$.*

Perspective 2: Minimizing Reconstruction Error

Approximating $\tilde{\mathbf{x}}_i$ using k orthonormal basis vectors $\mathbf{u}_1, \dots, \mathbf{u}_k$: $$\hat{\mathbf{x}}*i = |sum}^k (\tilde{\mathbf{x}}_i^\top \mathbf{u}_j)\mathbf{u*j

$$Minimizing the mean squared reconstruction error:$$

|min |frac{1}{m}|sum_i|^2$$ leads mathematically to the exact same solution: pick the eigenvectors corresponding to the k largest eigenvalues of Σ!}^m |\tilde{\mathbf{x}}_i - \hat{\mathbf{x}}

Variance Explained Ratio

$$\text{Fraction of Variance Explained by top } k \text{ components} = \frac{\sum_{i=1}^k \lambda_i}{\sum_{j=1}^d \lambda_j} = \frac{\sum_{i=1}^k \lambda_i}{\text{Tr}(\Sigma)}$$

2.4 Singular Value Decomposition (SVD)

Theorem

Any real matrix $M \in \mathbb{R}^{m\times n}$ of rank $r \leq \min(m, n)$ can be factorized as:

$$M = U\Sigma V^\top$$

where: - $U \in \mathbb{R}^{m\times m}$ is an orthogonal matrix ($U^\top U = I_m$). Its columns are the **left singular vectors**, which are eigenvectors of $MM^\top$. - $V \in \mathbb{R}^{n\times n}$ is an orthogonal matrix ($V^\top V = I_n$). Its columns are the **right singular vectors**, which are eigenvectors of $M^\top M$. - $\Sigma \in \mathbb{R}^{m\times n}$ is a rectangular diagonal matrix containing non-negative **singular values** sorted in descending order:

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0 = \cdots = 0$$

where $\sigma_i = \sqrt{\lambda_i(M^\top M)} = \sqrt{\lambda_i(MM^\top)}$.

Outer Product Expansion

$$M = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$$

Eckart-Young-Mirsky Theorem (Low-Rank Approximation)

The optimal rank-k ($k < r$) approximation M_k that minimizes the Frobenius error $|M - M_k| Is obtained by truncating the SVD to the top k singular values :$M_k = |sum_i^\top*

$$The approximation error is directly given by the discarded singular values:$$

*|M - M_k|}^k \sigma_i \mathbf{u}_i \mathbf{v*F^2 = |sum^r \sigma_j^2$$*

---

3. Core Formulas & Cheat Sheet

Mathematical Concept	Formula	Key Properties
Eigenvalue Equation	$A\mathbf{u} = \lambda\mathbf{u}$	$\det(A - \lambda I) = 0$
Matrix Trace	$\text{Tr}(A) = \sum A_{ii} = \sum \lambda_i$	Sum of diagonal entries = sum of eigenvalues
Matrix Determinant	$\det(A) = \prod \lambda_i$	Product of eigenvalues
Spectral Decomposition	$A = U\Lambda U^\top$	Valid for symmetric $A = A^\top$; $U^\top U = I$
Sample Covariance	$\Sigma = \frac{1}{m} X^\top X$	Data must be zero-centered first
Projected Variance	$\text{Var} = \mathbf{u}^\top \Sigma \mathbf{u} = \lambda$	Maximized along dominant eigenvector
SVD Factorization	$M = U\Sigma V^\top$	U: eigenvectors of $MM^\top$; V: eigenvectors of $M^\top M$
Singular Value Relation	$\sigma_i = \sqrt{\lambda_i(M^\top M)}$	Always non-negative ($\sigma_i \geq 0$)
Truncated SVD	$M_k = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$	Optimal rank-k approximation under Frobenius norm

4. Assignment Solutions with Detailed Step-by-Step Explanations

Case Study 1: Sensor Direction Scaling (Questions 1 to 5)

Context & Data

A diagonal linear transformation:

$$A = [3 \quad 0\,0 \quad 2]$$

Independent sensor directions: $\mathbf{u}_1 = (1, 0)^\top$, $\mathbf{u}_2 = (0, 1)^\top$.

Question 1

What is $A\mathbf{u}_1$? - A) $(3, 0)^\top$ - B) $(0, 3)^\top$ - C) $(2, 0)^\top$ - D) $(1, 3)^\top$

Correct Answer: A

- **Detailed Step-by-Step Explanation**:

$$A\mathbf{u}_1 = [3 \quad 0\,0 \quad 2]\,[1\ 0] = [3(1) + 0(0)\ 0(1) + 2(0)] = [3\ 0] = (3, 0)^\top$$

Question 2

What eigenvalue corresponds to $\mathbf{u}_2$? - A) 1 - B) 3 - C) 2 - D) 0

Correct Answer: C

- **Detailed Step-by-Step Explanation**:

$$A\mathbf{u}_2 = [3 \quad 0\,0 \quad 2]\,[0\ 1] = [0\ 2] = 2\,[0\ 1] = 2\mathbf{u}_2$$

  Since $A\mathbf{u}_2 = \lambda_2\mathbf{u}_2$, the eigenvalue is $\lambda_2 = 2$.

Question 3

Which statement is true about $\mathbf{u}_1$? - A) It is not changed by A for any input scale. - B) It is rotated by 90 degrees after multiplication by A. - C) It becomes the zero vector. - D) It is scaled by a factor of 3.

Correct Answer: D

- **Detailed Step-by-Step Explanation**: $A\mathbf{u}_1 = (3, 0)^\top = 3\mathbf{u}_1$. Its direction is completely preserved, and its length is scaled by 3.

Question 4

What is the trace of A? - A) 6 - B) 5 - C) 1 - D) 3

Correct Answer: B

- **Detailed Step-by-Step Explanation**: The trace of a matrix is the sum of its diagonal entries:

$$\mathrm{Tr}(A) = 3 + 2 = 5$$

(Also equals $\lambda_1 + \lambda_2 = 3 + 2 = 5$).

Question 5

How are the eigenvalues related to the diagonal entries in this matrix? - A) They are equal to the diagonal entries for this diagonal matrix. - B) They must both be zero for every diagonal matrix in this setting. - C) They are found by adding all entries in each row. - D) They depend only on the off-diagonal entries here.

Correct Answer: A

- **Detailed Step-by-Step Explanation**: For any diagonal matrix $D = \mathrm{diag}(d_1, \ldots, d_n)$, the characteristic equation is $\det(D - \lambda I) = \prod_{i=1}^{n}(d_i - \lambda) = 0$, so the eigenvalues are identically equal to the diagonal entries.

Case Study 2: Customer Feature Coordinate Review (Questions 6 to 10)

Context & Data

Basis vectors in $\mathbb{R}^2$:

$$\mathbf{b}_1 = (1, 1)^\top, \quad \mathbf{b}_2 = (1, -1)^\top$$

Customer feature vector:

$$\mathbf{x} = (4, 2)^\top$$

Coordinate representation:

$$\mathbf{x} = c_1\mathbf{b}_1 + c_2\mathbf{b}_2$$

Question 6

Which equation is obtained from the first coordinate? - A) $c_1 - c_2 = 4$ - B) $c_1 + c_2 = 4$ - C) $2c_1 = 4$ - D) $c_2 = 4$

Correct Answer: B

- **Detailed Step-by-Step Explanation**:

$$c_1\,[1\ 1] + c_2\,[1\ -1] = [c_1 + c_2\ c_1 - c_2] = [4\ 2]$$

Equating the first coordinate yields: $c_1 + c_2 = 4$.

Question 7

Which equation is obtained from the second coordinate? - A) $c_1 + c_2 = 2$ - B) $c_1 = 2$ - C) $c_1 - c_2 = 2$ - D) $2c_2 = 2$

Correct Answer: C

- **Detailed Step-by-Step Explanation**: Equating the second coordinate yields: $c_1 - c_2 = 2$.

Question 8

What is c_1? - A) 3 - B) 1 - C) 4 - D) 2

Correct Answer: A

- **Detailed Step-by-Step Explanation**: Add the two linear equations:

$$(c_1 + c_2) + (c_1 - c_2) = 4 + 2 \implies 2c_1 = 6 \implies c_1 = 3$$

Question 9

What is c_2? - A) 0 - B) 1 - C) 3 - D) 2

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Substitute $c_1 = 3$ into $c_1 + c_2 = 4$:

$$3 + c_2 = 4 \implies c_2 = 1$$

Question 10

Why do $\mathbf{b}_1$ and $\mathbf{b}_2$ form a basis for $\mathbb{R}^2$? - A) They point in the same direction. - B) They both have zero length. - C) They always produce only positive coordinates. - D) They are linearly independent and span $\mathbb{R}^2$.

Correct Answer: D

- **Detailed Step-by-Step Explanation**: Checking dot product: $\mathbf{b}_1^\top \mathbf{b}_2 = (1)(1) + (1)(-1) = 0$. Since they are non-zero and orthogonal, they are linearly independent. Any set of 2 linearly independent vectors in $\mathbb{R}^2$ spans $\mathbb{R}^2$ and forms a basis.

Case Study 3: Store Performance Feature Reduction (Questions 11 to 15)

Context & Data

Centered covariance matrix:

$$\Sigma = [4 \quad 0\,0 \quad 1]$$

Question 11

What is the variance along the first coordinate direction? - A) 1 - B) 2 - C) 4 - D) 0

Correct Answer: C

- **Detailed Step-by-Step Explanation**: The diagonal entries of a covariance matrix represent the individual feature variances. For feature 1, the variance is $\Sigma_{11} = 4$.

Question 12

Which direction is the first principal component? - A) The second coordinate direction - B) The direction $(1,1)^\top$ - C) The zero vector direction - D) The first coordinate direction

Correct Answer: D

- **Detailed Step-by-Step Explanation**: The first principal component corresponds to the eigenvector with the largest eigenvalue. Here $\lambda_1 = 4 > \lambda_2 = 1$. The eigenvector corresponding to $\lambda_1 = 4$ is $\mathbf{e}_1 = (1,0)^\top$, which is the first coordinate direction.

Question 13

What fraction of total variance is explained by the first principal component? - A) 4/5 - B) 1/5 - C) 1/4 - D) 5/4

Correct Answer: A

- **Detailed Step-by-Step Explanation**: Total variance $= \mathrm{Tr}(\Sigma) = 4 + 1 = 5$. Variance explained by component $1 = 4$.

$$\text{Fraction} = \frac{4}{4+1} = \frac{4}{5} = 80\%$$

Question 14

If the data is projected onto the first principal component only, how many dimensions remain? - A) 0 - B) 1 - C) 2 - D) 4

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Projecting d-dimensional data onto a single 1D line defined by the first principal component yields a 1-dimensional scalar representation.

Question 15

What is the purpose of centering data before PCA? - A) To make every covariance value negative after centering. - B) To remove all variance from the dataset before PCA. - C) To force all eigenvalues to be equal. - D) To express variation relative to the mean.

Correct Answer: D

- **Detailed Step-by-Step Explanation**: Centering shifts the coordinate origin to the centroid (sample mean) of the data cloud. This ensures that the covariance matrix measures true spread and dispersion about the mean, rather than being distorted by the offset from the origin.

Case Study 4: Product-User Interaction Compression (Questions 16 to 20)

Context & Data

Interaction matrix:

$$M = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$$

SVD form: $M = U\Sigma V^\top$ with singular values $\sigma_1 = 3, \sigma_2 = 1$.

Question 16

What is the largest singular value? - A) 1 - B) 2 - C) 3 - D) 0

Correct Answer: C

- **Detailed Step-by-Step Explanation**: The singular values are $\sigma_1 = 3$ and $\sigma_2 = 1$. The largest singular value is 3.

Question 17

What does $V^\top$ represent in the SVD expression? - A) The transpose of the right singular-vector matrix. - B) The loss value of the recommendation model after a full training step. - C) The number of users in every batch. - D) The covariance matrix after centering.

Correct Answer: A

- **Detailed Step-by-Step Explanation**: In $M = U\Sigma V^\top$, V is the orthogonal matrix of right singular vectors (eigenvectors of $M^\top M$). Thus, $V^\top$ is the transpose of the right singular-vector matrix.

Question 18

What rank-one approximation keeps only the largest singular value? - A) $\begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$ - B) $\begin{bmatrix} 3 & 0 \\ 0 & 0 \end{bmatrix}$ - C) $\begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$ - D) $\begin{bmatrix} 0 & 3 \\ 1 & 0 \end{bmatrix}$

Correct Answer: B

- **Detailed Step-by-Step Explanation**: By the Eckart-Young theorem, the best rank-1 approximation sets all singular values except the largest to zero:

$$M_1 = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^\top = 3 \begin{bmatrix} 1 \\ 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \end{bmatrix} = \begin{bmatrix} 3 & 0 \\ 0 & 0 \end{bmatrix}$$

Question 19

What is the rank of M? - A) 0 - B) 1 - C) 2 - D) 3

Correct Answer: C

- **Detailed Step-by-Step Explanation**: The rank of a matrix equals the number of strictly non-zero singular values. Since both $\sigma_1 = 3$ and $\sigma_2 = 1$ are non-zero, $\text{rank}(M) = 2$.

Question 20

Why is SVD useful for dimensionality reduction? - A) It requires every matrix entry to be exactly zero before the factorization. - B) It removes the need for any matrix factorization. - C) It changes all singular values to negative values. - D) It can keep dominant singular directions and discard smaller ones.

Correct Answer: D

- **Detailed Step-by-Step Explanation**: SVD decomposes a matrix into a sum of rank-1 matrices weighted by their singular values. Discarding smaller singular values filters out high-frequency noise and compresses the data while preserving the dominant geometric variation.

---

5. NPTEL Exam Traps & High-Yield Pitfalls

---

1. **Covariance Matrix Symmetry**:
2. Covariance matrix Σ is **always symmetric** ($\Sigma = \Sigma^\top$) and **positive semi-definite** ($\mathbf{x}^\top \Sigma \mathbf{x} \geq 0$). All eigenvalues of Σ are ≥ 0.
3. **Singular Values vs Eigenvalues**:
4. Singular values σ_i are **always real and non-negative** ($\sigma_i \geq 0$).
5. Eigenvalues λ_i of a general matrix can be negative or complex.
6. For any matrix M, $\sigma_i(M) = \sqrt{\lambda_i(M^\top M)}$.
7. **Fraction of Variance Formula**:
8. $\frac{\lambda_1}{\lambda_1 + \lambda_2 + \cdots + \lambda_d}$. Be careful not to square the eigenvalues when calculating variance explained in PCA (the eigenvalues of Σ *already* represent variances).

MODULE 6

Week 6: Autoencoders, Relation to PCA, and Regularized Architectures

Week 6: Autoencoders, Relation to PCA, and Regularized Architectures

1. Executive Summary & Exam Roadmap

Week 6 shifts the focus from supervised learning to unsupervised feature learning via **Autoencoders**: 1. **Autoencoder Fundamentals**: Encoder function $f(\mathbf{x})$, bottleneck latent representation $\mathbf{h}$, decoder function $g(\mathbf{h})$, reconstruction loss $\mathcal{L}(\mathbf{x}, \hat{\mathbf{x}})$, and tied weights ($\mathbf{W}' = \mathbf{W}^\top$). 2. **Autoencoders vs. PCA**: The Bourlard & Kamp (1988) theorem proving that a linear autoencoder with squared error loss spans the exact same k-dimensional subspace as the top k principal components of PCA. 3. **Undercomplete vs. Overcomplete Autoencoders**: - Undercomplete ($k < d$): Imposes an architectural bottleneck. - Overcomplete ($k > d$): Can memorize the identity mapping without regularization. 4. **Regularized Autoencoders**: - **Sparse Autoencoders**: KL divergence and L_1 penalties enforcing low average neuronal activation. - **Denoising Autoencoders (DAE)**: Reconstructing clean inputs $\mathbf{x}$ from stochastically corrupted inputs $\tilde{\mathbf{x}}$ (manifold learning). - **Contractive Autoencoders (CAE)**: Penalizing the Frobenius norm of the encoder's Jacobian matrix $|J_f(\mathbf{x})|_F^2$ to guarantee local invariance.

2. Comprehensive Lecture Notes & Mathematical Foundations

2.1 Basic Autoencoder Architecture

An autoencoder is a feedforward neural network trained to reconstruct its own input in an unsupervised manner:

```
Input x (d-dim) ---> [ Encoder: W, b ] ---> Latent Code h (k-dim) ---> [ Decoder: W', b' ] ---> Output x_hat (d-dim)
```

The Encoder

Maps an input vector $\mathbf{x} \in \mathbb{R}^d$ to a latent code $\mathbf{h} \in \mathbb{R}^k$:

$$\mathbf{h} = f(\mathbf{x}) = s_f(\mathbf{W}\mathbf{x} + \mathbf{b})$$

where $\mathbf{W} \in \mathbb{R}^{k \times d}$, $\mathbf{b} \in \mathbb{R}^k$, and s_f is an activation function (e.g., sigmoid, tanh, or identity).

The Decoder

Maps the latent code $\mathbf{h}$ back to the input space as a reconstruction $\hat{\mathbf{x}} \in \mathbb{R}^d$:

$$\hat{\mathbf{x}} = g(\mathbf{h}) = s_g(\mathbf{W}'\mathbf{h} + \mathbf{b}')$$

where $\mathbf{W}' \in \mathbb{R}^{d \times k}$, $\mathbf{b}' \in \mathbb{R}^d$, and s_g is the decoder activation function.

Tied Weights

To halve the number of parameters and regularize the model, the decoder weights are often "tied" to the encoder weights:

$$\mathbf{W}' = \mathbf{W}^\top$$

Reconstruction Loss Functions

1. **Mean Squared Error (for real-valued continuous inputs)**:

$$\mathcal{L}(\mathbf{x}, \hat{\mathbf{x}}) = \frac{1}{2}|\mathbf{x} - \hat{\mathbf{x}}|^2 = \frac{1}{2}\sum_{j=1}^{d}(x_j - \hat{x}_j)^2$$

2. **Binary Cross-Entropy (for normalized inputs $x_j \in [0, 1]$)**:

$$\mathcal{L}(\mathbf{x}, \hat{\mathbf{x}}) = -\sum_{j=1}^{d}\left[x_j \log \hat{x}_j + (1 - x_j)\log(1 - \hat{x}_j)\right]$$

2.2 Deep Link: Linear Autoencoders and PCA

The Bourlard & Kamp Theorem (1988)

Consider an undercomplete autoencoder where: 1. Both encoder and decoder activation functions are strictly **linear (identity)**:

$$\mathbf{h} = \mathbf{W}\mathbf{x}, \quad \hat{\mathbf{x}} = \mathbf{W}'\mathbf{h} = \mathbf{W}'\mathbf{W}\mathbf{x}$$

2. The loss function is the mean squared reconstruction error:

$$\min_{\mathbf{W}, \mathbf{W}'} \frac{1}{N}\sum_{i=1}^{N}|\mathbf{x}_i - \mathbf{W}'\mathbf{W}\mathbf{x}_i|^2$$

Theorem Result: - The global minimum of this objective function is achieved when the k-dimensional latent space spanned by the columns of $\mathbf{W}'$ (or rows of $\mathbf{W}$) is **identical to the subspace spanned by the top k principal eigenvectors of the data covariance matrix Σ**.

Key Differences Between Linear Autoencoders and PCA:

Property	PCA	Linear Autoencoder
Orthogonality	Projection vectors $\mathbf{u}_i$ are strictly orthogonal ($\mathbf{u}_i^\top \mathbf{u}_j = 0$)	Weights are not inherently orthogonal unless explicitly constrained
Ordering	Components are strictly sorted in descending order of explained variance	Latent dimensions have no inherent ordering
Uniqueness	Unique solution (up to sign flips)	Infinite equivalent solutions (any rotation within the k-dimensional subspace)
Optimization	Closed-form solution via SVD / Eigendecomposition	Iterative numerical optimization via Gradient Descent
Nonlinear Extensions	Requires kernel tricks (Kernel PCA)	Trivial: simply add nonlinear activation functions (σ, ReLU) and deeper layers!

2.3 Undercomplete vs. Overcomplete Autoencoders

- **Undercomplete Autoencoder ($k < d$)**: The hidden layer dimension is strictly less than the input dimension. The network cannot trivially copy inputs and is forced to extract the most informative, low-dimensional manifold.
- **Overcomplete Autoencoder ($k > d$)**: The hidden dimension is larger than the input dimension. Without constraints, the network can simply learn the identity function $\hat{\mathbf{x}} = \mathbf{x}$ (e.g., storing the input in the first d neurons and ignoring the rest), failing to learn any meaningful representation. To prevent this trivial memorization, we add **regularization penalties**.

2.4 Regularized Autoencoders

1. Sparse Autoencoders

A sparse autoencoder allows $k \geq d$, but penalizes the network if too many hidden neurons fire simultaneously. Let $a_j^{(i)} = h_j(\mathbf{x}^{(i)})$ be the activation of hidden unit j on sample $\mathbf{x}^{(i)}$. The average activation of neuron j over m training samples is: $$\hat{\rho}*j = |frac{1}{m} |sum$$ We enforce $\hat{\rho}_j \approx \rho$, where ρ is a tiny target sparsity parameter (e.g., $\rho = 0.05$ or 0.01).}^m a_j^{(i)

KL Divergence Sparsity Penalty: The difference between the Bernoulli distribution $\text{Bernoulli}(\rho)$ and $\text{Bernoulli}(\hat{\rho}*j)ispenalized :$|Omega}} = \sum_{j=1}^k \text{KL}(\rho | \hat{\rho*j) = |sum}^k \left[\rho \log \frac{\rho}{\hat{\rho*j} + (1 - |rho)|log |frac{1 - |rho}{1 - |hat{|rho}j} |right]

$$Totalobjective :$$

|mathcal{L}}} = |mathcal{L$$}} + \beta \, \Omega_{\text{sparse}} + \lambda \, \Omega_{\text{weights}

2. Denoising Autoencoders (DAE) (Vincent et al., 2008)

Instead of feeding the clean input $\mathbf{x}$, the DAE is fed a corrupted version $\tilde{\mathbf{x}} \sim q_D(\tilde{\mathbf{x}}|\mathbf{x})$. Common corruption processes: - Additive Gaussian noise: $\tilde{\mathbf{x}} = \mathbf{x} + \mathcal{N}(\mathbf{0}, \sigma^2 I)$. - Masking noise: Randomly setting a fraction ν of coordinates to 0. - Salt-and-pepper noise.

Objective: $$\mathcal{L}*{|text{DAE}} = |mathbb{E}})) \right]$$ Notice that the reconstruction loss is calculated against the } \sim p_{\text{data}}, \tilde{\mathbf{x}} \sim q_D(\tilde{\mathbf{x}}|\mathbf{x})} \left[\mathcal{L}(\mathbf{x}, g(f(\tilde{\mathbf{x}}**original, clean vector** $\mathbf{x}$, NOT the corrupted vector $\tilde{\mathbf{x}}$!

Geometric / Manifold Learning Intuition: Data points in high-dimensional space concentrate near a lower-dimensional manifold. Adding noise pushes the point orthogonal to the manifold. By learning to reconstruct the clean point, the DAE learns a vector field that projects off-manifold corrupted points directly back onto the true data manifold!

3. Contractive Autoencoders (CAE) (Rifai et al., 2011)

The CAE encourages the learned representation $\mathbf{h} = f(\mathbf{x})$ to be robust to infinitesimal input perturbations by penalizing the squared Frobenius norm of the encoder's Jacobian matrix $J_f(\mathbf{x})$: $$\Omega_{\text{contractive}} = |J_f(\mathbf{x})|F^2 = |sum \right)^2$$ For a sigmoid encoder $h_i = \sigma(\mathbf{w}}^k \sum_{j=1}^d \left(\frac{\partial h_i}{\partial x_j*{|top |mathbf{x} + b_i}:$|frac{|partial h_i}{|partial x_j} = h_i (1 - h_i) W

|J_f(\mathbf{x})|F^2 = |sum^2

$$Totalloss :$$

\mathcal{L}}^k \left(h_i (1 - h_i) \right)^2 \sum_{j=1}^d W_{ij*{|text{CAE}} = |mathcal{L}}|_F^2$$ - The contractive penalty flattens the mapping, pulling variations toward zero. - The reconstruction loss resists this flattening to preserve signal information. - Result: The representation is sensitive only along the directions of the data manifold (where variation is necessary for reconstruction) and flat/invariant orthogonal to the manifold.}} + \lambda \, |J_f(\mathbf{x

3. Core Formulas & Cheat Sheet

Architecture	Forward Equations	Objective Function	Key Mechanism
Undercomplete AE	$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$ $\hat{\mathbf{x}} = \sigma(\mathbf{W}'\mathbf{h} + \mathbf{b}')$	$\min \lvert\mathbf{x} - \hat{\mathbf{x}}\rvert^2$	Latent dim $k < d$ forces dimensionality compression
Linear AE	$\mathbf{h} = \mathbf{W}\mathbf{x}, \hat{\mathbf{x}} = \mathbf{W}'\mathbf{h}$	$\min \lvert\mathbf{x} - \mathbf{W}'\mathbf{W}\mathbf{x}\rvert^2$	Spans same subspace as PCA
Sparse AE	$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$	$\mathcal{L}_{\text{rec}} + \beta \sum_j \mathrm{KL}(\rho \lvert \hat{\rho}_j)$	Prevents memorization in overcomplete ($k > d$) networks
Denoising AE	$\tilde{\mathbf{x}} \sim q(\tilde{\mathbf{x}} \lvert \mathbf{x})$ $\hat{\mathbf{x}} = g(f(\tilde{\mathbf{x}}))$	$\mathcal{L}(\mathbf{x}, \hat{\mathbf{x}}) = \lvert\mathbf{x} - g(f(\tilde{\mathbf{x}}))\rvert^2$	Input is corrupted $\tilde{\mathbf{x}}$; target is clean $\mathbf{x}$
Contractive AE	$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$	$\mathcal{L}_{\text{rec}} + \lambda \sum_{ij} \left(\frac{\partial h_i}{\partial x_j}	Penalizes sensitivity via Frobenius norm of Jacobian

4. Assignment Solutions with Detailed Step-by-Step Explanations

Case Study 1: Machine Reading Compression (Questions 1 to 5)

Context & Data

A two-feature reading $\mathbf{x} = (4, 2)^\top$. Encoder:

$$z = 0.5x_1 + 0.5x_2$$

Decoder:

$$\hat{x}_1 = z, \quad \hat{x}_2 = z$$

Reconstruction loss:

$$\mathcal{L} = (x_1 - \hat{x}_1)^2 + (x_2 - \hat{x}_2)^2$$

Question 1

What is the latent value z? - A) 3 - B) 2 - C) 4 - D) 1

Correct Answer: A

- **Detailed Step-by-Step Explanation**:

$$z = 0.5(4) + 0.5(2) = 2.0 + 1.0 = 3$$

Question 2

What is the reconstructed vector $\hat{\mathbf{x}}$? - A) $(2, 2)^\top$ - B) $(3, 3)^\top$ - C) $(4, 4)^\top$ - D) $(3, 2)^\top$

Correct Answer: B

- **Detailed Step-by-Step Explanation**: The decoder sets both output components equal to z:

$$\hat{\mathbf{x}} = (\hat{x}_1, \hat{x}_2)^\top = (z, z)^\top = (3, 3)^\top$$

Question 3

What is the reconstruction loss? - A) 4 - B) 1 - C) 2 - D) 0

Correct Answer: C

- **Detailed Step-by-Step Explanation**:

$$\mathcal{L} = (x_1 - \hat{x}_1)^2 + (x_2 - \hat{x}_2)^2 = (4-3)^2 + (2-3)^2 = 1^2 + (-1)^2 = 1 + 1 = 2$$

Question 4

Which part of the autoencoder creates the latent representation? - A) The decoder output layer - B) The reconstruction loss - C) The target label table - D) The encoder layer

Correct Answer: D

- **Detailed Step-by-Step Explanation**: By architectural definition, the **encoder** maps the high-dimensional input vector into the compressed bottleneck/latent representation.

Question 5

What is the training objective of this autoencoder? - A) To minimize reconstruction error. - B) To assign external class labels before any reconstruction is attempted. - C) To ignore the input features. - D) To make every latent value zero.

Correct Answer: A

- **Detailed Step-by-Step Explanation**: Autoencoders are self-supervised models whose primary loss function optimizes the fidelity of the reconstructed output relative to the input.

Case Study 2: Report Feature Compression (Questions 6 to 10)

Context & Data

A document system uses $d = 5$ input features, compresses them into $k = 2$ latent hidden units, and reconstructs the 5 features. The model is undercomplete.

Question 6

What is the input dimension? - A) 2 - B) 5 - C) 7 - D) 10

Correct Answer: B

- **Detailed Step-by-Step Explanation**: There are 5 input features, so the input dimension is $d = 5$.

Question 7

What is the latent dimension? - A) 1 - B) 5 - C) 2 - D) 7

Correct Answer: C

- **Detailed Step-by-Step Explanation**: The latent bottleneck contains 2 hidden units, so the latent dimension is $k = 2$.

Question 8

Why is the model undercomplete? - A) The decoder has no reconstruction target. - B) The output layer ignores all reconstructed features during training. - C) The input has fewer features than the latent layer. - D) The latent dimension is smaller than the input dimension.

Correct Answer: D

- **Detailed Step-by-Step Explanation**: An autoencoder is formally classified as **undercomplete** when its latent bottleneck dimension is strictly less than its input dimension ($k < d$, here $2 < 5$).

Question 9

Which output dimension is required to reconstruct the original input? - A) 5 - B) 1 - C) 2 - D) 3

Correct Answer: A

- **Detailed Step-by-Step Explanation**: The decoder's output must reconstruct the input $\mathbf{x} \in \mathbb{R}^5$, so the output dimension must identically equal the input dimension, which is 5.

Question 10

How is this related to PCA in the linear case? - A) It must always produce a one-hot class label. - B) It can learn a low-dimensional representation similar to PCA. - C) It removes the need for reconstruction in every training example. - D) It makes every component have zero variance.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: When an undercomplete autoencoder employs linear activations with squared reconstruction error, the hidden layer spans the exact same optimal low-dimensional subspace as Principal Component Analysis (PCA).

Case Study 3: Meter Reading Recovery (Questions 11 to 15)

Context & Data

Denoising autoencoder: - Clean reading: $\mathbf{x} = (5, 1)^\top$ - Corrupted reading: $\tilde{\mathbf{x}} = (5, 0)^\top$ - Network output: $\hat{\mathbf{x}} = (4, 1)^\top$ - Loss: $\mathcal{L} = (x_1 - \hat{x}_1)^2 + (x_2 - \hat{x}_2)^2$

Question 11

Which vector is given to the encoder during training? - A) $\mathbf{x} = (5, 1)^\top$ - B) $\hat{\mathbf{x}} = (4, 1)^\top$ - C) $(0, 0)^\top$ - D) $\tilde{\mathbf{x}} = (5, 0)^\top$

Correct Answer: D

- **Detailed Step-by-Step Explanation**: In a Denoising Autoencoder (DAE), the encoder receives the stochastically corrupted vector $\tilde{\mathbf{x}} = (5, 0)^\top$ as input.

Question 12

Which vector is the clean reconstruction target? - A) $(4, 1)^\top$ - B) $(5, 0)^\top$ - C) $(5, 1)^\top$ - D) $(0, 1)^\top$

Correct Answer: C

- **Detailed Step-by-Step Explanation**: The loss in a DAE is measured by comparing the reconstructed output $\hat{\mathbf{x}}$ against the original **clean** target $\mathbf{x} = (5, 1)^\top$.

Question 13

What is the reconstruction loss? - A) 0 - B) 2 - C) 4 - D) 1

Correct Answer: D

- **Detailed Step-by-Step Explanation**:

$$\mathcal{L} = (x_1 - \hat{x}_1)^2 + (x_2 - \hat{x}_2)^2 = (5 - 4)^2 + (1 - 1)^2 = 1^2 + 0^2 = 1$$

Question 14

What is the purpose of using corrupted inputs? - A) To train the model to recover clean structure. - B) To remove the decoder from the network before training begins. - C) To force every output to equal zero. - D) To avoid computing any reconstruction loss.

Correct Answer: A

- **Detailed Step-by-Step Explanation**: Feeding corrupted data while targeting clean data forces the model to learn the true underlying data distribution and project noisy off-manifold points back onto the manifold.

Question 15

Which statement best describes this training setup? - A) The model ignores all input values. - B) The model maps noisy input toward a clean target. - C) The target is the corrupted vector only. - D) The encoder output is used as the final label for every noisy example.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: The training setup maps noisy, corrupted patterns $\tilde{\mathbf{x}}$ toward their clean, ground-truth targets $\mathbf{x}$.

Case Study 4: Compact Representation Control (Questions 16 to 20)

Context & Data

Base reconstruction loss: $\mathcal{L}_{\text{rec}} = 0.8$. Sparsity penalty: 0.3 with weighting hyperparameter $\lambda = 2$. Total loss: $\mathcal{L} + \lambda(0.3)$$ = \mathcal{L}_{\text{rec}

Question 16

What is the sparsity penalty contribution? - A) 0.3 - B) 0.8 - C) 0.6 - D) 1.1

Correct Answer: C

- **Detailed Step-by-Step Explanation**:

$$\text{Penalty contribution} = \lambda \times (\text{raw penalty}) = 2 \times 0.3 = 0.6$$

Question 17

What is $\mathcal{L}_{\text{sparse}}$? - A) 1.4 - B) 1.1 - C) 0.8 - D) 0.6

Correct Answer: A

- **Detailed Step-by-Step Explanation**: $$\mathcal{L}_{\text{sparse}} = \mathcal{L} = 0.8 + 0.6 = 1.4$$ + \text{Penalty contribution

---

Question 18

What does sparse regularization encourage? - A) Every hidden unit must always be active during the full reconstruction process. - B) The decoder is removed from the model. - C) Only a small number of hidden units should be active. - D) The input dimension must equal zero.

Correct Answer: C

- **Detailed Step-by-Step Explanation**: Sparsity regularization forces the vast majority of hidden neurons to remain inactive or near zero, ensuring that only a small, specialized subset of features fires for any given input.

Question 19

What is the goal of a contractive penalty? - A) To force reconstruction loss to be ignored throughout all training. - B) To make all weights increase without bound after every input change. - C) To replace the encoder with class labels. - D) To make the representation less sensitive to small input changes.

Correct Answer: D

- **Detailed Step-by-Step Explanation**: A contractive penalty penalizes the Frobenius norm of the Jacobian $\sum_{i,j}(\partial h_i/\partial x_j)^2$. This minimizes the derivative of the encoder outputs with respect to the inputs, ensuring that small perturbations in $\mathbf{x}$ do not cause large fluctuations in the hidden code $\mathbf{h}$.

Question 20

Why are such penalties added to the reconstruction loss? - A) To remove all training data from the objective. - B) To control the learned representation beyond plain reconstruction. - C) To stop the model from producing reconstructions. - D) To make every feature identical to every other feature before encoding.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Plain reconstruction alone can lead to trivial identity mappings (especially in overcomplete networks). Regularization terms (sparsity, contractive, denoising) impose inductive biases that shape the geometry and semantics of the latent manifold.

---

5. NPTEL Exam Traps & High-Yield Pitfalls

1. **Target in Denoising Autoencoder**:
2. The target for computing loss is **ALWAYS the clean** $\mathbf{x}$, NOT the corrupted $\tilde{\mathbf{x}}$. The input is $\tilde{\mathbf{x}}$, and the target is $\mathbf{x}$.
3. **Contractive vs Denoising**:
4. DAE achieves robustness globally through stochastic sampling of corrupted points.
5. CAE achieves robustness analytically and locally through the Jacobian Frobenius norm penalty.
6. **Linear Autoencoder vs PCA Subspace**:
7. A linear AE learns the *same subspace* as PCA, but **not necessarily the same orthogonal eigenvectors**. Any rotated basis of the PCA subspace yields the exact same minimum reconstruction error.

MODULE 7

Week 7: Regularization Techniques — Bias-Variance, L2/L1, Early Stopping, and Dropout

Week 7: Regularization Techniques — Bias-Variance Tradeoff, L2/L1, Early Stopping, Data Augmentation, Ensembles, and Dropout

1. Executive Summary & Exam Roadmap

Week 7 tackles the central challenge of statistical machine learning: **Generalization**: 1. **Bias-Variance Decomposition**: Mathematical derivation of expected test error: $\mathbb{E}[(y-\hat{f}(x))^2] = \text{Bias}^2 + \text{Variance} + \sigma^2$. 2. L_2 **Regularization (Weight Decay)**: Mathematical formulation, gradient step modification $(1-\eta\lambda)w$, and geometric shrinkage along Hessian eigenvectors. 3. L_1 **Regularization (Lasso)**: Subgradient derivation, diamond-shaped constraint geometry, and sparse feature selection. 4. **Early Stopping**: Using a validation set to halt gradient descent at minimal validation loss; equivalence to implicit L_2 regularization. 5. **Data Augmentation & Noise Injection**: Invariance engineering and injecting noise into inputs, weights, or labels. 6. **Ensemble Methods & Bagging**: Why averaging M independent estimators cuts variance by up to $\frac{1}{M}$. 7. **Dropout**: Training-time stochastic masking, inverted dropout scaling $(1/p)$, preventing feature co-adaptation, and interpreting dropout as an implicit ensemble of 2^H sub-networks.

2. Comprehensive Lecture Notes & Mathematical Foundations

2.1 The Bias-Variance Decomposition

Assume the true data-generating process is:

$$y = f(\mathbf{x}) + \epsilon, \quad \text{where } \mathbb{E}[\epsilon] = 0, \text{Var}(\epsilon) = \sigma^2$$

Given a training set $\mathcal{D}$, we fit an estimator $\hat{f}(\mathbf{x}; \mathcal{D})$. We evaluate the expected squared prediction error at a fixed query point $\mathbf{x}$ across all possible training datasets $\mathcal{D}$:

$$\mathbb{E}_{\mathcal{D},\epsilon}\left[(y - \hat{f}(\mathbf{x}))^2\right]$$

Mathematical Derivation:

Add and subtract $\mathbb{E}[\hat{f}(\mathbf{x})]$ inside the expectation:

$$y - \hat{f}(\mathbf{x}) = (f(\mathbf{x}) + \epsilon) - \hat{f}(\mathbf{x}) = \left(f(\mathbf{x}) - \mathbb{E}[\hat{f}(\mathbf{x})]\right) + \left(\mathbb{E}[\hat{f}(\mathbf{x})] - \hat{f}(\mathbf{x})\right) + \epsilon$$

Squaring and expanding the three terms: 1. $\left(f(\mathbf{x}) - \mathbb{E}[\hat{f}(\mathbf{x})]\right)^2$: Deterministic w.r.t $\mathcal{D}$ and ϵ. This is the Bias^2. 2. $\mathbb{E}{|mathcal{D}}|left[|left(|hat{f}(|mathbf{x}) - |mathbb{E}[|hat{f}(|mathbf{x})] |right)^2 |right]$: This is the Variance. 3. $|mathbb{E}[\epsilon^2] = \sigma^2$: The **Irreducible Error**. 4. All cross-product terms evaluate to zero because $\mathbb{E}[\epsilon] = 0$ and $\mathbb{E}_{\mathcal{D}}[\hat{f}(\mathbf{x}) - \mathbb{E}[\hat{f}(\mathbf{x})]] = 0$.

Therefore: $$\mathbb{E}\left[(y - \hat{f}(\mathbf{x}))^2 \right] = \underbrace{\left(f(\mathbf{x}) - \mathbb{E}[\hat{f}(\mathbf{x})] \right)^2}{|text{Bias}^2} + |underbrace{|mathbb{E}|left[(|hat{f}(|mathbf{x}) - |mathbb{E}[|hat{f}(|mathbf{x})])^2 |right]}$$ + \underbrace{\sigma^2}_{\text{Irreducible Error}}

- **High Bias (Underfitting)**: Model is overly simplistic (e.g. linear model for nonlinear curve); fails to capture true trends. Train error is high, validation error is high.
- **High Variance (Overfitting)**: Model is overly complex (e.g. unregularized deep neural network); fits random noise in the training set. Train error is low, validation error is high.

2.2 L_2 Regularization (Weight Decay / Ridge)

Objective Function

Adds a penalty on the squared Euclidean norm of the weight parameters: $$\mathcal{J}{|text{total}}(|mathbf{w}) = |mathcal{J}|}(\mathbf{w}) + \frac{\lambda}{2} |\mathbf{w}2^2 = |mathcal{J}} w_i^2$$ }(\mathbf{w}) + \frac{\lambda}{2} \sum_{i*(Note: Biases b are typically left unregularized because regularizing bias introduces unnecessary underfitting without controlling capacity).*

Gradient & Parameter Update

$$\nabla_{\mathbf{w}} \mathcal{J}{|text{total}} = |nabla}} \mathcal{J}{|text{data}} + |lambda |mathbf{w}

$$Performing a gradient descent update with step size \eta :$$

|mathbf{w}} = \mathbf{w}t - |eta |nabla}} \mathcal{J}{|text{total}} = |mathbf{w}t - |eta |nabla}} |mathcal{J}} - \eta \lambda \mathbf{w}t

$$Rearranging :$$

|mathbf{w}} = (1 - \eta \lambda) \mathbf{w}t - |eta |nabla$$}} \mathcal{J}_{\text{data}

IMPORTANT

> Because $0 < 1 - \eta\lambda < 1$, the parameter vector is multiplied by a shrinkage factor strictly less than 1 before taking the data gradient step. This is why L_2 regularization is called **Weight Decay**.

Effect on Hessian Eigenvectors

Using a quadratic approximation of $\mathcal{J}_{\text{data}}$ near optimal weights $\mathbf{w}^:$$w_i = \frac{\gamma_i}{\gamma_i + \lambda} w_i^$$ where γ_i are eigenvalues of the Hessian H: - If $\gamma_i \gg \lambda$ (direction of high curvature/steep descent): $\frac{\gamma_i}{\gamma_i+\lambda} \approx 1$, weights are preserved. - If $\gamma_i \ll \lambda$ (flat, uninformative direction): $\frac{\gamma_i}{\gamma_i+\lambda} \to 0$, weights are shunk toward zero!

2.3 L_1 Regularization (Lasso)

Objective Function

$$\mathcal{J}_{\text{total}}(\mathbf{w}) = \mathcal{J}(\mathbf{w}) + \alpha \|\mathbf{w}_1 = \mathcal{J} \|w_i|

$$Gradient (subgradient for $w_i \neq 0$) :$$

\nabla_{\mathbf{w}} \mathcal{J}}(\mathbf{w}) + \alpha \sum_{i{\text{total}} = \nabla} \mathcal{J{\text{data}} + \alpha \, \text{sign}(\mathbf{w})

$$Update rule :$$

w(w_t)$$ - While L_2 shrinks weights proportionally to their magnitude ($-\lambda w$), L_1 subtracts a constant force ($-\alpha \operatorname{sign}(w)$). - } = w_t - \eta \nabla \mathcal{J}_{\text{data}} - \eta \alpha \, \text{sign**Sparsity**: L **drives smal** unimportant weights all the way to **strictly zero**, acting as an embedded feature selector. Geometrically, the diamond-shaped L_1 ball has sharp corners on the coordinate axes where the elliptical loss contours intersect.

2.4 Early Stopping

Mechanism

1. Split available training data into a Training Set and a Validation Set.
2. At regular intervals (every epoch), evaluate the loss on the validation set.
3. Save the model checkpoint that achieves the minimal validation loss.
4. If validation loss fails to improve for a set number of epochs (the *patience* hyperparameter), terminate training early.

Theoretical Equivalence to L_2 Regularization

Starting from initial weights $\mathbf{w}_0 = \mathbf{0}$, limiting training to T gradient descent steps confines the weights to a ball of radius $\approx \eta T |\nabla \mathcal{L}|$. Mathematical analysis proves that for quadratic error surfaces, **Early Stopping after T iterations is mathematically equivalent to L_2 weight decay with parameter** $\lambda \approx \frac{1}{\eta T}$.

2.5 Dataset Augmentation & Invariance Injection

- Neural networks generalize best when trained on massive, varied data distributions.
- **Data Augmentation**: Generating synthetic examples by applying realistic, label-preserving transformations:
- Vision: Horizontal flips, random rotations ($\pm 15^\circ$), random crops, scaling, color jitter, cutout/mixup.
- Audio: Time shifting, pitch modification, background noise addition.
- NLP: Back-translation, synonym replacement, masked language token insertion.
- Effectively injects domain-specific prior knowledge into the model, enforcing invariance without needing architectural changes.

2.6 Ensemble Methods & Bagging

Let M separate, diverse models produce predictions $\hat{y}1, \dots, \hat{y}M. The ensemble prediction is :$\hat{y} \sum} = \frac{1}{M}$$ Assume each model's prediction error $\epsilon_m = \hat{y}}^M \hat{y m - y has zero mean \mathbb{E}[\epsilon_m] = 0, individual variance \mathbb{E}[\epsilon_m^2] = v , and average pairwise covariance c = \mathbb{E}[\epsilon_i \epsilon_j] (i \neq j). The expected squared error of the ensemble is :$\mathbb{E}\left[\left(\frac{1}{M} \sum^M \epsilon \right)^2 \right] = \frac{1}{M} v + \frac{M - 1}{M} c$$ - If the models are **completely uncorrelated** ($c = 0$), the ensemble variance drops to $\frac{v}{M}$, which approaches 0 as $M \to \infty$! - In practice, even partially uncorrelated errors significantly reduce prediction variance.

2.7 Dropout (Srivastava, Hinton, et al., 2014)

Training Time

During each forward pass on a mini-batch, each hidden unit j in a dropout layer is randomly retained with keep probability $p \in (0, 1]$ (or dropped with probability $1 - p$):

$$r_j \sim \text{Bernoulli}(p)$$

$$\tilde{h}_j = r_j \cdot h_j$$

Test Time (Standard Dropout vs. Inverted Dropout)

- **Standard Dropout**: At test time, all neurons remain active, but the incoming weight matrix is scaled down by p: $$\mathbf{W}_{\text{test}} = p \, \mathbf{W}$$ This ensures that the expected input to each neuron at test time matches its expectation during training: $\mathbb{E}[\tilde{h}_j] = p h_j$.}

- **Inverted Dropout (Modern standard)**: Scale activations during training by $\frac{1}{p}$:

$$\tilde{h}_j = \frac{r_j}{p} h_j$$

This preserves the expected activation magnitude during training ($\mathbb{E}[\tilde{h}_j] = h_j$), meaning **no modifications or scaling are needed during inference/testing**!

Why Dropout Works

1. **Prevents Feature Co-adaptation**: Neurons cannot rely on specific other neurons being present. Every neuron must learn features that are independently robust and useful in random combinations.
2. **Implicit Ensemble**: A network with H dropout units can form 2^H possible thinned subnetworks. Dropout implicitly trains an ensemble of 2^H sub-networks that share parameters!

---

3. Core Formulas & Cheat Sheet

Regularizer	Mathematical Formula	Key Operational Property
Bias-Variance	$\mathbb{E}[(y - \hat{f})^2] = \text{Bias}^2 + \text{Var} + \sigma^2$	Fundamental trade-off governing model capacity
L_2 **Regularization**	$\mathcal{J} = \mathcal{J}_{\text{data}} + \frac{\lambda}{2}\lvert\mathbf{w}\rvert^2$	Gradient update: $(1 - \eta\lambda)\mathbf{w} - \eta\nabla\mathcal{J}_{\text{data}}$
L_1 **Regularization**	$\mathcal{J} = \mathcal{J}_{\text{data}} + \alpha\lvert\mathbf{w}\rvert_1$	Drives weights to exact zero; produces sparse models
Early Stopping	Halt when $\mathcal{L}_{\text{val}}$ reaches minimum	Equivalent to L_2 regularization with $\lambda \propto \frac{1}{\eta t}$
Augmentation Size	$N_{\text{new}} = N_{\text{orig}} \times (1 + k_{\text{aug}})$	Increases effective training size by exposing transformations
Dropout Keep Prob	$\mathbb{E}[\text{active units}] = p \times N_{\text{units}}$	$p = 0.8$ with 50 units $\implies$ 40 active units
Inverted Dropout	$\tilde{\mathbf{h}} = \frac{\mathbf{r} \odot \mathbf{h}}{p}$	Requires no test-time scaling
Ensemble Averaging	$\text{Var}_{\text{ens}} = \frac{v}{M} + \frac{M-1}{M}c$	Reduces variance toward $\frac{v}{M}$ as correlation $c \to 0$

---

4. Assignment Solutions with Detailed Step-by-Step Explanations

Case Study 1: Water Demand Forecasting for a Residential Zone (Questions 1 to 5)

Context & Data

A municipal analytics team predicts next-day water demand: - Simple linear model: Similar predictions across different training samples, but far from true demand. - Neural network: Fits training data closely, but predictions change substantially across different training subsets. - True demand: $y = 48$ thousand liters. - Predictions of the neural network from 4 training subsets:

$$41, 49, 45, 53$$

- Error decomposition:

$$\mathbb{E}[(y - \hat{f}(x))^2] = \text{Bias}^2 + \text{Variance} + \sigma^2$$

Question 1

Which description best matches the simple linear model? - A) High bias and low variance - B) Low bias and high variance - C) Low bias and low variance - D) High variance and zero bias

Correct Answer: A

- **Detailed Step-by-Step Explanation**:
- Stable predictions across different training subsets indicate low variability/sensitivity, meaning **low variance**.
- Being consistently far from the true target value indicates systematic under-fitting, meaning **high bias**.

Question 2

What is the average neural-network prediction for the given input? - A) 46 - B) 48 - C) 47 - D) 50

Correct Answer: C

- **Detailed Step-by-Step Explanation**:

$$\mathbb{E}[\hat{f}(x)] = \frac{41 + 49 + 45 + 53}{4} = \frac{188}{4} = 47$$

Question 3

What is the squared bias of the neural-network prediction? - A) 0 - B) 1 - C) 4 - D) 9

Correct Answer: B

- **Detailed Step-by-Step Explanation**:

$$\text{Bias} = \mathbb{E}[\hat{f}(x)] - f(x) = 47 - 48 = -1$$

$$\text{Squared Bias} = (\text{Bias})^2 = (-1)^2 = 1$$

Question 4

What does the large variation among the neural-network predictions indicate? - A) The model has no irreducible error. - B) The model has high variance under the case assumptions. - C) The output labels are unavailable for the described model. - D) The model is too simple for the data.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Large differences in predictions (ranging from 41 to 53) when trained on different random samples of training data is the hallmark definition of **high variance** (overfitting).

Question 5

Which action is most directly aimed at reducing the neural network's variance? - A) Add a suitable regularization method. - B) Remove the validation set entirely. - C) Increase model depth without constraints. - D) Train until training loss becomes zero.

Correct Answer: A

- **Detailed Step-by-Step Explanation**: Regularization techniques (such as L_2 weight decay, dropout, data augmentation, and early stopping) constrain model complexity, directly curbing overfitting and reducing model variance.

Case Study 2: Small-Business Loan Risk Scoring (Questions 6 to 10)

Context & Data

Logistic regression with L_2 regularization: $$\mathcal{J}*{\text{total}}(\mathbf{w}) = \mathcal{J} w_i^2$$ Given: - Current parameter: $w = 3$ - Data-loss gradient: $\frac{\partial \mathcal{J}}}(\mathbf{w}) + \alpha \sum_{i*{\text{data}}}}{\partial w} = 4 - Regularization strength : \alpha = 2 - Learning rate : \eta = 0.1 - Regularized gradient formula : $\frac{\partial \mathcal{J}} = 4 + 2\alpha w$$}}}{\partial w

---

Question 6

What is the main purpose of adding L_2 regularization here? - A) To remove the need for training data for the described model - B) To discourage large weights for the described model - C) To guarantee perfect validation accuracy - D) To convert logistic regression into clustering

Correct Answer: B

- **Detailed Step-by-Step Explanation**: The quadratic penalty $\alpha \sum w_i^2$ penalizes large weight magnitudes, keeping the decision boundary smooth and preventing individual features from exerting disproportionate leverage.

Question 7

What is the regularization contribution $2\alpha w$ to the gradient? - A) 10 - B) 6 - C) 8 - D) 12

Correct Answer: D

- **Detailed Step-by-Step Explanation**:

$$\text{Regularization gradient} = 2\alpha w = 2(2)(3) = 12$$

Question 8

What is the full regularized gradient for this parameter? - A) 16 - B) 12 - C) 10 - D) 14

Correct Answer: A

- **Detailed Step-by-Step Explanation**: $$\frac{\partial \mathcal{J}}}{\partial w} = \frac{\partial \mathcal{J}} + 2\alpha w = 4 + 12 = 16$$}}}{\partial w

---

Question 9

What is the updated value of w after one gradient-descent step? - A) 2.0 - B) 2.6 - C) 1.4 - D) 3.6

Correct Answer: C

- **Detailed Step-by-Step Explanation**:

$$w_{\text{new}} = w - \eta \frac{\partial \mathcal{J}_{\text{total}}}{\partial w} = 3 - 0.1(16) = 3 - 1.6 = 1.4$$

Question 10

Which outcome is expected after applying an appropriate L_2 penalty? - A) Training accuracy must become zero for the data shown here. - B) Validation performance may improve due to reduced overfitting. - C) All feature weights must become exactly zero. - D) The model stops using gradient descent under the given conditions.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: By restricting over-parameterization, L_2 regularization reduces variance on unseen validation samples, often improving test generalization even if training accuracy drops slightly.

Case Study 3: Energy Consumption Prediction for an Office Campus (Questions 11 to 15)

Context & Data

Training and validation loss progression: | Epoch | Training Loss | Validation Loss | |:---:|:---:|:---:| | 5 | 0.62 | 0.70 | | 10 | 0.44 | 0.52 | | 15 | 0.31 | 0.43 | | 20 | 0.22 | 0.46 | | 25 | 0.16 | 0.51 |

Question 11

At which epoch is the validation loss minimum? - A) Epoch 10 - B) Epoch 20 - C) Epoch 15 - D) Epoch 25

Correct Answer: C

- **Detailed Step-by-Step Explanation**: Inspecting the validation loss column: $0.70, 0.52, 0.43, 0.46, 0.51$. The lowest value is 0.43, which occurs at **Epoch 15**.

Question 12

What pattern after epoch 15 suggests overfitting? - A) Both losses increase together. - B) Training loss increases while validation loss decreases. - C) Training loss becomes exactly zero. - D) Training loss decreases while validation loss increases.

Correct Answer: D

- **Detailed Step-by-Step Explanation**: From Epoch 15 to 25, the training loss steadily declines ($0.31 \to 0.22 \to 0.16$) while the validation loss rises ($0.43 \to 0.46 \to 0.51$). Divergence where training error drops while validation error deteriorates is the classic indicator of **overfitting**.

Question 13

Why should early stopping monitor validation loss rather than only training loss? - A) Validation loss reflects performance on unseen data. - B) Training loss is not computed during neural-network training. - C) Validation loss is always exactly equal to test loss. - D) Training loss cannot decrease across epochs.

Correct Answer: A

- **Detailed Step-by-Step Explanation**: Training loss solely reflects memorization of seen samples. Validation loss serves as an unbiased proxy for out-of-sample generalization.

Question 14

Which statement about early stopping is correct? - A) It guarantees the globally optimal weights. - B) It acts as a regularization method by limiting over-training. - C) It prevents all forms of noise in input data. - D) It removes the need for a validation set under the case assumptions.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Early stopping halts gradient descent before weights grow excessively large to fit idiosyncrasies of the training set, functioning as an effective regularizer.

Question 15

If the team keeps the model from epoch 25 instead of epoch 15, what is the main risk? - A) The training set becomes larger automatically. - B) The model cannot produce numerical predictions. - C) The validation loss is ignored by definition under the case assumptions. - D) The model may generalize worse despite lower training loss.

Correct Answer: D

- **Detailed Step-by-Step Explanation**: At epoch 25, the model has overfit the training data (validation loss is 0.51 vs 0.43 at epoch 15), meaning it will generalize worse on future unseen days.

Case Study 4: Product-Surface Image Quality Inspection (Questions 16 to 20)

Context & Data

A dataset contains $N = 900$ labeled images. - Train split: 70%, Validation split: 30%. - Data augmentation: Each original training image generates **two additional** augmented images. - Dropout: Hidden layer keep probability $p = 0.8$. - Ensemble: Averaging 3 independently trained CNNs.

Question 16

How many original images are in the training split before augmentation? - A) 270 - B) 900 - C) 630 - D) 450

Correct Answer: C

- **Detailed Step-by-Step Explanation**:

$$\text{Training images} = 0.70 \times 900 = 630$$

Question 17

What is the effective training-set size after adding two augmented images per original training image? - A) 1890 - B) 1260 - C) 630 - D) 1530

Correct Answer: A

- **Detailed Step-by-Step Explanation**: Each original image now yields $1\ (\text{original}) + 2\ (\text{augmented}) = 3$ training instances.

$$\text{Effective size} = 630 \times 3 = 1890$$

Question 18

What is the main reason augmentation can improve validation accuracy? - A) It changes all class labels randomly in the stated case setting. - B) It removes the need to learn features. - C) It guarantees that training loss is zero. - D) It exposes the model to realistic input variations.

Correct Answer: D

- **Detailed Step-by-Step Explanation**: Augmentation simulates realistic variations (rotations, illumination shifts, noise) that occur in real-world deployment, preventing the CNN from latching onto spurious pixel-level artifacts.

Question 19

During dropout training with keep probability $p = 0.8$, approximately how many units remain active in a layer of 50 hidden units? - A) 25 - B) 10 - C) 40 - D) 50

Correct Answer: C

- **Detailed Step-by-Step Explanation**: The number of active units follows a Binomial distribution $B(n, p)$ with expected value:

$$\mathbb{E}[\text{active units}] = n \times p = 50 \times 0.8 = 40$$

Question 20

Why can averaging three independently trained CNNs improve reliability? - A) It increases the learning rate of each CNN. - B) It can reduce variance by combining diverse predictions. - C) It removes all bias from every CNN for the data shown here. - D) It makes the dataset labels unnecessary.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Independent models make errors on different subsets of the data space. Averaging their posterior probabilities cancels out idiosyncratic mistakes, significantly reducing prediction variance.

---

5. NPTEL Exam Traps & High-Yield Pitfalls

1. **Augmentation Multiplier**:
2. Pay close attention to wording: "adding two augmented images *per* original image" means the total is $1 + 2 = 3\times$ the original size, **not** $2\times$!
3. **Dropout Probability (p vs $1-p$)**:
4. Check whether the problem defines p as the **retention/keep probability** or the **drop probability**.
5. If keep probability is 0.8, then 80% of units are active.
6. If drop probability is 0.2, then $1 - 0.2 = 0.8$ are active.
7. **Weight Decay Factor**:
8. In L_2 regularization, the shrinkage factor is $(1 - \eta\lambda)$. If $\eta = 0.01$ and $\lambda = 0.1$, weights are scaled by $1 - 0.001 = 0.999$ each step.

MODULE 8

Week 8: Training Deep Networks — Greedy Pre-training, Activations, Initialization, and BN

Week 8: Training Deep Networks — Greedy Pre-training, Modern Activations, Initialization, and Batch Normalization

1. Executive Summary & Exam Roadmap

Week 8 explores the breakthroughs that enabled the training of deep architectures without falling victim to vanishing or exploding gradients: 1. **Vanishing and Exploding Gradients Pathology**: The mathematical cause of gradient decay through deep chains of saturated activations ($\sigma'(z) \leq 0.25$). 2. **Greedy Layer-wise Unsupervised Pre-training (Hinton, Bengio 2006)**: Layer-by-layer unsupervised feature discovery, freezing weights, and end-to-end supervised fine-tuning. 3. **Modern Activation Functions**: - **ReLU**: Non-saturating positive regime ($f'(x) = 1$), computational speed, and the "Dying ReLU" pathology. - **Leaky ReLU, PReLU, ELU, and tanh**: Preserving negative gradients and zero-centered dynamics. 4. **Principled Weight Initialization**: - Preserving variance across layers ($\text{Var}(y) \approx \text{Var}(x)$). - **Xavier (Glorot) Initialization**: $\text{Var}(W) = \frac{1}{n_{\text{in}}}$ or $\frac{2}{n_{\text{in}}+n_{\text{out}}}$ (optimal for tanh/sigmoid). - **He (Kaiming) Initialization**: $\text{Var}(W) = \frac{2}{n_{\text{in}}}$ (compensating for ReLU's half-space clipping). 5. **Batch Normalization (Ioffe & Szegedy, 2015)**: Reducing internal covariate shift, batch normalization transform equations, learnable scale (γ) and shift (β), and test-time running statistics.

2. Comprehensive Lecture Notes & Mathematical Foundations

2.1 The Vanishing and Exploding Gradient Problem

Consider an L-layer feedforward network with scalar activations:

$$\frac{\partial \mathcal{L}}{\partial a_1} = \frac{\partial \mathcal{L}}{\partial a_L} \cdot \prod_{l=1}^{L-1} \frac{\partial a_{l+1}}{\partial a_l} = \frac{\partial \mathcal{L}}{\partial a_L} \cdot \prod_{l=1}^{L-1} \left(W_{l+1} \cdot g'(a_l) \right)$$

In vectorized form: $$\nabla_{\mathbf{a}1} \mathcal{L} = \left[\prod^L \mathbf{W}_l^\top \text{diag}(g'(\mathbf{a}l)) \right] \nabla$$_L} \mathcal{L

The Two Failure Modes

1. **Vanishing Gradients**:
2. If the singular values or eigenvalues of $\mathbf{W}_l^\top \text{diag}(g'(\mathbf{a}l))are strictly less than 1, multiplying L$ such matrices causes the gradient to decay **exponentially with depth**: $$|\nabla \lambda < 1)$$_1} \mathcal{L}| \propto \lambda^L \to 0 \quad (\text{for
3. For a sigmoid neuron, $\max g'(z) = 0.25$. Even if weights are of order 1, the product includes $(0.25)^L$. For $L = 10$, $(0.25)^{10} \approx 9.5 \times 10^{-7}$, rendering the earliest layers completely frozen and incapable of learning.
4. **Exploding Gradients**:
5. If the eigenvalues are significantly greater than 1, the gradient norm grows exponentially ($\lambda^L \to \infty$), causing numerical overflow (`NaN`), wild parameter oscillations, and optimization divergence.

2.2 Greedy Layer-wise Pre-training (2006 Breakthrough)

Before 2006, training networks deeper than 2–3 layers from random initializations consistently failed. Geoff Hinton and Yoshua Bengio solved this using **Greedy Layer-wise Pre-training**:

```
Step 1: Train Layer 1 Autoencoder (Input x ---> Latent h_1 ---> Output x_hat)
Step 2: Freeze Layer 1 weights W_1. Compute h_1 = f(W_1 * x).
Step 3: Train Layer 2 Autoencoder (Input h_1 ---> Latent h_2 ---> Output h_1_hat)
Step 4: Repeat greedily up to Layer L.
Step 5: Attach classification/regression head. Fine-tune ALL layers via Backprop.
```

Why It Works: Regularization & Optimization Perspective

1. **Unsupervised Discovery**: Uses vast unlabeled data to discover the underlying geometric manifold of the data distribution.
2. **Better Optimization Basin**: The error surface of a deep network is non-convex with numerous poor local plateaus. Pre-training initializes the weights in an attractor basin close to a good local optimum, avoiding bad saddle regions.

2.3 Modern Activation Functions



1. Rectified Linear Unit (ReLU) (Nair & Hinton, 2010)

$$f(x) = \max(0, x)$$

$$f'(x) = \{1 \quad \text{if } x > 0\, 0 \quad \text{if } x < 0$$

- **Advantages**: - **No saturation in the positive domain**: For any $x > 0$, the derivative is strictly 1. This enables gradients to flow through dozens of layers without decay! - **Extreme computational efficiency**: Computed via simple hardware thresholding (no costly transcendentals like $\exp$). - **Sparse representations**: Approximately 50% of neurons are zero for random inputs. - **The "Dying ReLU" Problem**: - If a large gradient updates a neuron such that its pre-activation is negative for *all* samples in the dataset, its activation is always 0 and its gradient is always 0. The neuron becomes permanently dead and never recovers.

2. Leaky ReLU & PReLU

$$f(x) = \max(\alpha x, x) = \{x \quad \text{if } x > 0\, \alpha x \quad \text{if } x \leq 0$$

- In Leaky ReLU, α is a small fixed constant (e.g., $\alpha = 0.01$). - In Parametric ReLU (**PReLU**), α is treated as a learnable parameter updated via backprop. - Ensures a small, non-zero gradient (α) flows even when $x < 0$, preventing dead neurons.

3. Exponential Linear Unit (ELU)

$$f(x) = \{x \quad \text{if } x > 0\, \alpha(e^x - 1) \quad \text{if } x \leq 0$$

- Smooth everywhere; brings mean activations closer to zero, accelerating learning.

2.4 Principled Weight Initialization

To prevent signals from either vanishing to zero or exploding to infinity as they traverse forward and backward through L layers, we require the **variance of activations to remain constant across layers**:

$$\text{Var}(h_l) = \text{Var}(h_{l-1})$$

Let $y = \sum_{i=1}^{n_{\text{in}}} w_i x_i$, where w_i and x_i are independent, zero-mean random variables:

$$\text{Var}(y) = \sum_{i=1}^{n_{\text{in}}} \text{Var}(w_i x_i) = \sum_{i=1}^{n_{\text{in}}} \text{Var}(w_i)\text{Var}(x_i) = n_{\text{in}} \,\text{Var}(w)\,\text{Var}(x)$$

To ensure $\text{Var}(y) = \text{Var}(x)$, we must set:

$$n_{\text{in}} \,\text{Var}(w) = 1 \implies \mathbf{Var(w) = \frac{1}{n_{in}}}$$

1. Xavier (Glorot) Initialization (for Symmetric Activations: tanh, Sigmoid)

Taking both forward flow (n_{in}) and backward gradient flow (n_{out}) into account:

$$\text{Var}(w) = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

- **Gaussian**: $W \sim \mathcal{N}\left(0,\, \frac{2}{n_{\text{in}}+n_{\text{out}}}\right)$ - **Uniform**: $W \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}},\, +\sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}}\right)$

2. He (Kaiming) Initialization (for ReLU Activations)

Because ReLU sets all negative inputs to zero, it discards half of the variance:

$$\text{Var}(f(x)) = \frac{1}{2}\text{Var}(x)$$

To counteract this 50% energy loss per layer, the weight variance must be **doubled**:

$$\mathbf{Var(w) = \frac{2}{n_{in}}}$$

- **Gaussian**: $W \sim \mathcal{N}\left(0,\, \frac{2}{n_{\text{in}}}\right)$ - **Uniform**: $W \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}}}},\, +\sqrt{\frac{6}{n_{\text{in}}}}\right)$

2.5 Batch Normalization (Ioffe & Szegedy, 2015)

Internal Covariate Shift

During training, as parameters in earlier layers update, the distribution of inputs to later layers changes continuously. Later layers must constantly adapt to a non-stationary input distribution, forcing the use of very small learning rates.

The Batch Normalizing Transform

For a mini-batch $\mathcal{B} = x_1, \ldots, x_m$ of a specific neuron's pre-activation: 1. **Mini-batch Mean**:

$$\mu_{\mathcal{B}} = \frac{1}{m}\sum_{i=1}^{m} x_i$$

2. **Mini-batch Variance**:

$$\sigma_{\mathcal{B}}^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2$$

3. **Normalize to Zero Mean, Unit Variance**: $$\hat{x}*i = |frac{x_i - |mu$$ }}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}*(where $\epsilon > 0$ is a stability constant, typically 10^{-5}).* 4. **Scale and Shift (Learnable Affine Transformation)**: $$y_i = \text{BN}{\gamma, \beta}(x_i) = \gamma \hat{x}i + \beta$$ *where γ (scale) and β (shift) are learnable parameters optimized via backpropagation! - Why include γ and β? If normalizing restricts the representation capacity (e.g. forcing a sigmoid into its linear regime), the network can adaptively learn $\gamma = \sqrt{\sigma^2_{and}\beta = \mu}^2 + \epsilon$* to restore the original identity mapping.}

Placement in Architecture

Batch Normalization is standardly placed **immediately after the linear transformation and right before the nonlinear activation function**:

$$\mathbf{x} \longrightarrow [\mathbf{W}\mathbf{x}] \longrightarrow [\text{Batch Norm}] \longrightarrow [\text{Activation } g(\cdot)] \longrightarrow \mathbf{h}$$

Test Time / Inference

During inference, mini-batches may not exist (e.g. evaluating 1 sample). The network uses fixed running moving averages computed during training:

$$\mu_{\text{test}} = \mathbb{E}[\mu_{\mathcal{B}}], \quad \sigma^2_{\text{test}} = \frac{m}{m-1}\mathbb{E}[\sigma^2_{\mathcal{B}}]$$

$$\hat{x} = \frac{x - \mu_{\text{test}}}{\sqrt{\sigma^2_{\text{test}} + \epsilon}}, \quad y = \gamma\hat{x} + \beta$$

---

3. Core Formulas & Cheat Sheet

Technique	Mathematical Equation	Key Purpose / Rule of Thumb
ReLU	$f(x) = \max(0, x)$	Gradient is 1 for $x > 0$; solves vanishing gradients
Leaky ReLU	$f(x) = \max(\alpha x, x)$	Prevents dead neurons; $\alpha \approx 0.01$
Xavier Variance	$\text{Var}(W) = \frac{1}{n_{\text{in}}}$ or $\frac{2}{n_{\text{in}}+n_{\text{out}}}$	Used with tanh / Sigmoid activations
He Variance	$\text{Var}(W) = \frac{2}{n_{\text{in}}}$	Used with ReLU / Leaky ReLU activations
BN Mean	$\mu_{\mathcal{B}} = \frac{1}{m}\sum_{i=1}^{m} x_i$	Centering mini-batch
BN Variance	$\sigma^2_{\mathcal{B}} = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2$	Normalizing mini-batch dispersion
BN Normalization	$\hat{x}*i = \|frac{x_i - \|mu$}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}$	Produces zero mean and unit variance
BN Scale & Shift	$y_i = \gamma\hat{x}_i + \beta$	Restores network representation power

---

4. Assignment Solutions with Detailed Step-by-Step Explanations

Case Study 1: Insurance Claim Approval Prediction (Questions 1 to 5)

Context & Data

A deep feedforward network with 8 hidden layers classifies insurance claims. Output layer: Sigmoid (binary probability). Hidden layers originally: Sigmoid. Hidden layers replaced with:

$$\text{ReLU}(x) = \max(0, x)$$

Given values: $\sigma(2) \approx 0.88$, $\sigma(-4) \approx 0.018$.

Question 1

Why can sigmoid hidden layers slow down training in a deep network? - A) They can saturate and produce very small gradients. - B) They always output negative values under the given conditions. - C) They remove all nonlinear behavior. - D) They force every weight to become zero.

Correct Answer: A

- **Detailed Step-by-Step Explanation**: For inputs with large positive or negative magnitudes, the sigmoid function saturates near 1 or 0, where its derivative $\sigma'(z) = \sigma(z)(1 - \sigma(z)) \to 0$. Multiplying these tiny fractions through 8 hidden layers drives the gradients to near zero (vanishing gradients), halting learning.

Question 2

What is $\text{ReLU}(-4)$? - A) -4 - B) -1 - C) 0 - D) 4

Correct Answer: C

- **Detailed Step-by-Step Explanation**:

$$\mathrm{ReLU}(-4) = \max(0, -4) = 0$$

Question 3

What is $\mathrm{ReLU}(2)$? - A) 0 - B) -2 - C) 1 - D) 2

Correct Answer: D

- **Detailed Step-by-Step Explanation**:

$$\mathrm{ReLU}(2) = \max(0, 2) = 2$$

Question 4

Which property makes ReLU helpful for positive pre-activation values? - A) It bounds hidden outputs between 0 and 1, as sigmoid would. - B) It avoids saturation and keeps a useful gradient. - C) It converts classification into clustering. - D) It makes the loss function unnecessary.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: For all $x > 0$, the derivative of ReLU is constantly 1. Gradients flow backward without attenuation, effectively eliminating saturation and the vanishing gradient problem.

Question 5

Why is the output layer still kept as sigmoid in this binary classifier? - A) It maps the final score to a probability-like value between 0 and 1. - B) It is required for every hidden layer for the described model. - C) It guarantees no validation error after training on every future test case. - D) It prevents all negative input features for the data shown here.

Correct Answer: A

- **Detailed Step-by-Step Explanation**: The task is binary classification. A sigmoid output squashes the real-valued logit into $(0, 1)$, allowing it to be directly interpreted as the posterior probability $P(y = 1|\mathbf{x})$.

Case Study 2: Solar Power Forecasting for a Rooftop Plant (Questions 6 to 10)

Context & Data

A 7-hidden-layer network with ReLU activations predicts solar generation. Formulas: - Xavier Initialization: $\mathrm{Var}(w) = \frac{1}{n}$ - He Initialization: $\mathrm{Var}(w) = \frac{2}{n}$ Hidden layer with $n = 200$ inputs.

Question 6

What problem can poor initialization create in a deep network? - A) It automatically removes all noise from data. - B) It can cause activations or gradients to explode or vanish. - C) It guarantees underfitting only under the given conditions. - D) It changes regression targets into class labels.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: If initial weights are too small, forward activations and backward gradients shrink exponentially with layer depth (vanishing). If initial weights are too large, activations and gradients explode, causing unstable training.

Question 7

What is the Xavier variance for $n = 200$? - A) 0.020 - B) 0.200 - C) 0.100 - D) 0.005

Correct Answer: D

- **Detailed Step-by-Step Explanation**:

$$\mathrm{Var}(w) = \frac{1}{n} = \frac{1}{200} = 0.005$$

Question 8

What is the He variance for $n = 200$? - A) 0.010 - B) 0.005 - C) 0.020 - D) 0.100

Correct Answer: A

- **Detailed Step-by-Step Explanation**:

$$\mathrm{Var}(w) = \frac{2}{n} = \frac{2}{200} = 0.010$$

Question 9

Why is He initialization more suitable for these hidden layers? - A) The output layer uses no activation. - B) The network has only one layer. - C) The hidden layers use ReLU activations. - D) The target variable is binary in the stated case setting.

Correct Answer: C

- **Detailed Step-by-Step Explanation**: He initialization is specifically derived for ReLU activations, where the extra factor of 2 compensates for the fact that half of the inputs are clipped to zero.

Question 10

Which observation suggests the new initialization improved training stability? - A) Hidden activations are all forced to zero. - B) Loss decreases more smoothly across epochs. - C) No validation set is needed in the stated case setting. - D) All weights become identical.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Stable gradient and activation variances prevent numerical oscillations, resulting in steady, monotonic decreases in training loss across epochs.

Case Study 3: Warehouse Sensor Fault Detection (Questions 11 to 15)

Context & Data

A deep neural network with 10 hidden layers detects sensor faults. Direct end-to-end supervised training is slow due to vanishing gradients in early layers. The team uses **greedy layer-wise pre-training**.

Question 11

What is the main goal of greedy layer-wise pretraining here? - A) To delete the hidden layers before training. - B) To avoid using input features during both unsupervised pretraining and fine-tuning. - C) To give each layer a useful initialization before supervised fine-tuning. - D) To guarantee zero test error in this specific scenario.

Correct Answer: C

- **Detailed Step-by-Step Explanation**: Greedy layer-wise pre-training provides a structured initialization for each layer's weights based on input data geometry, placing the network in a favorable region of parameter space before full backpropagation.

Question 12

During pretraining of the second hidden layer, what happens to the already trained first layer? - A) It is randomly reinitialized in the stated case setting. - B) It is removed from the model. - C) It is trained with the output labels only. - D) It is kept fixed while the next layer is trained.

Correct Answer: D

- **Detailed Step-by-Step Explanation**: In greedy pre-training, previously trained layers are frozen, and their output representations serve as fixed inputs for training the subsequent layer.

Question 13

Why is the first pretraining objective described as unsupervised? - A) It uses the input data without using the abnormal-condition labels. - B) It uses no numerical features under the given conditions. - C) It uses only the final classifier output for the data shown here. - D) It requires the validation labels to be hidden in the stated case setting.

Correct Answer: A

- **Detailed Step-by-Step Explanation**: The pre-training phase relies solely on input reconstruction (e.g. autoencoders) without utilizing target class labels.

Question 14

What happens after all hidden layers have been pretrained? - A) The network is discarded under the given conditions. - B) The full model is fine-tuned with the supervised task objective. - C) The input features are replaced by random values. - D) The loss function is no longer required in the stated case setting.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Once all layers are initialized, the complete network is un-frozen and fine-tuned end-to-end using standard supervised backpropagation on the labeled task.

Question 15

Which issue is layer-wise pretraining intended to reduce? - A) The need to store data files in this specific scenario. - B) The number of output classes under the case assumptions. - C) The difficulty of training early layers in a very deep network. - D) The existence of nonlinear activations under the given conditions.

Correct Answer: C

- **Detailed Step-by-Step Explanation**: Because supervised gradients vanish before reaching early layers in very deep networks, unsupervised pre-training ensures that early layers already compute meaningful hierarchical features.

Case Study 4: Laboratory Test Result Prediction (Questions 16 to 20)

Context & Data

A 6-hidden-layer network predicts test repeats. Batch Normalization is inserted between linear transformations and ReLU. For one mini-batch, a neuron's pre-activations are:

$$1, 3, 5, 7$$

Question 16

What problem does batch normalization primarily address? - A) It changes labels from binary to continuous. - B) It removes the need for backpropagation. - C) It guarantees a perfect classifier in the stated case setting. - D) It reduces shifting layer-input distributions during training.

Correct Answer: D

- **Detailed Step-by-Step Explanation**: Batch Normalization reduces internal covariate shift by stabilizing the mean and variance of layer inputs across mini-batches during training.

Question 17

What is the mean of the mini-batch values $1, 3, 5, 7$? - A) 3 - B) 4 - C) 5 - D) 16

Correct Answer: B

- **Detailed Step-by-Step Explanation**:

$$\mu_B = \frac{1+3+5+7}{4} = \frac{16}{4} = 4$$

Question 18

What is the batch variance using the average squared deviation? - A) 4 - B) 6 - C) 3 - D) 5

Correct Answer: D

- **Detailed Step-by-Step Explanation**: Compute squared deviations from mean $\mu_B = 4$:
- $(1-4)^2 = (-3)^2 = 9$
- $(3-4)^2 = (-1)^2 = 1$
- $(5-4)^2 = 1^2 = 1$
- $(7-4)^2 = 3^2 = 9$

$$\sigma_B^2 = \frac{9+1+1+9}{4} = \frac{20}{4} = 5$$

Question 19

Where is batch normalization placed in this case? - A) Only after the output probability for the described model. - B) Before every input feature is collected. - C) Between the linear transformation and ReLU activation. - D) After the final prediction is thresholded.

Correct Answer: C

- **Detailed Step-by-Step Explanation**: The standard placement introduced by Ioffe & Szegedy is directly after the linear transformation $\mathbf{Wx} + \mathbf{b}$ and before the nonlinear activation function (ReLU).

Question 20

Why are learnable scale and shift parameters useful in batch normalization? - A) They allow the network to choose an appropriate normalized representation. - B) They delete the mini-batch statistics in this specific scenario. - C) They prevent gradients from being computed under the case assumptions. - D) They force every activation to remain exactly zero for the described model.

Correct Answer: A

- **Detailed Step-by-Step Explanation**: The parameters γ and β allow the network to learn any scale and shift it requires, ensuring that normalization does not restrict the model's expressive capacity.

---

5. NPTEL Exam Traps & High-Yield Pitfalls

1. **He vs Xavier Factor**:
2. For **tanh / Sigmoid**: Use Xavier: $\mathrm{Var}(W) = \frac{1}{n_{\text{in}}}$.
3. For **ReLU**: Use He: $\mathrm{Var}(W) = \frac{2}{n_{\text{in}}}$.
4. Forgetting the factor of 2 for ReLU is one of the most frequent mistakes in numerical MCQs!
5. **ReLU Derivative at Zero**:
6. Technically $f'(0)$ is undefined, but by convention in deep learning frameworks, it is set to either 0 or 1 (a subgradient).
7. **Batch Norm Parameters Count**:
8. For a layer with D channels/neurons, Batch Normalization introduces $2D$ **learnable parameters**: D scale parameters (γ) and D shift parameters (β).

MODULE 9

Week 9: Learning Vectorial Representations of Words — Word Embeddings and Word2Vec

Week 9: Learning Vectorial Representations of Words — Word Embeddings, Word2Vec, and SVD

1. Executive Summary & Exam Roadmap

Week 9 explores how discrete symbolic text is converted into dense, continuous mathematical vectors for deep neural networks: 1. **One-Hot Representation & Bag-of-Words (BoW)**: High dimensionality (V), severe sparsity, orthogonality, and the inability to capture semantic relatedness. 2. **The Distributional Hypothesis (Firth, 1957)**: "A word is characterized by the company it keeps." 3. **Count-based Approaches (Co-occurrence Matrix + Truncated SVD)**: - Constructing word-context matrices $C \in \mathbb{R}^{V \times V}$. - Truncated SVD dimensionality reduction: $C \approx U_k \Sigma_k V_k^\top$. - Strengths and computational limitations ($O(V^3)$). 4. **Prediction-based Approaches (Word2Vec - Mikolov et al., 2013)**: - **Continuous Bag of Words (CBOW)**: Predicting center target word from context words. - **Skip-Gram**: Predicting context words from the center target word. 5. **Computational Acceleration**: - The softmax denominator bottleneck over vocabulary V. - **Negative Sampling (SGNS)**: Converting multi-class classification into k binary classification tasks with $P_n(w) \propto U(w)^{3/4}$. - **Hierarchical Softmax**: Huffman tree routing ($O(\log V)$ complexity). 6. **Vector Arithmetic & Evaluation**: Cosine similarity, word analogies ($\mathbf{v}_b - \mathbf{v}_a + \mathbf{v}_c$), and correlation with human judgments.

2. Comprehensive Lecture Notes & Mathematical Foundations

2.1 One-Hot Word Representations and Their Fatal Flaws

Let $\mathcal{V} = w_1, w_2, \dots, w_V$ be a fixed vocabulary of V unique words. In a one-hot representation, each word w_i is mapped to a standard basis vector $\mathbf{e}_i \in \{0, 1\}^V: $\mathbf{e}_i$ = [0, 1, 0, \dots, 0]^\top$$} = [1, 0, 0, \dots, 0]^\top, \quad \mathbf{e}_{\text{dog}}

Severe Limitations:

1. **Orthogonality (Zero Semantic Similarity)**: For any two distinct words $i \neq j$:

$$\mathbf{e}_i^\top \mathbf{e}_j = 0, \quad \text{cosine_sim}(\mathbf{e}_i, \mathbf{e}_j) = \frac{\mathbf{e}_i^\top \mathbf{e}_j}{|\mathbf{e}_i||\mathbf{e}_j|} = 0$$

   The distance between "cat" and "kitten" is identically equal to the distance between "cat" and "airplane".
2. **Sparsity & Memory Explosion**: For modern corpora, $V \approx 10^5$ to 10^6. Every word requires a megabyte-scale sparse vector containing exactly one 1 and $999,999$ zeros.

2.2 The Distributional Hypothesis & Count-Based SVD

The Distributional Hypothesis

Proposed by linguist J.R. Firth (1957): Words that appear in similar linguistic contexts tend to share similar semantic and syntactic meanings.

Co-occurrence Matrix Construction

Define a symmetric context window of size c (e.g. $c = 1$ considers 1 word to the left and 1 word to the right). A square matrix $C \in \mathbb{R}^{V \times V}$ is constructed where entry C_{ij} records the number of times word w_j appears within a window of c tokens around word w_i.

```
Corpus: "deep learning is fun" (window c = 1)
Target "learning" has context words: "deep" (left) and "is" (right).
```

Dimensionality Reduction via Truncated SVD

The raw matrix $C \in \mathbb{R}^{V \times V}$ is sparse and noisy. Applying SVD:

$$C = U \Sigma V^\top$$

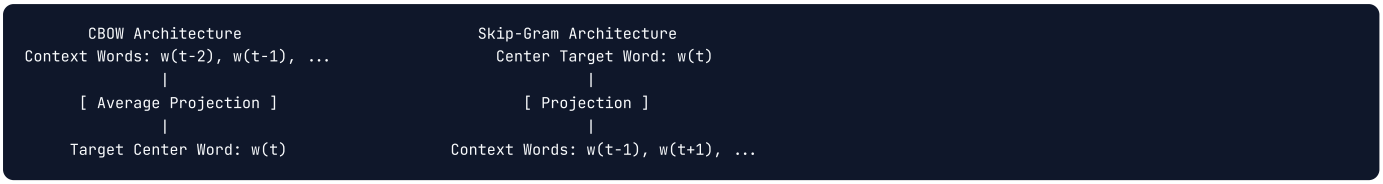
Retaining only the top k singular values ($k \ll V$, typically $k \in [100, 300]$):

$$C \approx U_k \Sigma_k V_k^\top$$

- $U_k \in \mathbb{R}^{V \times k}$: Each row i of U_k (or $U_k \Sigma_k$) provides a compact, dense, k-dimensional continuous vector embedding for word w_i. - **Major Drawbacks**: - Computational complexity is $O(V^3)$, making it intractable for $V > 50,000$. - High-frequency functional stopwords ("the", "is", "a") dominate co-occurrence counts without carrying semantic content.

2.3 Prediction-Based Models: Word2Vec (Mikolov et al., 2013)

Instead of counting global frequencies and factoring a huge matrix, Word2Vec learns word vectors iteratively using small sliding windows and backpropagation.



1. Continuous Bag of Words (CBOW)

- **Objective**: Given a sequence of $2c$ context words $w_{t-c}, \dots, w_{t+c} \setminus w_t$, predict the center target word w_t.
- **Hidden Layer Representation**: Let $\mathbf{v}_w \in \mathbb{R}^d$ be the input embedding of word w. The hidden layer computes the average of the context embeddings : $$\mathbf{h} = \frac{1}{2c} \sum \mathbf{v}_w$$
- **Output Probability (Softmax)**: Let $\mathbf{u}_w \in \mathbb{R}^d$ denote the output representation of word w: $$P(w_t \mid \text{context}) = \frac{\exp(\mathbf{u}_w^\top \mathbf{h})}{\sum_{w \in \mathcal{V}} \exp(\mathbf{u}_w^\top \mathbf{h})}$$
- **Loss**: Categorical Cross-Entropy: $\mathcal{L} = -\log P(w_t|\text{context})$.

2. Skip-Gram Model

- **Objective**: Given the center target word w_t, predict each of the surrounding context words w_{t+j} for $-c \le j \le c, j \neq 0$.
- **Probability of context word w_c given center word w_t**: $$P(w_c \mid w_t) = \frac{\exp(\mathbf{u}_{w_c}^\top \mathbf{v}_{w_t})}{\sum_{w \in \mathcal{V}} \exp(\mathbf{u}_w^\top \mathbf{v}_{w_t})}$$
- **Total Corpus Objective**: Maximize the average log probability:

$$\mathcal{L} = \sum_{t=1}^{T} \sum_{-c \le j \le c, j \neq 0} \log P(w_{t+j}|w_t)$$

2.4 Accelerating Training: Negative Sampling (SGNS)

The Computational Bottleneck

In the standard softmax: $$P(w_c \mid w_t) = \frac{\exp(\mathbf{u}_{w_c}^\top \mathbf{v}_{w_t})}{\sum_{w \in \mathcal{V}} \exp(\mathbf{u}_w^\top \mathbf{v}_{w_t})}$$ The denominator requires summing over all V words in the vocabulary for every training step! For $V = 10^6$, this is impossible to train.)

Negative Sampling Formulation

Mikolov et al. replaced the multinomial softmax with multiple independent binary logistic regressions: 1. Maximize the probability that the true target-context pair (w_t, w_c) came from the data ($\mathcal{D} = 1$). 2. Minimize the probability that K randomly sampled noise pairs (w_t, w_{neg}) came from the data ($\mathcal{D} = 0$).

Loss Function for a Target Word w_t and Context Word w_c: $$\mathcal{L}_{\text{SGNS}} = \log \sigma(\mathbf{u}_{w_c}^\top \mathbf{v}_{w_t}) + \sum_{k=1}^{K} \mathbb{E}_{w_k \sim P_n(w)} \left[\log \sigma(-\mathbf{u}_{w_k}^\top \mathbf{v}_{w_t}) \right]$$ where: - $\sigma(z) = \frac{1}{1+e^{-z}}$ is the logistic sigmoid. - K is the number of negative samples (typically $5 \le K \le 20$ for small datasets; $2 \le K \le 5$ for massive datasets). - ***The Noise Distribution*** $P_n(w)$: Sampled from the unigram frequency distribution raised to the $3/4$ power: $$P_n(w) = \frac{U(w)^{3/4}}{\sum$$ } U(w')^{3/4}Why $3/4$? Raising to 0.75 slightly suppresses the overwhelming frequency of common words ("the", "of") while boosting the sampling probability of rare words.

2.5 Vector Space Geometry & Word Analogies

Dense word embeddings capture linear relational composition in the semantic vector space: - **Cosine Similarity**:

$$\text{cosine_sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{|\mathbf{u}||\mathbf{v}|}$$

- **Linear Analogies ($a : b :: c : d$)**: "Man is to King as Woman is to ?" $$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} \approx \mathbf{v}_{\text{queen}} - \mathbf{v}_{\text{woman}}$$

$$To solve for the unknown word d, search for the vocabulary vector $\mathbf{v}_w$ ($w \neq a, b, c$) that maximizes cosine similarity to :$$

$$\mathbf{v}^* = \mathbf{v}_b - \mathbf{v}_a + \mathbf{v}_c$$

---

3. Core Formulas & Cheat Sheet

Concept	Mathematical Equation	Key Notes
One-Hot Vector	$\mathbf{e}_i = [0, \dots, 1, \dots, 0]^\top \in 0, 1^V$	Length V, exactly one 1; all pairs orthogonal
Co-occurrence SVD	$C_{V\times V} \approx U_{V\times k}\Sigma_{k\times k}V^\top_{k\times V}$	Embeddings are rows of U_k; cost $O(V^3)$
CBOW Target Prob	$P(w_i\|\mathbf{h}) = \frac{\exp(\mathbf{u}_{w_i}^\top \mathbf{h})}{\sum_w \exp(\mathbf{u}_w \mathbf{h})}$	$\mathbf{h} = \frac{1}{2c}\sum \mathbf{v}_c$ (context average)
Skip-Gram Prob	$P(w_c\|w_t) = \frac{\exp(\mathbf{u}_{w_c}^\top \mathbf{v}_{w_t})}{\sum_w \exp(\mathbf{u}_w^\top \mathbf{v}_{w_t})}$	Predicts context given center word
Negative Sampling	$\log \sigma(\mathbf{u}_{w_c}^\top \mathbf{v}_{w_t}) + \sum_{k=1}^{K} \log \sigma(-\mathbf{u}_{w_k}^\top \mathbf{v}_{w_t})$	Reduces $O(V)$ softmax to $O(K)$ binary steps
Noise Distribution	$P_n(w) \propto U(w)^{0.75}$	Dampens stopwords; enhances rare words
Vector Analogy	$\mathbf{d} = \arg\max_w \text{sim}(\mathbf{v}_w, \mathbf{v}_b - \mathbf{v}_a + \mathbf{v}_c)$	Linear semantic vector arithmetic

4. Assignment Solutions with Detailed Step-by-Step Explanations

Case Study 1: Customer Support Ticket Classification (Questions 1 to 5)

Context & Data

A telecom company processes 50,000 tickets with a vocabulary of $V = 12,000$ unique words. Each word is represented as a one-hot vector of length V. Bag-of-words ticket vectors are formed by summing word vectors: - Ticket A: `recharge failed refund` - Ticket B: `recharge plan refund`

Question 1

What is the length of each one-hot word vector? - A) 12,000 - B) 50,000 - C) 3 - D) 4

Correct Answer: A

- **Detailed Step-by-Step Explanation**: A one-hot vector has a dedicated dimension for every unique token in the vocabulary. Since the vocabulary size is $V = 12,000$, each one-hot vector has length $12,000$.

Question 2

How many non-zero entries does a single one-hot word vector contain? - A) 12,000 - B) 0 - C) 1 - D) 3

Correct Answer: C

- **Detailed Step-by-Step Explanation**: By mathematical definition, a one-hot vector has exactly one coordinate set to 1 (corresponding to the word's index) and all other $V - 1$ coordinates set to 0.

Question 3

How many shared words appear in Ticket A and Ticket B? - A) 1 - B) 3 - C) 0 - D) 2

Correct Answer: D

- **Detailed Step-by-Step Explanation**: Ticket A words: `{"recharge", "failed", "refund"}` . Ticket B words: `{"recharge", "plan", "refund"}` . Intersection: `{"recharge", "refund"}` $\implies$ 2 shared words.

Question 4

What is a major limitation of one-hot word vectors? - A) They are always dense for the data shown here. - B) They do not directly capture semantic similarity between words. - C) They require labels for every context window for the described model. - D) They have variable length for the same vocabulary.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: All pairs of distinct one-hot vectors are mutually orthogonal ($\mathbf{e}_i^\top \mathbf{e}_j = 0$), meaning they cannot encode linguistic nuances, synonyms, or conceptual similarity.

Question 5

Why can cosine similarity still be useful for bag-of-words ticket vectors? - A) It measures overlap in word usage while reducing sensitivity to length. - B) It learns neural hidden layers automatically from the sparse ticket vectors. - C) It converts sparse vectors into images in this specific scenario. - D) It guarantees semantic understanding under the given conditions.

Correct Answer: A

- **Detailed Step-by-Step Explanation**: Cosine similarity evaluates the normalized dot product $\frac{\mathbf{u}^\top \mathbf{v}}{|\mathbf{u}||\mathbf{v}|}$, which reflects relative word co-occurrence while normalizing for document length.

Case Study 2: Product Review Term Analysis (Questions 6 to 10)

Context & Data

Corpus: 1. `battery lasts long` 2. `screen looks bright` 3. `battery charges fast` 4. `bright screen response` Vocabulary: `[battery, screen, bright, long, fast]` ($V = 5$). Context window: $c = 1$ (immediate left and right neighbors).

Question 6

What is the size of the co-occurrence matrix for this selected vocabulary? - A) 4×5 - B) 5×5 - C) 5×4 - D) 25×25

Correct Answer: B

- **Detailed Step-by-Step Explanation**: The word-word co-occurrence matrix has rows corresponding to target words ($V = 5$) and columns corresponding to context words ($V = 5$), resulting in a 5×5 matrix.

Question 7

What does a context window of 1 mean? - A) Only the target word is counted. - B) All words in the corpus are counted as context. - C) The vocabulary is reduced to one word. - D) Only immediate neighboring words are counted.

Correct Answer: D

- **Detailed Step-by-Step Explanation**: A symmetric window of size 1 encompasses only the immediate predecessor (left neighbor) and successor (right neighbor) of the target token.

Question 8

In the fragment "battery lasts long", how many context contributions does "lasts" receive with window size 1? - A) 2 - B) 1 - C) 0 - D) 3

Correct Answer: A

- **Detailed Step-by-Step Explanation**: With $c = 1$, the word "lasts" has one word to its left ("battery") and one word to its right ("long"), giving 2 context contributions.

Question 9

How many times are "screen" and "bright" adjacent in the toy corpus? - A) 0 - B) 2 - C) 1 - D) 3

Correct Answer: C

- **Detailed Step-by-Step Explanation**: Let's check each fragment:
- `battery lasts long` : Neither word appears.
- `screen looks bright` : "screen" and "bright" are separated by "looks" (distance is 2, not adjacent).
- `battery charges fast` : Neither appears.
- `bright screen response` : "bright" is immediately adjacent to "screen" (distance 1). Thus, they are immediately adjacent exactly **1** time.

Question 10

What does a row of the co-occurrence matrix represent? - A) The number of labels in the classifier under the case assumptions. - B) The vectorial representation of a target word based on its context counts. - C) The one-hot position of a document category in the downstream product-review classifier. - D) The learning rate selected for truncated SVD.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Each row of the co-occurrence matrix corresponds to a target word, and the entries represent its co-occurrence distribution across all context words.

Case Study 3: Legal Search Term Representation (Questions 11 to 15)

Context & Data

A legal search co-occurrence matrix X has dimension 6000×6000. Truncated SVD:

$$X \approx U_k \Sigma_k V_k^\top, \quad k = 200$$

Question 11

Why is truncated SVD applied here? - A) To increase the vocabulary size in the stated case setting. - B) To make every context vector one-hot. - C) To reduce dimensionality while preserving important structure. - D) To remove the need for context words in every downstream search task.

Correct Answer: C

- **Detailed Step-by-Step Explanation**: Truncated SVD compresses the high-dimensional (6000) sparse matrix into a compact (200) dense semantic embedding space while filtering out random noise.

Question 12

What is the dimension of each reduced word vector when $k = 200$? - A) 6000 - B) 36,000,000 - C) 400 - D) 200

Correct Answer: D

- **Detailed Step-by-Step Explanation**: Selecting $k = 200$ singular components projects each word onto a 200-dimensional latent subspace.

Question 13

What is the size of U_k in this case? - A) 6000×200 - B) 200×200 - C) 200×6000 - D) 6000×6000

Correct Answer: A

- **Detailed Step-by-Step Explanation**: For an input matrix of size $V \times V$ (6000×6000), the left singular matrix truncated to k columns has dimension $V \times k = 6000 \times 200$.

Question 14

Which advantage does the reduced representation provide? - A) It makes all unrelated words identical under the given conditions. - B) It can make similarity computation more efficient. - C) It prevents any word from appearing in context. - D) It removes all latent semantic structure.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Computing dot products and cosine similarities in 200 dimensions requires drastically fewer FLOPs and less memory than computing over 6000 dimensions.

Question 15

What kind of information can SVD help uncover? - A) The number of documents in chronological order. - B) The exact spelling of every rare word. - C) The chronological order of every document. - D) Latent patterns in word-context usage.

Correct Answer: D

- **Detailed Step-by-Step Explanation**: SVD discovers latent semantic concepts by grouping words that share similar context distributions, even if the words never co-occur directly.

Case Study 4: Search Term Similarity Evaluation (Questions 16 to 20)

Context & Data

Candidate cosine similarities for a query q: - A: 0.18 - B: 0.74 - C: 0.41 - D: 0.62 Analogy formulation: $a : b :: c :?$ uses vector search near $\mathbf{v}_b - \mathbf{v}_a + \mathbf{v}_c$.

Question 16

Which candidate is selected by the synonym rule? - A) A - B) B - C) C - D) D

Correct Answer: B

- **Detailed Step-by-Step Explanation**: The synonym rule selects the candidate with the maximum cosine similarity. Among $0.18, 0.74, 0.41, 0.62$, the maximum is 0.74 (Candidate B).

Question 17

What does a high correlation with human relatedness scores indicate? - A) The model's similarity judgments agree well with human judgments. - B) The vectors remain one-hot after training. - C) The vocabulary has only one word under the given conditions. - D) The model ignores cosine similarity during all human-score evaluations.

Correct Answer: A

- **Detailed Step-by-Step Explanation**: A high Spearman/Pearson correlation between vector cosine similarities and human-annotated similarity scores demonstrates that the model's geometric representation accurately reflects human linguistic intuition.

Question 18

In an analogy task $a : b :: c :?$, what vector operation is commonly used? - A) $\mathbf{v}_a + \mathbf{v}_b + \mathbf{v}_c$ only - B) $\mathbf{v}_a - \mathbf{v}_b - \mathbf{v}_c$ - C) $\mathbf{v}_b - \mathbf{v}_a + \mathbf{v}_c$ - D) $\mathbf{v}_a / \mathbf{v}_b$

Correct Answer: C

- **Detailed Step-by-Step Explanation**: The relationship vector is $\mathbf{d}_{\text{rel}} = \mathbf{v}_b - \mathbf{v}_a$. Adding this displacement to $\mathbf{v}_c$ yields $\mathbf{v}_b - \mathbf{v}_a + \mathbf{v}_c$.

Question 19

Why are dense word vectors useful for query expansion? - A) They make every vector a one-hot vector under the given conditions. - B) They require no training corpus under the given conditions. - C) They make every pair of different words orthogonal. - D) They can place related words near each other in vector space.

Correct Answer: D

- **Detailed Step-by-Step Explanation**: Dense word embeddings encode distributional semantics, clustering synonyms and related concepts in close proximity in vector space.

Question 20

What metric is used in the synonym rule described in the case? - A) Training loss under the given conditions - B) Matrix rank only - C) Cosine similarity - D) Vocabulary size

Correct Answer: C

- **Detailed Step-by-Step Explanation**: The decision rule explicitly ranks candidates based on their **cosine similarity** to the query vector.

---

5. NPTEL Exam Traps & High-Yield Pitfalls

1. **CBOW vs Skip-Gram Direction**:
2. **CBOW**: Input = Context words $\longrightarrow$ Output = Target center word.
3. **Skip-Gram**: Input = Target center word $\longrightarrow$ Output = Context words.
4. **Negative Sampling Exponent**:
5. The unigram distribution is raised to the power of 0.75 **(or $3/4$)**, not 0.5 or 1.0.
6. **Analogy Vector Sign**:
7. For $a : b :: c : d$, the formula is $\mathbf{v}_d \approx \mathbf{v}_b - \mathbf{v}_a + \mathbf{v}_c$. Be careful not to swap $\mathbf{v}_a$ and $\mathbf{v}_b$!

MODULE 10

Week 10: Convolutional Neural Networks (CNNs), Classic Architectures, and Visualization

Week 10: Convolutional Neural Networks (CNNs), Classic Architectures, and Visualization Techniques

1. Executive Summary & Exam Roadmap

Week 10 covers computer vision architectures and visual interpretability: 1. **The Convolution Operation**: Mathematical formulation (cross-correlation), receptive fields, kernel sliding, stride (S), and padding (P). 2. **Output Dimensions & Parameter Calculations**: - Spatial dimension formula: $O = \lfloor \frac{W-K+2P}{S} \rfloor + 1$. - Convolutional layer parameter count: $(\text{kernel_h} \times \text{kernel_w} \times C_{\text{in}} + 1) \times C_{\text{out}}$. 3. **Pooling Layers**: Max pooling and Average pooling; parameter-free spatial downsampling; translation invariance. 4. **Global Average Pooling (GAP)**: Collapsing $H \times W \times C$ tensors into $1 \times 1 \times C$ vectors, eliminating multi-million parameter dense layers. 5. **Evolution of Classic Architectures**: - **LeNet-5 (1998)** & **AlexNet (2012)**. - **VGGNet (2014)**: The power of stacked 3×3 filters. - **GoogLeNet / Inception (2014)**: Multi-scale parallel convolutions and 1×1 bottleneck reductions. - **ResNet (2015)**: Residual learning $\mathcal{H}(\mathbf{x}) = \mathcal{F}(\mathbf{x}) + \mathbf{x}$ and identity skip connections solving network degradation. 6. **Visualizing and Interpreting CNNs (Lecture 12)**: - Occlusion sensitivity analysis. - Guided Backpropagation and Class Activation Maps (CAM). - Deep Dream, Neural Style Transfer (Gram matrices), and Adversarial Perturbations (FGSM).

2. Comprehensive Lecture Notes & Mathematical Foundations

2.1 The Convolution Operation in Neural Networks

In deep learning, 2D convolution is implemented as cross-correlation between an input tensor I and a learnable kernel/filter K:

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(i+m, j+n) K(m,n)$$

For multi-channel inputs (e.g. RGB with $C_{\text{in}} = 3$): A single filter has dimensions $K_h \times K_w \times C_{\text{in}}$. It performs 3D spatial-channel dot products and sums over all input channels:

$$S(i,j) = b + \sum_{c=1}^{C_{\text{in}}} \sum_{m=1}^{K_h} \sum_{n=1}^{K_w} I(i+m, j+n, c) K(m,n,c)$$

If a layer has C_{out} distinct filters, it outputs an activation tensor with C_{out} channels/feature maps.

Spatial Dimension Formula

Given input spatial dimension W (height or width), filter size K, padding P, and stride S:

$$O = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

- **Valid Padding ($P = 0$)**: No padding; output shrinks: $O = \frac{W-K}{S} + 1$. - **Same Padding**: Preserves input spatial resolution ($O = W$) when $S = 1$:

$$W = \frac{W - K + 2P}{1} + 1 \implies 2P = K - 1 \implies P = \frac{K-1}{2}$$

(Requires an odd filter size K, e.g. $K = 3 \implies P = 1$; $K = 5 \implies P = 2$).

2.2 Pooling Layers & Global Average Pooling

Max Pooling & Average Pooling

- Operates independently on each feature map channel.
- A 2×2 window with stride 2 extracts the maximum (or average) in non-overlapping 2×2 blocks, exactly **halving the spatial height and width**:

$$O = \frac{W-2}{2} + 1 = \frac{W}{2}$$

- **Key Characteristics**:
- **Zero learnable parameters**: Pooling has no weights or biases.
- **Translation Invariance**: If an object shifts by a few pixels, the maximum value within the pooling window remains identical.

Global Average Pooling (GAP) (Lin et al., 2013)

Instead of flattening an $H \times W \times C$ feature tensor into a huge 1D vector of length $H \cdot W \cdot C$ and multiplying by a massive weight matrix, GAP takes the spatial average over each $H \times W$ feature map:

$$y_c = \frac{1}{H \times W} \sum_{i=1}^{H} \sum_{j=1}^{W} A(i, j, c)$$

- Maps an $H \times W \times C$ tensor directly to a $1 \times 1 \times C$ vector. - Eliminates millions of dense parameters, drastically reducing overfitting.

2.3 Evolution of Seminal CNN Architectures

```
LeNet-5 (1998) ---> AlexNet (2012) ---> VGGNet (2014) ---> GoogLeNet (2014) ---> ResNet (2015)
(5x5 filters)       (ReLU, Dropout)     (Stacked 3x3)      (Inception, 1x1)     (Skip Connections)
```

1. VGGNet (Simonyan & Zisserman, 2014)

Core Innovation: Replace large filters ($5 \times 5, 7 \times 7$) with homogeneous stacks of small 3×3 **filters** with stride 1. - **Receptive Field Equivalence**: - Two stacked 3×3 layers cover a receptive field of $(3-1)+(3-1)+1 = 5 \times 5$. - Three stacked 3×3 layers cover a receptive field of 7×7. - **Parameter Savings**: - One 5×5 filter across C channels has $5^2 \cdot C^2 = 25C^2$ parameters. - Two 3×3 filters across C channels have $2 \times (3^2 \cdot C^2) = 18C^2$ parameters (28% **fewer parameters**!). - Furthermore, incorporating two nonlinearities (e.g. ReLUs) instead of one makes the decision function significantly more discriminative.

2. GoogLeNet / Inception (Szegedy et al., 2014)

Core Innovation: **Inception Modules** with multi-scale parallel filter paths. - Instead of picking whether to use 1×1, 3×3, or 5×5, an Inception module runs all of them in parallel and concatenates their outputs depth-wise. - 1×1 **Convolutions (Network-in-Network)**: Used as computational bottlenecks before 3×3 and 5×5 filters to reduce channel depth ($C_{\text{in}} \to C_{\text{reduced}}$), drastically slashing multiply-accumulate operations.

3. ResNet (He et al., 2015)

The Degradation Problem: As plain networks get deeper (e.g. 56 layers vs 20 layers), training accuracy *saturates and then degrades rapidly*, not due to overfitting, but due to severe optimization difficulty.

Residual Learning Formulation: Instead of hoping a stack of layers fits an underlying mapping $\mathcal{H}(\mathbf{x})$, let the layers fit a residual mapping $\mathcal{F}(\mathbf{x}) = \mathcal{H}(\mathbf{x}) - \mathbf{x}$. The original mapping is recast as:

$$\mathcal{H}(\mathbf{x}) = \mathcal{F}(\mathbf{x}) + \mathbf{x}$$

via a feedforward **identity shortcut (skip connection)**.

Why Residual Connections Cure Gradient Vanishing:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathcal{H}} \cdot \frac{\partial \mathcal{H}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathcal{H}} \left(\frac{\partial \mathcal{F}}{\partial \mathbf{x}} + \mathbf{I} \right) = \frac{\partial \mathcal{L}}{\partial \mathcal{H}} \frac{\partial \mathcal{F}}{\partial \mathbf{x}} + \frac{\partial \mathcal{L}}{\partial \mathcal{H}}$$

- Even if the layer gradients $\frac{\partial \mathcal{F}}{\partial \mathbf{x}}$ vanish to zero, the additive $+\mathbf{I}$ term ensures that the gradient $\frac{\partial \mathcal{L}}{\partial \mathcal{H}}$ flows directly and unimpeded backwards through the skip connection across hundreds of layers!

2.4 Visualizing and Understanding CNNs (Lecture 12)

1. **Occlusion Sensitivity Analysis (Zeiler & Fergus, 2013)**:
2. Systematically place a gray masking square over different spatial patches (i, j) of the input image and record the model's output probability for the correct class.
3. If masking a specific region causes the class confidence to plummet, that region contains the critical discriminative visual evidence.
4. **Gradient-Based Saliency & Guided Backpropagation**:
5. Compute the gradient of the target class score w.r.t input pixels: $\nabla_{\mathbf{x}} S_c(\mathbf{x})$. Pixels with large gradient magnitudes have the highest influence on the classification.
6. **Guided Backpropagation**: Zeroes out negative gradients during backpropagation through ReLUs, suppressing noise and producing sharp, human-interpretable feature visualizations.
7. **Adversarial Examples (Goodfellow et al., 2014)**:
8. Deep CNNs learn linear decision boundaries in high-dimensional pixel space that can be fooled by adding human-imperceptible perturbations aligned with the sign of the loss gradient: $$\mathbf{x}*{\text{adv}} = \mathbf{x} + \epsilon \, \text{sign}(\nabla_{\mathbf{x}} J(\mathbf{x}, y))$$ }} \mathcal{L}(\mathbf{x*(Fast Gradient Sign Method - FGSM).*

---

3. Core Formulas & Cheat Sheet

Concept	Mathematical Equation	Key Notes
Output Spatial Size	$O = \lfloor \frac{W-K+2P}{S} \rfloor + 1$	W: input, K: kernel, P: padding, S: stride
"Same" Padding	$P = \frac{K-1}{2}$	Valid when $S = 1$ and K is odd
Conv Layer Weights	$K_h \times K_w \times C_{\text{in}} \times C_{\text{out}}$	Ignores biases
Conv Layer Biases	C_{out}	Exactly 1 bias per output feature map
Pooling Output Size	$O = \frac{W-K_{\text{pool}}}{S_{\text{pool}}} + 1$	Has 0 learnable parameters
Global Avg Pooling	$y_c = \frac{1}{HW} \sum_{i,j} A_{ijc}$	Collapses $H \times W \times C \to 1 \times 1 \times C$
Residual Block	$\mathcal{H}(\mathbf{x}) = \mathcal{F}(\mathbf{x}) + \mathbf{x}$	Identity skip connection guarantees gradient highway
FGSM Adversarial	$\mathbf{x}_{\text{adv}} = \mathbf{x} + \epsilon \, \text{sign}(\nabla	Perturbation of magnitude ϵ fools CNN

4. Assignment Solutions with Detailed Step-by-Step Explanations

Case Study 1: Warehouse Package Image Inspection (Questions 1 to 5)

Context & Data

A logistics CNN inspects grayscale package images of size 96×96 ($C_{\text{in}} = 1$). - First convolution: 3×3 filters, stride $S = 1$, padding $P = 1$. - 24 filters in the layer. - Followed by 2×2 max pooling with stride $S = 2$.

Question 1

What is the output spatial size after the 3×3 convolution with stride 1 and padding 1? - A) 96×96 - B) 94×94 - C) 98×98 - D) 48×48

Correct Answer: A

- **Detailed Step-by-Step Explanation**: Using the dimension formula:

$$O = \frac{W - K + 2P}{S} + 1 = \frac{96 - 3 + 2(1)}{1} + 1 = \frac{95}{1} + 1 = 96$$

  The spatial size is completely preserved (96×96).

Question 2

If the convolution layer uses 24 filters, how many output feature maps are produced? - A) 3 - B) 1 - C) 24 - D) 96

Correct Answer: C

- **Detailed Step-by-Step Explanation**: Each individual convolutional filter produces exactly one 2D output feature map. Therefore, 24 filters produce 24 output feature maps ($C_{\text{out}} = 24$).

Question 3

What is the output size after applying 2×2 max pooling with stride 2 to a 96×96 feature map? - A) 96×96 - B) 24×24 - C) 94×94 - D) 48×48

Correct Answer: D

- **Detailed Step-by-Step Explanation**:

$$O_{\text{pool}} = \frac{W - K_{\text{pool}}}{S_{\text{pool}}} + 1 = \frac{96 - 2}{2} + 1 = \frac{94}{2} + 1 = 47 + 1 = 48$$

  Both dimensions are halved to 48×48.

Question 4

Why is pooling useful for this inspection task? - A) It removes the need for convolution under the case assumptions. - B) It reduces spatial size and gives some robustness to small shifts. - C) It increases every image dimension in the stated case setting. - D) It converts grayscale images into RGB images in this specific scenario.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Pooling downsamples feature maps (reducing computation in deeper layers) and provides local translation invariance, ensuring that minor shifts in package placement do not alter feature activations.

Question 5

What do early convolution filters usually learn in such image tasks? - A) Low-level patterns such as edges and corners - B) The final class probability only - C) The batch size for the described model - D) The optimizer momentum in this specific scenario

Correct Answer: A

- **Detailed Step-by-Step Explanation**: Hierarchical feature learning in CNNs dictates that early layers learn primitive, low-level spatial features (Gabor-like edge detectors, color blobs, corners), while deeper layers compose them into complex semantic parts.

Case Study 2: Crop Image Disease Detection (Questions 6 to 10)

Context & Data

A CNN classifies crop leaves. Techniques used: - Occlusion analysis (masking rectangular patches). - Gradient-based pixel saliency.

Question 6

If masking one leaf patch sharply reduces disease probability, what does that suggest? - A) The patch is irrelevant to the prediction and all class evidence. - B) The patch contains information important for the prediction. - C) The model has no learned filters for the data shown here. - D) The image has no disease signal for the data shown here.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: A steep drop in target class confidence upon occluding a region indicates that the model's decision boundary is heavily reliant on visual features located within that specific region.

Question 7

What is the name of the technique that masks image regions and observes prediction changes? - A) Batch normalization - B) Weight decay - C) Dropout - D) Occlusion analysis

Correct Answer: D

- **Detailed Step-by-Step Explanation**: Systematically covering parts of an input image with a neutral gray or black mask to track changes in output classification probability is called **Occlusion Analysis** (Zeiler & Fergus).

Question 8

Why are first-layer filters usually easier to interpret? - A) They operate directly on local pixel patterns. - B) They contain no weights in this specific scenario. - C) They always output class labels. - D) They are identical across all filters.

Correct Answer: A

- **Detailed Step-by-Step Explanation**: First-layer filters have small receptive fields and interact directly with raw input RGB pixels, making them readily visualizable as edge detectors, color contrasts, or oriented gradients.

Question 9

A neuron that activates strongly for diseased leaves but weakly for healthy leaves has likely learned what? - A) The file name pattern in this specific scenario - B) A disease-related visual feature - C) The learning rate schedule - D) A fixed image size only

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Class-selective activation across samples indicates that the filter is tuned to visual hallmarks characteristic of the disease class (e.g. necrotic spots, yellow lesions).

Question 10

In gradient-based visualization, what does a large gradient magnitude for a pixel indicate? - A) The pixel has no influence for the data shown here. - B) The pixel is outside the image. - C) The activation is sensitive to that pixel. - D) The filter has zero parameters.

Correct Answer: C

- **Detailed Step-by-Step Explanation**: By definition of partial derivatives, $\frac{\partial S_c}{\partial I_{ij}}$ represents the sensitivity of score S_c to pixel I_{ij}. A high absolute value means small modifications to that pixel produce large changes in model confidence.

Case Study 3: Road Surface Image Classification (Questions 11 to 15)

Context & Data

Architectures compared: VGG (3×3 stacks), Inception (multi-scale parallel filters), and ResNet (residual skip connections).

Question 11

Why does VGG prefer stacks of small 3×3 filters? - A) To remove nonlinearities in the stated case setting. - B) To avoid feature maps throughout the network. - C) To force all images to be grayscale before every convolutional layer. - D) To build deeper representations with controlled parameter growth.

Correct Answer: D

- **Detailed Step-by-Step Explanation**: Stacking two 3×3 filters matches the receptive field of a 5×5 filter while using 28% fewer parameters and inserting two ReLU nonlinearities, enabling deeper, more expressive representations.

Question 12

What is the main purpose of Inception-style parallel filters? - A) To delete pooling layers. - B) To use only one spatial scale. - C) To capture features at multiple spatial scales. - D) To remove convolution in this specific scenario.

Correct Answer: C

- **Detailed Step-by-Step Explanation**: The Inception block applies $1 \times 1, 3 \times 3$, and 5×5 convolutions in parallel, allowing the network to simultaneously extract fine details and large contextual features at the same layer.

Question 13

Why do ResNet skip connections help very deep networks train? - A) They improve information and gradient flow. - B) They remove the input image. - C) They replace all filters with one scalar. - D) They make the output size independent of stride.

Correct Answer: A

- **Detailed Step-by-Step Explanation**: The identity skip connection creates an additive gradient highway ($\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathcal{H}} \frac{\partial \mathcal{F}}{\partial \mathbf{x}} + \frac{\partial \mathcal{L}}{\partial \mathcal{H}}$) that allows error signals to backpropagate directly without vanishing.

Question 14

A module computes $\mathcal{H}(x) = \mathcal{F}(x) + x$. What architecture idea does this represent? - A) Bag-of-words - B) Residual learning - C) One-hot encoding - D) Average pooling only

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Formulating the output as the sum of a learned residual function $\mathcal{F}(x)$ and the identity shortcut x is the defining equation of **Residual Learning (ResNet)**.

Question 15

Which architecture is most directly suited to mixing $1 \times 1, 3 \times 3$, and 5×5 convolutions in parallel? - A) Inception in this specific scenario - B) A single-layer perceptron - C) Plain linear regression - D) A one-hot encoder for the described model

Correct Answer: A

- **Detailed Step-by-Step Explanation**: The **Inception architecture** (GoogLeNet) is explicitly designed around parallel multi-branch modules mixing $1 \times 1, 3 \times 3$, and 5×5 kernels.

Case Study 4: Satellite Land-Cover Image Classification (Questions 16 to 20)

Context & Data

A satellite classification network processes 64×64 RGB patches ($C_{\text{in}} = 3$). - Convolution: 32 filters, 5×5, stride $S = 1$, padding $P = 0$. - Later stage: Global Average Pooling applied to an $8 \times 8 \times 128$ tensor.

Question 16

What is the output spatial size after the 5×5, stride 1, no-padding convolution on a 64×64 image? - A) 64×64 - B) 32×32 - C) 60×60 - D) 68×68

Correct Answer: C

- **Detailed Step-by-Step Explanation**:

$$O = \frac{W - K + 2P}{S} + 1 = \frac{64 - 5 + 0}{1} + 1 = 59 + 1 = 60$$

  The spatial size is 60×60.

Question 17

How many output channels are produced by 32 filters? - A) 5 - B) 32 - C) 64 - D) 128

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Each filter generates exactly one output channel. Thus, 32 filters generate 32 output channels ($C_{\text{out}} = 32$).

Question 18

How many weight parameters are in one 5×5 RGB filter, ignoring bias? - A) 25 - B) 50 - C) 125 - D) 75

Correct Answer: D

- **Detailed Step-by-Step Explanation**: An RGB image has 3 color channels ($C_{\text{in}} = 3$). One filter spans all 3 channels:

$$\text{Parameters} = K_h \times K_w \times C_{\text{in}} = 5 \times 5 \times 3 = 75$$

Question 19

What is the output dimension after global average pooling of an $8 \times 8 \times 128$ tensor? - A) 64 - B) 1024 - C) 128 - D) 8192

Correct Answer: C

- **Detailed Step-by-Step Explanation**: Global average pooling computes the spatial average across the $8 \times 8 = 64$ positions for each channel independently, reducing each 2D slice to 1 scalar. With 128 channels, the output is a 128-dimensional vector.

Question 20

Why can global average pooling reduce parameters compared with flattening followed by a large fully connected layer? - A) It increases the spatial dimensions. - B) It removes all learned filters in this specific scenario. - C) It increases the number of dense connections. - D) It produces one summary value per feature map.

Correct Answer: D

- **Detailed Step-by-Step Explanation**: Flattening an $8 \times 8 \times 128$ tensor yields an 8,192-dimensional vector. Connecting that to a dense layer of 1024 units requires $8192 \times 1024 \approx 8.4 \text{ million}$ weights. In contrast, GAP has **zero** weights and condenses each channel to a single summary statistic.

---

5. NPTEL Exam Traps & High-Yield Pitfalls

1. **Number of Parameters in Conv Layer**:
2. For 1 filter: $K_h \times K_w \times C_{\text{in}}$ weights $+1$ bias.
3. For all filters in the layer: $(K_h \times K_w \times C_{\text{in}} + 1) \times C_{\text{out}}$.
4. **Common Trap**: Forgetting to multiply by the input channels C_{in}!
5. **Pooling Parameters**:
6. Pooling layers (Max pooling, Average pooling, Global Average Pooling) have **EXACTLY ZERO learnable parameters**.
7. **ResNet Residual Formula**:
8. $\mathcal{H}(\mathbf{x}) = \mathcal{F}(\mathbf{x}) + \mathbf{x}$. The gradient is $\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathcal{H}} \left(\frac{\partial \mathcal{F}}{\partial \mathbf{x}} + \mathbf{I} \right)$. The identity matrix $\mathbf{I}$ prevents gradient vanishing.

MODULE 11

Week 11: Recurrent Neural Networks (RNNs), BPTT, LSTMs, and GRUs

Week 11: Recurrent Neural Networks (RNNs), Backpropagation Through Time, LSTMs, and GRUs

1. Executive Summary & Exam Roadmap

Week 11 covers temporal and sequential deep learning architectures: 1. **Sequence Modeling Paradigms**: One-to-one, one-to-many, many-to-one, and many-to-many sequence problems. 2. **Vanilla Recurrent Neural Networks (RNNs)**: Hidden state recurrence $\mathbf{s}t = |tanh(|mathbf{W}|mathbf{s}t)$, parameter sharing across time, and forward unfolding. 3. } + \mathbf{U}\mathbf{x}_t + \mathbf{bBackpropagation Through Time (BPTT)**: Unrolling the computational graph across T steps and calculating temporal parameter gradients. 4. **Vanishing and Exploding Gradients in Sequences**: The exponential decay/growth of Jacobian products $\prod \frac{\partial \mathbf{s}jj}{|partial |mathbf{s}}$; }**Gradient Clipping** and **Truncated BPTT**. 5. **Long Short-Term Memory Networks (LSTMs)**: The Constant Error Carousel (CEC), the additive cell state $\mathbf{c}**, and the three regulating** tes: Forget ($\mathbf{f}_t$), Input ($\mathbf{i}_t$), and Output ($\mathbf{o}_t$). 6. **Gated Recurrent Units (GRUs)**: Compact architectural design merging cell and hidden states, utilizing only two gates: Reset ($\mathbf{r}_t$) and Update ($\mathbf{z}_t$).

2. Comprehensive Lecture Notes & Mathematical Foundations

2.1 The Need for Recurrent Networks

Standard feedforward networks suffer from three fundamental limitations when modeling sequences: 1. **Fixed Input / Output Size**: Cannot process variable-length inputs (e.g. sentences with 5 vs. 50 words). 2. **No Position/Order Awareness**: Permuting input coordinates destroys meaning in temporal series. 3. **No Feature Sharing Across Time**: Features learned at time t_1 are not reused if they appear at time t_2.

Taxonomy of Sequence Learning Tasks

- **One-to-One**: Standard classification (Image → Category).
- **One-to-Many**: Image Captioning (Image → Sequence of words).
- **Many-to-One**: Sentiment Analysis / Transcript Intent (Sequence of words → Class label).
- **Many-to-Many (Synchronized)**: Video frame tagging / POS Tagging (Sequence of frames → Sequence of labels of equal length).
- **Many-to-Many (Asynchronous / Seq2Seq)**: Machine Translation (Sequence in source language → Sequence in target language of differing length).

2.2 Vanilla Recurrent Neural Networks (RNNs)



Forward Equations

At each discrete time step $t \in 1, 2, \dots, T$: 1. **Hidden State Pre-activation**: $$\mathbf{a}t = |mathbf{W}|mathbf{s}$$ 2. } + \mathbf{U}\mathbf{x}_t + \mathbf{bHidden State Activation**: $$\mathbf{s}_t = \tanh(\mathbf{a}t) = |tanh(|mathbf{W}|mathbf{s})$$ 3. } + \mathbf{U}\mathbf{x}_t + \mathbf{bOutput Layer**:

$$\hat{\mathbf{y}}_t = \text{softmax}(\mathbf{V}\mathbf{s}_t + \mathbf{c})$$

where: - $\mathbf{U} \in \mathbb{R}^{d_h \times d_x}$: Input-to-hidden weight matrix. - $\mathbf{W} \in \mathbb{R}^{d_h \times d_h}$: Recurrent hidden-to-hidden weight matrix. - $\mathbf{V} \in \mathbb{R}^{d_y \times d_h}$: Hidden-to-output weight matrix. - **Shared Parameters**: $\mathbf{U}, \mathbf{W}, \mathbf{V}$ are identically shared across all time steps.

2.3 Backpropagation Through Time (BPTT) & Gradient Pathologies

The total loss across a sequence of length T is: $$\mathcal{L} = \sum_{t=1}^T \mathcal{L}t

$$Evaluating the gradient w.r.t the recurrent matrix \mathbf{W} :$$

|frac{|partial |mathcal{L}}{|partial |mathbf{W}} = |sum

$$Using the chain rule across recurrent states :$$

\frac{\partial \mathcal{L}}^T \frac{\partial \mathcal{L}_t}{\partial \mathbf{W}t}{|partial |mathbf{W}} = |sum

$$The critical term is the temporal Jacobian product :$$

\frac{\partial \mathbf{s}}^t \frac{\partial \mathcal{L}_t}{\partial \mathbf{s}_t} \frac{\partial \mathbf{s}_t}{\partial \mathbf{s}_k} \frac{\partial \mathbf{s}_k}{\partial \mathbf{W}*t}{\partial \mathbf{s}k} = \prod^t \frac{\partial \mathbf{sj}{\partial \mathbf{s} = \prod}_j^2)$$}^t \mathbf{W}^\top \text{diag}(1 - \mathbf{s

The Dilemma:

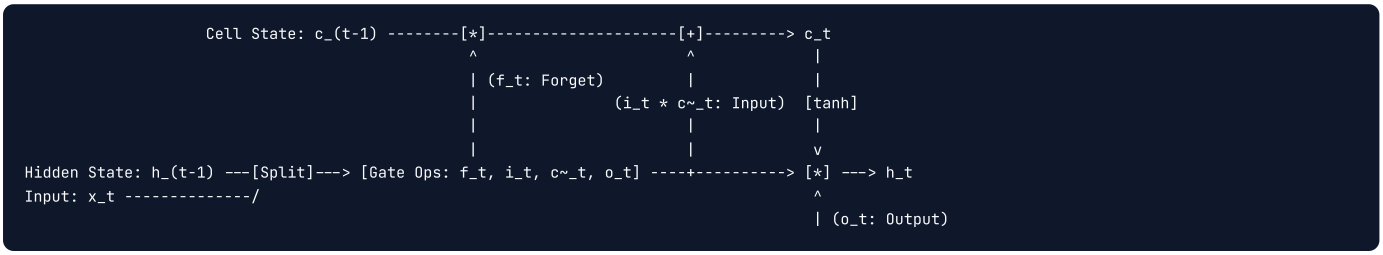
Let λ be the dominant eigenvalue of $\mathbf{W}$: 1. **If $\lambda > 1$ (Exploding Gradients)**: The product grows as λ^{t-k}. For large T, gradients become astronomical, causing NaN overflows and destabilizing parameters. - **Remedy: Gradient Clipping**:

$$\mathbf{g} \leftarrow \begin{cases} \mathbf{g} & \text{if } |\mathbf{g}|_2 \leq \theta \\ \frac{\theta}{|\mathbf{g}|_2}\mathbf{g} & \text{if } |\mathbf{g}|_2 > \theta \end{cases}$$

2. **If $\lambda < 1$ (Vanishing Gradients)**: The factor $(1 - \mathbf{s}_j^2) \leq 1$ combined with $\lambda < 1$ causes $\frac{\partial \mathbf{s}_t}{\partial \mathbf{s}_k} \to 0$ exponentially fast as $t - k$ grows. The network cannot bridge dependencies beyond 5–10 steps (forgetting long-term history). - **Remedy: Gated Architectures (LSTM / GRU)** and **Truncated BPTT**.

2.4 Long Short-Term Memory (LSTM) (Hochreiter & Schmidhuber, 1997)

The LSTM resolves vanishing gradients by replacing the plain hidden state with an internal memory cell governed by a **Constant Error Carousel (CEC)** with additive linear updates.



The Six Core LSTM Equations:

1. **Forget Gate ($\mathbf{f}_t$)**: Decides what fraction of information to erase from the previous cell state $\mathbf{c}_{t-1}$: $$\mathbf{f}_t = \sigma(\mathbf{W}*f \|mathbf{h}*f)$$} + \mathbf{U}_f \mathbf{x}_t + \mathbf{b
2. **Input Gate ($\mathbf{i}_t$)**: Decides which new components to write into the cell state: $$\mathbf{i}_t = \sigma(\mathbf{W}*i \|mathbf{h}*i)$$} + \mathbf{U}_i \mathbf{x}_t + \mathbf{b
3. **Candidate Cell State ($\tilde{\mathbf{c}}_t$)**: New candidate information synthesized from current input: $$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}*c \|mathbf{h}*c)$$} + \mathbf{U}_c \mathbf{x}_t + \mathbf{b
4. **Cell State Update ($\mathbf{c}_t$)**: Linear combination of past memory and new candidate: $$\mathbf{c}_t = \mathbf{f}*t \|odot \|mathbf{c}*t$$} + \mathbf{i}_t \odot \tilde{\mathbf{c}}
5. **Output Gate ($\mathbf{o}_t$)**: Controls what parts of the internal cell state are exposed to the external network: $$\mathbf{o}_t = \sigma(\mathbf{W}*o \|mathbf{h}*o)$$} + \mathbf{U}_o \mathbf{x}_t + \mathbf{b
6. **Hidden State ($\mathbf{h}_t$)**:

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

> **NOTE**
> > **Why LSTM Doesn't Suffer Vanishing Gradients**: > In the cell update equation $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \ldots$, the derivative w.r.t past memory is $\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \mathbf{f}_t$. If the forget gate is learned to be near 1 ($\mathbf{f}_t \approx 1$), the gradient flows through the cell state across hundreds of time steps with **zero attenuation**!

2.5 Gated Recurrent Unit (GRU) (Cho et al., 2014)

The GRU simplifies the LSTM by eliminating the separate cell state $\mathbf{c}_t$ and merging the forget and input gates into a coupled mechanism:

The Four Core GRU Equations:

1. **Reset Gate ($\mathbf{r}_t$)**: Determines how much past hidden state to forget when computing the candidate state: $$\mathbf{r}_t = \sigma(\mathbf{W}*r \|mathbf{h}*r)$$} + \mathbf{U}_r \mathbf{x}_t + \mathbf{b
2. **Update Gate ($\mathbf{z}_t$)**: Controls how much of the old state to carry forward vs. how much of the new candidate to accept: $$\mathbf{z}_t = \sigma(\mathbf{W}*z \|mathbf{h}*z)$$} + \mathbf{U}_z \mathbf{x}_t + \mathbf{b
3. **Candidate Hidden State ($\tilde{\mathbf{h}}_t$)**: $$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W} (\mathbf{r}*t \|odot \|mathbf{h}))$$}) + \mathbf{U} \mathbf{x}_t + \mathbf{b
4. **Hidden State Update ($\mathbf{h}_t$)**: Linear interpolation between previous state $\mathbf{h}_{t-1}$ and candidate $\tilde{\mathbf{h}}_t$: $$\mathbf{h}_t = (1 - \mathbf{z}*t) \|odot \|mathbf{h}*t$$} + \mathbf{z}_t \odot \tilde{\mathbf{h}}

---

3. Core Formulas & Cheat Sheet

Model	Gate Formulation	State Update Rule	Parameter Multiplier
Vanilla RNN	None	$\mathbf{s}*t = \|tanh(\|\mathbf{W}\|\mathbf{s}*)$} + \mathbf{U}\mathbf{x}_t + \mathbf{b	$1 \times (d_h^2 + d_h d_x)$
LSTM	$\mathbf{f}_t, \mathbf{i}_t, \mathbf{o}_t = \sigma(\dots)$ $\tilde{\mathbf{c}}_t = \tanh(\dots)$	$\mathbf{c}_t = \mathbf{f}*t |odot |\mathbf{c}*_t)Extra close brace or missing open brace $\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c	$4 \times (d_h^2 + d_h d_x + d_h)$
GRU	$\mathbf{r}_t, \mathbf{z}_t = \sigma(\dots)$ $\tilde{\mathbf{h}}_t = \tanh(\dots)$	$\mathbf{h}_t = (1 - \mathbf{z}*t) \|odot \|\mathbf{h}*_t$} + \mathbf{z}_t \odot \tilde{\mathbf{h}	$3 \times (d_h^2 + d_h d_x + d_h)$
Gradient Clipping	$\|\mathbf{g}\|_2 > \theta \implies \mathbf{g} \leftarrow \frac{\theta}{\|\mathbf{g}\|_2}\mathbf{g}$	Preserves gradient direction while capping magnitude	Prevents catastrophic parameter divergence

4. Assignment Solutions with Detailed Step-by-Step Explanations

Case Study 1: Call Transcript Intent Prediction (Questions 1 to 5)

Context & Data

A customer-care system predicts intent from phone transcripts. The model processes variable-length sequences using a many-to-one RNN architecture. In vanilla RNNs, early transcript details are forgotten by the time the final word is read.

Question 1

Why is an RNN suitable for transcript modeling? - A) It maintains a hidden state across sequence steps. - B) It requires fixed-size images under the given conditions. - C) It ignores word order completely. - D) It removes the need for training.

Correct Answer: A

- **Detailed Step-by-Step Explanation**: Recurrent neural networks maintain an evolving hidden state vector $\mathbf{s}_t$ that carries contextual memory from prior words across arbitrary sequence lengths.

Question 2

What does forgetting early transcript details mainly indicate? - A) The vocabulary is empty under the given conditions. - B) The hidden state has no update rule. - C) Earlier information is overwritten or weakened over time. - D) The output layer is convolutional in the stated case setting.

Correct Answer: C

- **Detailed Step-by-Step Explanation**: In a vanilla RNN, the hidden state is repeatedly overwritten via nonlinear matrix multiplications at each step, causing early information to be attenuated or overwritten (vanishing gradient problem).

Question 3

Which task structure describes predicting one transcript intent from a sequence of words? - A) Many-to-one - B) One-to-many - C) One-to-one - D) Many-to-many without recurrence

Correct Answer: A

- **Detailed Step-by-Step Explanation**: Processing a variable-length sequence of input tokens to produce a single final category prediction is the canonical definition of a **Many-to-One** sequence learning problem.

Question 4

Why can parameter sharing across time steps help in this task? - A) It forces every sentence to contain exactly the same words. - B) It allows the same pattern to be recognized anywhere in the transcript. - C) It removes the need for backpropagation through time at all. - D) It replaces word vectors with scalar numbers only.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Sharing transition matrices $\mathbf{W}, \mathbf{U}$ across all time steps enforces temporal translation invariance: a complaint phrase carries the same semantic meaning whether it appears at the beginning, middle, or end of the transcript.

Question 5

Which architecture is designed to reduce the forgetting of early information using gates? - A) Gated architectures such as LSTM and GRU - B) Unregularized linear regression - C) Plain bag-of-words without state - D) Single convolution filters only

Correct Answer: A

- **Detailed Step-by-Step Explanation**: Gated architectures (LSTMs and GRUs) use learnable multiplicative gates to explicitly regulate what to remember, write, and erase, protecting early signals from being overwritten.

Case Study 2: Machine Log Monitoring for Factory Alerts (Questions 6 to 10)

Context & Data

An RNN monitors factory event logs using Backpropagation Through Time (BPTT). - Truncated BPTT uses a window of 12 steps. - Gradient clipping is added to prevent unstable parameter updates.

Question 6

What does BPTT do? - A) It trains only the output layer under the given conditions. - B) It backpropagates errors through the unrolled sequence. - C) It removes recurrence under the given conditions. - D) It converts logs into images for the data shown here.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Backpropagation Through Time unfolds the recurrent computational graph across the time steps of the sequence and backpropagates error gradients from later steps back to earlier steps.

Question 7

What limitation is introduced by truncated BPTT with window size 12? - A) The network cannot make predictions for the described model. - B) The input sequence must have exactly 12 symbols. - C) Gradients are propagated only across the chosen recent window. - D) The hidden state must be zero forever under the case assumptions.

Correct Answer: C

- **Detailed Step-by-Step Explanation**: Truncated BPTT cuts the backward gradient flow after a fixed window (here, 12 steps), preventing the network from learning dependencies that span beyond that temporal horizon.

Question 8

What is the symptom of exploding gradients? - A) Very large parameter updates and unstable loss - B) No hidden state in this specific scenario - C) Fixed output vocabulary under the case assumptions - D) Smaller-than-zero sequence length

Correct Answer: A

- **Detailed Step-by-Step Explanation**: When gradients explode, parameter updates become excessively large, causing weights to oscillate wildly, resulting in loss spikes and numerical overflow (`NaN`).

Question 9

What does gradient clipping do? - A) It removes all gradients in this specific scenario. - B) It limits the magnitude of gradients above a threshold. - C) It increases every gradient without bound. - D) It replaces recurrence with convolution for the described model.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Gradient clipping rescales the gradient vector whenever its L_2 norm exceeds a threshold θ, ensuring the update step remains bounded while preserving the gradient's directional vector.

Question 10

Why can long recurrent chains create gradient problems? - A) The input has no meaningful order. - B) The sequence has no time steps during backpropagation through time at all. - C) The model has no trainable weights under the given conditions. - D) Repeated multiplication through time can shrink or grow gradients.

Correct Answer: D

- **Detailed Step-by-Step Explanation**: Backpropagating through time involves repeated matrix multiplications by $\mathbf{W}^\top \mathrm{diag}(1-\mathbf{s}^2)$, which exponentially contracts gradients to zero (if eigenvalues < 1) or blows them up to infinity (if eigenvalues > 1).

Case Study 3: Weather Alert Prediction from Hourly Observations (Questions 11 to 15)

Context & Data

An LSTM predicts storm alerts from hourly sensor logs using: - Forget gate ($\mathbf{f}_t$) - Input gate ($\mathbf{i}_t$) - Output gate ($\mathbf{o}_t$) - Cell state ($\mathbf{c}_t$)

Question 11

Which LSTM component directly stores long-term information? - A) The softmax label only - B) The one-hot input only - C) The cell state - D) The batch size

Correct Answer: C

- **Detailed Step-by-Step Explanation**: The internal **cell state** ($\mathbf{c}_t$) serves as the primary linear memory highway where information is preserved across extended temporal horizons.

Question 12

Which gate decides how much old cell-state information is retained? - A) Output gate - B) ReLU gate - C) Reset gate - D) Forget gate

Correct Answer: D

- **Detailed Step-by-Step Explanation**: The **forget gate** $\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{h}} + \mathbf{U}_f \mathbf{x}_t + \mathbf{b}_f) $outputsvaluesin[0, 1]$ that modu*ate how much of $\mathbf{c}_{t-1}$ is retained.

Question 13

Which gate controls how much new candidate information is written? - A) Input gate for the described model - B) Output gate in this specific scenario - C) Pooling gate - D) Softmax gate

Correct Answer: A

- **Detailed Step-by-Step Explanation**: The **input gate** $\mathbf{i}_t$ scales the candidate vector $\tilde{\mathbf{c}}_t$, deciding which new features are accumulated into the cell state.

Question 14

Why are sigmoid activations commonly used in gates? - A) They always output integers in this specific scenario. - B) They output values between 0 and 1 for soft control. - C) They remove all memory for the data shown here. - D) They make gradients infinite.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: The sigmoid function bounds outputs strictly between 0 (completely closed / block signal) and 1 (completely open / pass signal), providing a smooth, differentiable gating mechanism.

Question 15

What is the main advantage of LSTM gates in this case? - A) They remove all sequence dependence from the model. - B) They require fixed-length inputs only under the case assumptions. - C) They regulate remembering, writing, and exposing information over time. - D) They replace all observations with images before recurrent processing begins for alerts.

Correct Answer: C

- **Detailed Step-by-Step Explanation**: The gates allow the network to selectively retain persistent trends (e.g. barometric pressure drops) while filtering out transient sensor noise.

Case Study 4: Delivery Status Prediction from Event Logs (Questions 16 to 20)

Context & Data

A courier company uses a Gated Recurrent Unit (GRU) to predict delivery status: - Update gate ($\mathbf{z}_t$) - Reset gate ($\mathbf{r}_t$)

Question 16

What is a key difference between a GRU and an LSTM? - A) A GRU uses convolution filters only. - B) A GRU has no recurrent state under the case assumptions. - C) A GRU cannot process sequences. - D) A GRU is a more compact gated recurrent architecture.

Correct Answer: D

- **Detailed Step-by-Step Explanation**: The GRU combines cell state and hidden state into one representation and uses 2 gates instead of 3, resulting in a more compact model with fewer parameters and faster execution.

Question 17

Which GRU gate controls how much previous hidden state is retained? - A) Update gate - B) Output gate - C) Pooling gate - D) Vocabulary gate

Correct Answer: A

- **Detailed Step-by-Step Explanation**: In a GRU, the **update gate** $\mathbf{z}_t$ acts as a linear blender between the old hidden state and the candidate state: $\mathbf{h}_t = (1 - \mathbf{z}*t) \odot *mathbf{h}*_t$.} + \mathbf{z}_t \odot \tilde{\mathbf{h}}

Question 18

What does the reset gate control? - A) The number of output classes for the described model - B) The amount of past hidden state used in forming the candidate state - C) The image filter size used by the model - D) The batch-normalization variance computed before each recurrent candidate update

Correct Answer: B

- **Detailed Step-by-Step Explanation**: The **reset gate** $\mathbf{r}t scales *mathbf{h}*when computing the candidate activation :\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}(\mathbf{r*t \odot *mathbf{h}*}$.}) + \mathbf{U}\mathbf{x}_t + \mathbf{b

---

Question 19

Why might the courier team choose a GRU? - A) It removes all trainable parameters in this specific scenario. - B) It requires no event ordering in this specific scenario. - C) It is compact while still handling sequence dependencies. - D) It works only on images for the data shown here.

Correct Answer: C

- **Detailed Step-by-Step Explanation**: GRUs offer comparable modeling capacity to LSTMs on many real-world tasks while utilizing fewer parameters, training faster, and requiring less memory.

Question 20

Which problem do GRU and LSTM architectures primarily address compared with vanilla RNNs? - A) Increasing vocabulary sparsity in this specific scenario - B) Removing all sequence order from the data - C) Removing the need for hidden states in recurrent sequence models entirely - D) Capturing longer-range dependencies using controlled information flow

Correct Answer: D

- **Detailed Step-by-Step Explanation**: Both gated architectures prevent gradient vanishing through additive highways, allowing the network to capture dependencies across long temporal intervals.

---

5. NPTEL Exam Traps & High-Yield Pitfalls

1. **Parameter Count Comparison**:
2. An LSTM layer with d_h hidden units and d_x input units has $4\times$ the parameters of a vanilla RNN:

$$\text{Params}_{\text{LSTM}} = 4 \times [d_h(d_h + d_x) + d_h]$$

3. A GRU layer has $3\times$ the parameters of a vanilla RNN:

$$\text{Params}_{\text{GRU}} = 3 \times [d_h(d_h + d_x) + d_h]$$

4. **Gradient Clipping Function**:
5. Clipping scales down the *norm* of the gradient vector: $\mathbf{g} \leftarrow \frac{\theta}{|\mathbf{g}|}\mathbf{g}$. It **does not** change the direction of the gradient, only its magnitude.
6. **Activation Functions in Gates**:
7. Gates always use **Sigmoid (σ)** because outputs must be strictly in $[0, 1]$ to act as percentage filters.
8. States and candidates always use $\tanh$ to allow activations to be both positive and negative around zero.

MODULE 12

Week 12: Encoder-Decoder Models, Attention Mechanisms, and Transformers

Week 12: Encoder-Decoder Models, Attention Mechanisms, and Transformers

1. Executive Summary & Exam Roadmap

Week 12 culminates the course with modern sequence transduction and the Attention revolution: 1. **The Sequence-to-Sequence (Seq2Seq) Architecture**: - The Encoder compresses an arbitrary-length input sequence into a fixed-length vector. - The Decoder unrolls autoregressively, predicting target tokens one step at a time conditioning on previous outputs. 2. **The Information Bottleneck**: Performance degrades precipitously as sequence length increases because a single vector cannot retain all syntactic and factual nuances. 3. **The Attention Mechanism (Bahdanau et al., 2014; Luong et al., 2015)**: - Calculating alignment scores e_{ti} between decoder states and all encoder hidden states. - Softmax normalization yielding non-negative attention weights α_{ti} ($\sum \alpha_{ti} = 1$). - Dynamic context vector computation as a weighted sum $\mathbf{c}*t = |sum_i |alpha_*i$. 4. } \mathbf{h**Self-Attention & Scaled Dot-Product Attention (Vaswani et al., 2017)**: - Query (Q), Key (K), and Value (V) projections. - The scaled dot-product formula: $\text{softmax}\left(\frac{QK^\top}{\sqrt{d**}\right)V$. - Whscale by $\sqrt{d**}$? Preventing softmax saturati** for large vector dimensions. 5. **Positional Encodings**: Supplying essential word order and distance signals to permutation-invariant attention matrices. 6. **Hierarchical Attention Networks**: Word-level attention composing sentences, followed by sentence-level attention composing documents.

2. Comprehensive Lecture Notes & Mathematical Foundations

2.1 The Classic Encoder-Decoder (Seq2Seq) Architecture

Used for asynchronous sequence-to-sequence mappings where input and output lengths differ ($T_x \neq T_y$), such as Machine Translation, Summarization, and Question Answering.

```
Source: x_1, x_2, ..., x_Tx
          |    |          |
Encoder: (h_1)-(h_2)-...-(h_Tx) ═══[ Fixed Context Vector c = h_Tx ]===> Decoder: (s_1) ---> (s_2) ---> ...
                                                                                 |          |
                                                                      Target:  y_1        y_2
```

Encoder

Processes source sequence $\mathbf{x} = (x_1, x_2, \ldots, x_{T_x})$: $$\mathbf{h}*t = f(|mathbf{x}*t, |mathbf{h}})$$ *The final hidden state $|mathbf{h}*:\mathbf{c} = \mathbf{h}_{T_x}$* Extra close brace or missing open brace is taken as the single fixed-length summary context vector $\mathbf{c

Decoder

Autoregressively generates output tokens $\mathbf{y} = (y_1, y_2, \ldots, y_{T_y})$: $$\mathbf{s}*t = g(y}, \mathbf{s*{t-1}, |mathbf{c}})$$

$$P(y_t \mid y_<t)$$, \mathbf{x}) = \text{softmax}(\mathbf{V}\mathbf{s

The Information Bottleneck

Forcing all semantic details of a 50-word sentence or multi-page medical record into a single fixed-size vector (e.g., 512 dimensions) causes severe information loss. Early critical clauses (e.g., "avoid aspirin", "recharge failed") are washed away by subsequent tokens.

2.2 The Attention Mechanism (Bahdanau et al., 2014)

Instead of forcing the encoder to compress everything into one fixed vector $\mathbf{c}$, the Attention mechanism retains **all intermediate encoder states** ${\mathbf{h}_1, \mathbf{h}*2, |dots, |mathbf{h}*{T}**$. At every decoding time ste**t, the decoder generates a tailored, **time-dependent context vector** $\mathbf{c}_t$ by dynamically attending to the most relevant source positions.

```
Decoder State s_(t-1)
       |
       +-----> [ Alignment Score e_(t, i) ] <----- All Encoder States h_i
                        |
                [ Softmax Normalization ]
                        |
                Attention Weights alpha_(t, i)  (sum to 1)
                        |
                Context Vector c_t = Sum(alpha_(t, i) * h_i)
```

The Four Mathematical Steps of Attention:

1. **Alignment / Scoring Function (e_{ti})**: Measures the affinity or relevance between the previous decoder state $\mathbf{s}_{t-1}$ and each source encoder state $\mathbf{h}_i$:

2. **Dot product (Luong)**:

$$e_{ti} = \mathbf{s}_{t-1}^\top \mathbf{h}_i$$

3. **General / Multiplicative (Luong)**:

$$e_{ti} = \mathbf{s}_{t-1}^\top \mathbf{W}_a \mathbf{h}_i$$

4. **Additive / Concatenation (Bahdanau)**: $$e_{ti} = \mathbf{v}_a^\top \tanh(\mathbf{W}*a |mathbf{s}*i)$$ + \mathbf{U}_a \mathbf{h
5. **Softmax Normalization (Attention Weights α_{ti})**: Converts raw unbounded alignment scores into a valid probability distribution over source positions:

$$\alpha_{ti} = \frac{\exp(e_{ti})}{\sum_{k=1}^{T_x} \exp(e_{tk})}, \quad \text{where } \alpha_{ti} \geq 0 \text{ and } \sum_{i=1}^{T_x} \alpha_{ti} = 1$$

6. **Dynamic Context Vector Computation ($\mathbf{c}_t$)**: Takes the expectation (weighted average) of the encoder hidden states: $$\mathbf{c}*t = |sum*i$$}^{T_x} \alpha_{ti} \mathbf{h
7. **Conditional Prediction**: The current decoder state $\mathbf{s}_t$ and output probability are conditioned directly on $\mathbf{c}*t:$|mathbf{s}*t = g(|mathbf{s}*, y}, \mathbf{c*t)

*P(y_t | y_t])$$}, \mathbf{x}) = \text{softmax}(\mathbf{W}_o [\mathbf{s}_t; \mathbf{c

---

2.3 Self-Attention & The Transformer (Vaswani et al., 2017)

While Bahdanau attention connects a decoder to an encoder, **Self-Attention** connects tokens within the *same* sequence to one another, removing recurrent loops and enabling massive GPU parallelization.

Query, Key, and Value (Q, K, V) Abstraction

For each input token embedding $\mathbf{x}_i \in \mathbb{R}^d$, linear projection matrices project it into three distinct roles: 1. **Query ($\mathbf{q}_i = \mathbf{W}_Q \mathbf{x}_i \in \mathbb{R}^{d_k}$)**: What this token is looking for. 2. **Key ($\mathbf{k}_i = \mathbf{W}_K \mathbf{x}_i \in \mathbb{R}^{d_k}$)**: What this token contains (used for matching). 3. **Value ($\mathbf{v}_i = \mathbf{W}_V \mathbf{x}_i \in \mathbb{R}^{d_v}$)**: The actual informational content passed forward.

Scaled Dot-Product Attention

Pack vectors into matrices $Q, K \in \mathbb{R}^{N \times d_k}$ and $V \in \mathbb{R}^{N \times d_v}$:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Why Scale by $\frac{1}{\sqrt{d_k}}$?

Assume components of $\mathbf{q}$ and $\mathbf{k}$ are independent random variables with zero mean and variance 1. The dot product $\mathbf{q}^\top \mathbf{k} = \sum_{i=1}^{d_k} q_i k_i$ has mean 0 and variance d_k:

$$\text{Var}(\mathbf{q}^\top \mathbf{k}) = d_k$$

- For large dimensions (e.g. $d_k = 64$ or 128), the dot products grow very large in magnitude. - Feeding large values into the softmax function pushes it into extreme saturation regions where gradients $\frac{\partial \text{softmax}}{\partial z}$ are virtually zero. - Dividing by $\sqrt{d_k}$ scales the variance back down to 1, preserving healthy, robust gradient flow during backpropagation!

2.4 Positional Encodings

The Permutation Invariance Problem

In self-attention, the operation $\text{softmax}(QK^\top/\sqrt{d_k})V$ is strictly permutation-equivariant: shuffling the input tokens shuffles the output representations identically without changing the computed features. Unlike RNNs or CNNs, attention has **no inherent notion of word order**.

Sinusoidal Positional Encoding

To inject order, fixed positional vectors $PE_{(pos)}$ of dimension d are directly added to the input token embeddings: $$\mathbf{x}*{|text{input}} = |text{Embedding} (|text{token}) + PE

$$Using sine and cosine functions of varying frequencies:$$

PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)

PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$ - Allows the model to attend easily by relative position, because for any fixed offset k, $PE_{(pos+k)}$ can be represented as a linear function of $PE_{(pos)}$.

2.5 Multi-Head Attention & Hierarchical Structures

Multi-Head Attention

Instead of performing a single attention function, Multi-Head Attention linearly projects queries, keys, and values h times with different, learned linear projections to d_k, d_k, and d_v dimensions:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}^O$$

$$\text{where head}_j = \text{Attention}(Q\mathbf{W}_j^Q, K\mathbf{W}_j^K, V\mathbf{W}_j^V)$$

- Enables the network to jointly attend to information from different representation subspaces (e.g., one head tracks syntactic subject-verb dependencies, another tracks coreference, another tracks topic semantics).

Hierarchical Attention Networks (HAN)

For long hierarchical documents: 1. **Word-Level Attention**: Encodes word vectors into a unified sentence vector, weighting key phrases. 2. **Sentence-Level Attention**: Encodes sentence vectors into a final document vector, weighting key paragraphs.

---

3. Core Formulas & Cheat Sheet

Mechanism	Mathematical Formula	Function / Role
Alignment Score	$e_{ti} = \mathbf{s}_{t-1}^\top \mathbf{h}_i$	Affinity between decoder state $\mathbf{s}$ and encoder state $\mathbf{h}$
Attention Weights	$\alpha_{ti} = \frac{\exp(e_{ti})}{\sum_k \exp(e_{ik})}$	Softmax probability distribution ($\sum_i \alpha_{ti} = 1$)
Context Vector	$\mathbf{c}_t = \sum_i \alpha_i \mathbf{h}	Dynamic, time-dependent weighted representation
Scaled Dot-Product	$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$	Fundamental engine of Transformer architectures
Scaling Factor	$\frac{1}{\sqrt{d_k}}$	Normalizes variance of dot products to 1; prevents softmax saturation
Positional Encoding	$PE_{(pos,2i)} = \sin(pos/10000^{2i/d})$	Injects sequence order into attention representations
Multi-Head Attention	$\text{Concat}(\text{head}_1, \dots, \text{head}_h)\mathbf{W}^O$	Captures diverse subspace relationships simultaneously

---

4. Assignment Solutions with Detailed Step-by-Step Explanations

Case Study 1: Clinic Discharge Note Conversion (Questions 1 to 5)

Context & Data

A hospital converts doctors' discharge notes into patient-friendly language using an encoder-decoder architecture. In an early prototype, a single fixed-length summary vector is passed from the encoder to the decoder.

Question 1

In the early prototype, what is the main risk of using only one final summary vector? - A) Important source details may be compressed away. - B) The decoder must output every source word unchanged. - C) The encoder is unable to read numerical tokens. - D) The vocabulary becomes empty during generation.

Correct Answer: A - **Detailed Step-by-Step Explanation**: Forcing a long, detailed document through a single fixed-length vector creates an information bottleneck, causing critical early details (e.g., drug contraindications or fever warnings) to be lost.

Question 2

What does the encoder mainly provide to the decoder in this sequence model? - A) A sorted list of output tokens for the note - B) The final patient label after thresholding - C) Representations derived from the source note - D) A replacement for the training loss function

Correct Answer: C

- **Detailed Step-by-Step Explanation**: The encoder's role is to ingest the source text tokens and construct contextual hidden representations that capture the semantic content of the document.

Question 3

Why is the decoder described as producing the converted note token by token? - A) It ignores all tokens generated earlier. - B) Each new token can depend on earlier output tokens. - C) It computes the encoder states after every output only. - D) Every output token must have the same probability.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Autoregressive decoding generates language sequentially, allowing each newly predicted word to condition on the full history of previously generated tokens: $P(y_t | y_1, \dots, y_{t-1}, \mathbf{c})$.

Question 4

Which change most directly helps the decoder recover a warning sentence from early in a long note? - A) Removing all source-token representations - B) Replacing the decoder with a lookup table - C) Using only the last generated token as input - D) Allowing access to relevant encoder states

Correct Answer: D

- **Detailed Step-by-Step Explanation**: Implementing an attention mechanism grants the decoder direct, dynamic access to all intermediate encoder hidden states, allowing it to retrieve early warning tokens regardless of document length.

Question 5

If a long converted note omits an early allergy warning, what does that suggest about the fixed-summary design? - A) The final summary did not preserve all needed information. - B) The output language has no word order requirement at any step. - C) The source note was processed without any token representations. - D) The decoder cannot produce medical vocabulary for short notes.

Correct Answer: A

- **Detailed Step-by-Step Explanation**: Omitting early source information is the classic symptom of information degradation caused by fixed-vector bottleneck compression.

Case Study 2: Maintenance Ticket Digest (Questions 6 to 10)

Context & Data

A support desk generates digests from maintenance tickets with 4 sections: 1. Machine ID 2. Fault symptoms 3. Technician notes 4. Replacement parts For a specific output token, the attention weights over the four sections are:

$$0.10, \quad 0.25, \quad 0.15, \quad 0.50$$

Question 6

What is the role of the attention weights in this decoder step? - A) They replace all encoder states with fixed zero vectors. - B) They set the number of output classes for the digest. - C) They specify how much each section contributes. - D) They sort the ticket sections into alphabetical order.

Correct Answer: C

- **Detailed Step-by-Step Explanation**: Attention weights act as a convex combination weighting: they define the exact percentage contribution of each source section to the synthesized context vector.

Question 7

Why is softmax applied to the alignment scores? - A) To convert scores into normalized nonnegative weights. - B) To remove the decoder hidden state from every step. - C) To force the largest raw score to become exactly one. - D) To make all ticket sections contribute with equal weights.

Correct Answer: A

- **Detailed Step-by-Step Explanation**: The softmax function exponentiates and normalizes raw, unbounded alignment scores into non-negative values that strictly sum to 1.0 ($0.10 + 0.25 + 0.15 + 0.50 = 1.00$).

Question 8

Which ticket section contributes most to this output token? - A) Section 1 - B) Section 2 - C) Section 3 - D) Section 4

Correct Answer: D

- **Detailed Step-by-Step Explanation**: Comparing the weights $0.10, 0.25, 0.15, 0.50$, Section 4 has the largest weight (0.50 or 50%).

Question 9

How is the context vector computed after these weights are available? - A) By choosing the first encoder state only - B) By forming a weighted sum of encoder states - C) By averaging decoder outputs from all steps - D) By discarding the highest-weight section

Correct Answer: B

- **Detailed Step-by-Step Explanation**: The context vector is the linear weighted sum: $\mathbf{c}*t* = |*sum*_4$.}^4 \alpha_i \mathbf{h}_i = 0.10 \mathbf{h}_1 + 0.25 \mathbf{h}_2 + 0.15 \mathbf{h}_3 + 0.50 \mathbf{h

---

Question 10

If the support team expects the replacement-part section to drive this token, what attention pattern would support that interpretation? - A) All four sections receiving negative attention weights - B) The decoder state being removed from the scoring step - C) The replacement-part section receiving the largest weight -

D) Every ticket section receiving exactly the same weight

Correct Answer: C

- **Detailed Step-by-Step Explanation**: If the replacement-part section receives the highest attention weight (as Section 4 does with 0.50), it mathematically dominates the context vector, confirming its primary influence.

Case Study 3: Procurement Rule Checking (Questions 11 to 15)

Context & Data

A model reviews tender documents using **self-attention** with Query (Q), Key (K), and Value (V) vectors, scaled by $\sqrt{d_k}$, followed by softmax and positional encodings.

Question 11

What advantage does self-attention give for checking references across distant clauses? - A) It forces every token to use only its previous neighbor. - B) It removes all long-distance comparisons. - C) It replaces the document with a single class label. - D) It lets tokens compare directly with other tokens.

Correct Answer: D

- **Detailed Step-by-Step Explanation**: In self-attention, every token computes an inner product directly with every other token in $O(1)$ path length, allowing instantaneous reference checking across arbitrary document distances without recurrent decay.

Question 12

In the query-key comparison, what do the resulting scores represent? - A) The final number of tender documents reviewed - B) Relevance between one token and another token - C) The optimizer's learning-rate value during training - D) The length of the output vocabulary for labels

Correct Answer: B

- **Detailed Step-by-Step Explanation**: The query-key dot product $\mathbf{q}_i^\top \mathbf{k}_j$ measures the alignment and semantic relevance between the query of token i and the key of token j.

Question 13

What is the purpose of scaling the dot product by $\sqrt{d_k}$? - A) To keep score magnitudes controlled before softmax - B) To erase the effect of all value vectors in the layer - C) To make positional encodings unnecessary for ordering - D) To force all attention scores to become identical

Correct Answer: A

- **Detailed Step-by-Step Explanation**: Dividing by $\sqrt{d_k}$ counteracts the variance growth of high-dimensional dot products, preventing the softmax function from saturating into regions with vanishingly small gradients.

Question 14

What role do Value vectors play after attention weights are computed? - A) They choose the batch size for training runs. - B) They remove token-order information from clauses. - C) They provide the content that gets combined. - D) They store only the document identifier value.

Correct Answer: C

- **Detailed Step-by-Step Explanation**: Once attention weights α_{ij} are derived from query-key matching, they are multiplied by the Value vectors $\mathbf{v}_j$ $to compute the weighted output representation: $\sum_j \alpha_j$.} \mathbf{v

---

Question 15

Why are positional encodings needed in this tender-review model? - A) They delete repeated words from the document. - B) They replace softmax in attention scoring. - C) They make every token attend to itself only. - D) They supply information about token order.

Correct Answer: D

- **Detailed Step-by-Step Explanation**: Self-attention is fundamentally order-invariant. Positional encodings provide distinct mathematical coordinates for word order, allowing the model to distinguish antecedent clauses from subsequent clauses.

Case Study 4: Campus Feedback Review (Questions 16 to 20)

Context & Data

A hierarchical model processes student feedback: - Words form sentences (Word-level attention). - Sentences form reports (Sentence-level attention). - Multi-head attention tracks diverse feedback categories.

Question 16

Why is a two-level representation natural for these reports? - A) Reports contain only a single unordered word list. - B) Words form sentences, and sentences form reports. - C) The model must ignore sentence boundaries. - D) Every paragraph has exactly one token.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Text documents naturally follow a hierarchical compositional structure: characters compose words, words compose sentences, and sentences compose documents.

Question 17

What does word-level attention help identify? - A) The number of reports in the dataset split - B) The final optimizer update rule for training - C) The batch size used for report classification - D) Important words or phrases inside a sentence

Correct Answer: D

- **Detailed Step-by-Step Explanation**: Word-level attention highlights salient words (e.g. "hazardous", "defective") within a sentence when constructing the sentence embedding.

Question 18

What does sentence-level attention help identify? - A) Sentences that are most useful for the report decision - B) Characters that should be removed from each word - C) Output tokens generated by a decoder layer - D) The number of attention heads in the layer

Correct Answer: A

- **Detailed Step-by-Step Explanation**: Sentence-level attention assigns weights to individual sentences, prioritizing the most informative sentences when constructing the overall document summary.

Question 19

Why can multiple attention heads be useful in this feedback model? - A) They force all heads to learn identical attention weights. - B) They delete word-level and sentence-level evidence. - C) They allow different evidence patterns to be tracked. - D) They make positional information impossible to use.

Correct Answer: C

- **Detailed Step-by-Step Explanation**: Multi-head attention enables distinct heads to independently focus on different semantic concerns simultaneously (e.g. one head attends to safety keywords, another to equipment complaints).

Question 20

If sentence attention is almost uniform across a long report, what concern should the team consider? - A) The report contains no sentence representations. - B) The model may not be selecting the most informative sentences. - C) Multi-head attention cannot operate on textual inputs. - D) Word-level attention must always be zero for all words.

Correct Answer: B

- **Detailed Step-by-Step Explanation**: Uniform attention weights ($1/N$ for all sentences) imply an inability to discriminate key sentences from routine sentences, acting as a simple unweighted average.

---

5. NPTEL Exam Traps & High-Yield Pitfalls

1. **Context Vector Computation**:
2. The context vector is a **linear combination of encoder states** $\mathbf{c}*t = \sum_i \alpha_i$, NOT decoder states!} \mathbf{h
3. **Softmax Normalization Dimension**:
4. In attention, softmax is computed across the **source sequence length** T_x, ensuring the weights over source tokens sum to 1.
5. **Scaling Factor in Self-Attention**:
6. Remember the formula $\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$.

7. The scaling denominator is the **square root of key dimension** $\sqrt{d_k}$, not d_k and not sequence length N.